



# Exploration et visualisation d'espaces du calcul cellulaire

Thierry Delamare

Université de Paris-8

Département d'Informatique,

2, rue de la Liberté 93526 SAINT-DENIS CEDEX 02

Tél: (1) 49 40 64 04, email: [thy@ai.univ-paris8.fr](mailto:thy@ai.univ-paris8.fr)

Mémoire de DEA d'intelligence artificielle

Université Paris-8 et Paris-13

27 septembre 1994

Direction: Patrick Greussay

# Résumé

Nous proposons des méthodes et des outils d'exploration et de visualisation d'espaces de règles de grande dimension pour des automates cellulaires. Ces objectifs nous ont amené à réaliser des expériences de calcul parallèle cellulaire et à concevoir des architectures logicielles bien adaptées qui nous ont permis de mener à bien nos expériences sur des machines entièrement séquentielles.

Nous avons réalisé des moteurs cellulaires rapides basés sur une représentation des lois par la lookup-table résultante d'un morphisme et sur une collecte pipeline du voisinage cellulaire. Afin de démontrer la validité de cette approche nous avons également réalisé une implémentation d'un processeur cellulaire SIMD particulier sous forme du morphisme associé à un prédécodage des instructions pour une exploitation optimale de la lookup-table; nous proposons en outre une méthode de transformation d'une lookup-table quelconque en séquence de contrôle pour cette machine. Nous avons expérimenté des modes d'analyse et de visualisation spécifiques, en particulier des *réductions différentielles* et des *projections relativistes*. Nous avons tourné ces outils vers la production de trajectoires pour des règles des *familles* Anneal, Life, Qmean, Tube-Worm, Fox-Rabbit, et testé les limites de calcul accessible en réalisant des mesures plus complètes sur la règle M46789. En vue de la généralisation de ces expériences de calcul cellulaire à la visualisation de la structure de l'espace des règles, nous proposons un nouveau paramètre de contrôle pour l'échantillonnage des espaces de règles, le *paramètre d'enchâssement*, défini pour chaque élément de voisinage du réseau d'interconnexion associé à une règle, comme l'influence de ce voisin sur la règle.

Nous proposons de considérer la programmation cellulaire par tables comme une méthode d'exploitation des ressources disponibles sur les machines SISD modernes (50 Mips, 32 Mégas), sans préjudice de l'usage du calcul cellulaire sur des architectures cellulaires matérielles. Nous tentons de définir les objets élémentaires permettant de spécifier des objets cellulaires arbitrairement complexes constituant des expériences de calcul cellulaire. Nous considérons d'une part l'existence de dualités épistémologiques ou empiriques organisant les objets cellulaires selon des axes synthétique-analytique, temps-espace, algorithme-heuristique, séquentiel-parallèle. Nous considérons d'autre part la plasticité de l'approche par table, qui nous permet de projeter les objets du calcul cellulaire sur ces différents axes.

Dans le chapitre 2 nous présentons la structure des objets de notre étude: l'espace, le temps et les lois, ainsi que les méthodes applicables sur ces objets: édition, réduction, transformation, visualisation pour l'espace et le temps, ainsi que construction, altération, composition, mélange et séparation pour les lois. Nous présentons également dans ce chapitre l'architecture de nos moteurs cellulaires.

Dans le chapitre 3 nous présentons les outils conceptuels utilisés pour la quantification de l'espace des règles, et nous proposons un nouveau paramètre de contrôle pour l'exploration et la structuration de cet espace: le paramètre d'enchâssement.

Le chapitre 4 est consacré à une étude expérimentale de la règle **Anneal** basée sur l'exploitation de nos outils. Les expériences réalisées nous amènent à proposer l'existence d'une relative invariance spatio-temporelle des trajectoires associées à une transition de phase dans l'espace des configurations initiales.

La conclusion introduit les limites de notre approche qui à associé un constructivisme bottom-up à une *gestion de tables généralisée*. Nous évoquons pour finir une approche alternative de réalisation cellulaire purement algorithmique.

# Table des matières

|          |                                                                                                     |           |
|----------|-----------------------------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                                                 | <b>1</b>  |
| 1.1      | Nos objectifs et contraintes pour un environnement de développement et d'observation d'AC . . . . . | 3         |
| 1.2      | Les outils existants: pas de standard . . . . .                                                     | 6         |
| 1.3      | Un nouvel environnement: exploration par le calcul et la visualisation . . . . .                    | 7         |
| 1.4      | Pourquoi notre nouvel environnement? . . . . .                                                      | 11        |
| 1.5      | Des AC pour l'optimisation d'usage de ressources . . . . .                                          | 16        |
| 1.6      | Des AC comme outils de réflexion morphogénétique . . . . .                                          | 22        |
| 1.7      | Des AC comme outils de réflexion sur l'espace, le temps et le calcul . . . . .                      | 24        |
| 1.8      | Plan de lecture . . . . .                                                                           | 29        |
| <b>2</b> | <b>Vers des objets cellulaires: espace, temps, lois</b>                                             | <b>33</b> |
| 2.1      | L'espace: la mémoire . . . . .                                                                      | 34        |
| 2.1.1    | Structure de l'espace: dimension et voisinage . . . . .                                             | 35        |
| 2.1.2    | Édition d'espace: les configurations cellulaires . . . . .                                          | 36        |
| 2.1.3    | Réduction d'espace: taille et contenu . . . . .                                                     | 37        |
| 2.1.4    | Transformation d'espace: d'une forme à l'autre . . . . .                                            | 38        |
| 2.1.5    | Visualisation d'espace: comprendre les lois . . . . .                                               | 39        |
| 2.1.6    | Ressources associées à l'espace: des limites à choisir . . . . .                                    | 46        |
| 2.2      | Le temps: les cycles . . . . .                                                                      | 49        |
| 2.2.1    | Structure du temps: un axe d'organisation privilégié . . . . .                                      | 49        |
| 2.2.2    | Édition du temps: une métaphore cinématographique . . . . .                                         | 50        |
| 2.2.3    | Réductions du temps: sommes et différences . . . . .                                                | 52        |
| 2.2.4    | Transformations du temps: l'espace-temps . . . . .                                                  | 59        |
| 2.2.5    | Visualisation du temps: séquences de réductions et projections de plongements . . . . .             | 59        |

---

---

|          |                                                                                                                   |            |
|----------|-------------------------------------------------------------------------------------------------------------------|------------|
| 2.3      | Les lois: la fonction de transition . . . . .                                                                     | 61         |
| 2.3.1    | Structure des lois: voisinage et lookup-table . . . . .                                                           | 61         |
| 2.3.2    | Opérations: plasticité des tables de transitions . . . . .                                                        | 63         |
| 2.3.3    | Composition: les chemins de données . . . . .                                                                     | 65         |
| 2.3.4    | Voisinages spéciaux: nécessaire compromis ou généralisation? . . . . .                                            | 66         |
| 2.3.5    | Utilisation: construction, composition, mélange et séparation . . . . .                                           | 71         |
| 2.3.6    | Des machines virtuelles: compilation d'architectures SIMD pour une architecture cellulaire presque pure . . . . . | 77         |
| 2.3.7    | Et leur compilateur: la décompilation des lois pour des architectures SIMD . . . . .                              | 80         |
| 2.4      | Les séquenceurs . . . . .                                                                                         | 82         |
| <b>3</b> | <b>Structure de l'espace des automates cellulaires</b>                                                            | <b>88</b>  |
| 3.1      | Une version minimale du problème . . . . .                                                                        | 89         |
| 3.2      | La classification de Wolfram . . . . .                                                                            | 89         |
| 3.3      | Le paramètre $\lambda$ de Langton . . . . .                                                                       | 91         |
| 3.4      | Expérimentations sur le paramétrage des classes de comportements . . . . .                                        | 93         |
| 3.5      | Proposition pour une recherche d'un paramétrage de l'enchâssement . . . . .                                       | 96         |
| <b>4</b> | <b>Quelques remarques sur la règle Anneal</b>                                                                     | <b>99</b>  |
| 4.1      | Trajectoires et espaces de trajectoires . . . . .                                                                 | 99         |
| 4.2      | Variété des objets cellulaires . . . . .                                                                          | 101        |
| 4.3      | Quelques résultats expérimentaux à propos de Anneal . . . . .                                                     | 104        |
| <b>5</b> | <b>Conclusion</b>                                                                                                 | <b>110</b> |
| <b>A</b> | <b>Les fonctions de transitions utilisées</b>                                                                     | <b>119</b> |
| <b>B</b> | <b>Prédateur/proie</b>                                                                                            | <b>122</b> |
| <b>C</b> | <b>Exemple de code Tcl top-level</b>                                                                              | <b>128</b> |
| <b>D</b> | <b>Outils logiciels</b>                                                                                           | <b>131</b> |

---

# Chapitre 1

## Introduction

Les automates cellulaires (AC)<sup>1</sup> sont des systèmes dynamiques virtuels, discrets en espace, en temps et en états, homogènes, synchrones et déterministes. L'espace est constitué de cellules identiques dotées d'un réseau d'interconnexion régulier (une grille de dimension 2 ou 3 par exemple). Une cellule est une variable discrète (une chaîne de bits dont la taille détermine le nombre d'états de la cellule). Le temps est une séquence d'espaces générée par l'application synchrone d'une fonction de transition déterministe. La fonction de transition est locale dans le temps et dans l'espace, son argument est un ensemble fini d'états de cellules voisines du réseau d'interconnexion, pour un nombre fini de points antérieurs.

Les AC sont sujets d'études ou d'applications dans les domaines des mathématiques (Jen, 1990; Vichniac, 1990), de l'architecture des ordinateurs (Legendi, 1987; Zsoter, 1987), de la programmation parallèle (Preston et Duff, 1984; Almasi et al., 1987) et de la simulation de systèmes naturels tant physiques<sup>2</sup> (Margolus, 1984; Toffoli, 1984; Rossi, 1993) que biologiques<sup>3</sup> (Tamayo et Hartman, 1989; de Boer et al., 1993). Ils constituent un champ d'études des mathématiques discrètes et un modèle important en théorie du

---

<sup>1</sup>Pour des collections thématiques voir Demongeot et al. (1985); Wolfram (1986); Legendi et al. (1987); Gutowitz (1990), pour des ouvrages thématiques voir Preston et Duff (1984); Toffoli et Margolus (1987); Jackson (1991), pour une approche didactique voir Casti (1989); Schroeder (1990).

<sup>2</sup>En particulier les *Lattice Gas Automata* (Despain et al., 1990; Margolus et Toffoli, 1990).

<sup>3</sup>L'étude de la vie artificielle inspire de nombreux travaux sur les AC, souvent assez éloignés d'observations biologiques. Une exception notable étant Hameroff et al. (1989), qui propose l'existence d'une capacité au calcul cellulaire pour le cytosquelette.

---

calcul<sup>4</sup>, en particulier pour les problèmes liés au parallélisme<sup>5</sup>; leur structure est très proche de l'architecture des machines SIMD<sup>6</sup>; indépendamment de l'architecture des machines, de nombreux algorithmes se prêtent à une formulation cellulaire (Uhr, 1987, 1984), en particulier les algorithmes de traitement d'images (Preston et Duff, 1984; Hasselbring, 1992); enfin, les AC peuvent être vus comme une version discrète de systèmes d'équations différentielles<sup>7</sup>.

---

<sup>4</sup>Voir par exemple Lindgren et Nordahl (1990) pour une équivalence entre certains AC 1D et une machine de Turing.

<sup>5</sup>Les AC mathématiques mettent en jeu un espace infini, en particulier à limites périodiques. Notre point de vue est également empirique et implémentatoire: nous vivrons spécifiquement la réalisation de programmes dotés du meilleur rendement. En conséquence, dans un contexte de ressources mémoire limitées, nos AC seront évidemment dotés d'un espace restreint, par défaut torique.

<sup>6</sup>Greussay (1985) indique que la structure des AC est très proche de l'architecture des machines SIMD, et nous pensons que le progrès des connaissances et des outils dans ces deux domaines sont intimement liés. Les travaux de Rozin (1987, 1988, 1993) sur la programmation plane et de Signorini (1992, 1994) sur la programmation par configurations cellulaires témoignent de la fertilité de cette approche. La puissance de calcul et la taille de la mémoire disponible sur des machines aujourd'hui largement accessibles permet d'envisager la réalisation de multiples expériences dans le domaine du parallélisme synchrone massif, sans attendre l'arrivée de machines SIMD. Les AC fournissent un outil conceptuel en mesure de jouer un rôle central dans la recherche d'objets naturels (comportement émergent) ou artificiels (comportement construit), pouvant servir de base à la définition d'outils pratiques d'exploitation des ressources du parallélisme synchrone massif.

<sup>7</sup>Et à ce titre permettent la modélisation de nombreux systèmes naturels. La modélisation d'un système naturel par un ensemble d'équations différentielles établit des relations différentielles entre les variables macroscopiques du système (des populations par exemple). La modélisation par AC établit des relations entre les composants discrets élémentaires (les cellules) du système. Les variables macroscopiques deviennent des propriétés émergentes du système dont le choix peut être fait a posteriori, mais le modèle doit impérativement reposer sur une description en termes de comportement collectif du phénomène étudié.

La description d'un système naturel en termes de comportement collectif n'implique pas l'identification d'une cellule à un individu. Par exemple, dans une simulation de dynamique des populations, une cellule peut être identifiée à une densité de population en un point d'espace. Néanmoins la validité de la modélisation (conformité de la dynamique des populations d'un AC à un système d'équations différentielles standard de référence par exemple) dépend toujours de la présence d'un nombre minimum de cellules dans la simulation.

## 1.1 Nos objectifs et contraintes pour un environnement de développement et d'observation d'AC

Nous cherchons à définir et à réaliser un environnement de développement et d'observation de ces systèmes. Une des motivations principales de ce travail réside dans le désir de voir (et montrer) les séquences animées générées par un AC. En effet s'il est aisé de construire des AC en vue de donner des instantanés photographiques d'états successifs, il est plus difficile de les visualiser en continu filmique. Bien qu'il existe des outils<sup>8</sup> de spécification d'AC et d'observation de leur dynamique, aucun de ceux dont nous connaissons l'existence ne répond pleinement aux contraintes que nous allons définir. Nous souhaitons également que cet outil permette immédiatement la recherche et la création de nouveaux AC, puis la constitution d'un catalogue d'AC connus. Enfin nous espérons qu'il serve à l'identification d'AC pouvant servir de primitives dans un futur langage basé sur la combinaison d'AC. Les qualités recherchées sont la rapidité, l'extensibilité, ainsi que la facilité de spécification et d'interaction<sup>9</sup>.

---

<sup>8</sup>Cellsim<sup>[1]</sup>, scamper<sup>[2]</sup>, CA LAB<sup>[3]</sup> (Rucker, 1989), xlife<sup>[4]</sup>, HODGE-C<sup>[5]</sup>, cellang<sup>[6]</sup>. Les numéros en indice supérieur entre crochets renvoient au répertoire des outils, donné en appendice.

<sup>9</sup>La *rapidité* peut être définie comme le rendement d'une implémentation pour une architecture donnée. Dans ce contexte, et pour une architecture cible séquentielle, la rapidité s'exprime comme le nombre d'instructions élémentaires de la machine cible utilisée pour simuler une instruction de la machine source. Cette définition est néanmoins trop générale, en effet, si nous proposons dans le cadre de ce travail des moteurs cellulaires à rendement constant (i.e. indépendant de l'espace initial et de la règle de transition), il est possible de concevoir des moteurs cellulaires à rendement variable: xlife<sup>[4]</sup> découpe l'espace cellulaire en blocs et permet le calcul sur des configurations cellulaires virtuelles de grandes tailles; Gosper (1984) décrit un algorithme basé sur une mémoire cache asynchrone des configurations cellulaires (le cache contient un treillis de morceaux d'espace-temps cellulaire).

L'extensibilité peut, dans notre contexte, se ramener à la conception d'un ensemble d'objets constituant à la fois la boîte à outils et les éléments de base de la construction des expériences cellulaires. La recherche de cette qualité peut entrer en conflit avec la recherche de la rapidité, par exemple les variations de paramètres sur les moteurs cellulaires à rendement constant que nous avons réalisés passent par une phase de macro-génération de code source.

## Le cadre système, matériel, et logiciel: les performances

Notre environnement de développement est Unix et X11, notre architecture cible est une machine standard de 10 à 100 mips et 8 à 64 *mégabytes*<sup>10</sup> de mémoire utile. Pour un AC bidimensionnel composé de  $256 \times 256$  cellules de 2 bits<sup>11</sup> avec un voisinage de Moore, le système doit être capable de calculer une dizaine d'états par seconde (10 Hz)<sup>12</sup>. Le coût lié à la visualisation est plus difficile à évaluer, mais la chaîne de traitement complète (production, extraction, visualisation) doit permettre l'observation d'un film à une fréquence supérieure à 1 Hz<sup>13</sup>. Nous avons ainsi réalisé un noyau d'objets permettant la construction d'expériences, sous la forme d'une librairie C,

<sup>10</sup>Nous emploierons indifféremment les termes octet et byte, ainsi que leurs dérivés: kilobyte, mégaoctet ou gigaoctet.

<sup>11</sup>Avec un voisinage de Moore (9 bits de voisinage), 2 bits d'états donnent une table de taille 512 kilobit ( $2^{2 \times 9} \times 2$ ), ce qui est très loin des limites que nous nous sommes fixés, mais le passage à trois bits de voisinage donne une table de taille 48 mégaoctets ( $\frac{2^{3 \times 9} \times 3}{8}$ ) qui se trouve à la limite des ressources mémoires considérées. Nous insistons cependant sur le fait que rien n'interdit en pratique l'usage de telles tables, même si la connaissance de la distribution des configurations locales devient nécessaire pour prédire les performances du système constitué du triplet *table* de transition, *configuration* initiale, *performance* intrinsèque du système de gestion mémoire de la machine supportant l'application.

<sup>12</sup>D'un point de vue *esthétique*, cette fréquence correspond à un minimum subjectif permettant de restituer l'illusion du mouvement continu (les images télévisuelles ont une fréquence de 25 Hz, les images cinématographiques modernes une fréquence de 24 Hz, les images cinématographiques originelles une fréquence de 14 à 20 Hz, le passage à des fréquences plus élevées est dicté par des considérations techniques liées aux besoins du parlant pour le cinéma et à la fréquence du secteur alternatif pour la télévision). Cette fréquence est donc élevée, car d'un point de vue *système* elle correspond (pour notre espace de référence  $256^2$ ) à un débit de 640 kilo opérations / cellules par seconde, ce qui pour une machine hypothétique à 20 mips, correspond à 32 instructions de la machine cible pour une instruction de la machine source (une mise à jour cellulaire).

<sup>13</sup>Eisenstein, Gance ou Godard par leur utilisation occasionnelle de l'alternance de plans à une telle fréquence illustrent le procédé esthétique.

Cette fréquence correspond par ailleurs à un impératif ergonomique subjectif concernant le *temps de réponse* d'une application, elle borne la taille des espaces accessibles à un mode de visualisation séquentielle lorsque la contrainte de l'illusion du mouvement est levée.

La perte de l'illusion du mouvement, n'est pas contradictoire à l'étude du mouvement. Le physiologiste Marey étudie le vol des oiseaux: son *fusil photographique* (1882) et son *Chronophotographe à pellicule* (1888) servent à l'analyse photographique du mouvement. Bien que considéré comme précurseur du cinématographe, il ne se préoccupe pas de restituer l'illusion du mouvement. En cinéma d'animation, la *pixillation* utilise sciemment la séquence contre l'illusion.

“L'animation n'est pas l'art des dessins qui bougent, mais l'art des mouvements dessinés. Ce qui se passe entre chaque image a beaucoup plus d'importance que ce que l'on voit sur chaque image.” N. McLAREN. .../...

interfacée à Tc1<sup>14</sup> (Ousterhout, 1990, 1991).

## Spécification: la description des expériences cellulaires

La spécification concerne les réseaux d'interconnexions, les fonctions de transitions, et le séquençement des opérations applicables sur l'espace. Les fonctions de transitions sont écrites en C, et sont dotées d'une convention d'interface prévue pour permettre une indépendance des paramètres à leur localisation dans la mémoire d'états d'une cellule et la réutilisation dans des fonctions de plus haut niveau. Les réseaux d'interconnexions sont définis et créés en Tc1 à partir des primitives de la librairie. Les espaces sont créés en Tc1. Le séquençement est défini par un programme Tc1.

---

En 1941 dans "L'évolution créatrice", Bergson défend l'idée que "le mécanisme de notre connaissance usuelle est de nature cinématographique" (Bergson, 1989, page 305). Pour lui "la forme n'est qu'un instantané pris sur une transition", seul est réel le changement continu de forme. Il place cette métaphore au cœur d'une réflexion historique qui remonte à Zénon d'Élée.

Dans son acharnement à défendre "L'antériorité ontologique du continu sur le discret", Thom (1992) va jusqu'à proposer une "version informatisée de la preuve cartésienne de l'existence de Dieu" déclarant contradictoire la finitude du nombre de neurones cérébraux et la pensée de l'infini. Sans prétendre trancher une question aussi débattue, nous pouvons néanmoins suggérer que l'étude des AC propose une approche à rebours de cette question, la recherche pratique de l'immanence phénoménologique du continu remplaçant les tentatives de démonstration de son antériorité ontologique.

Sur notre configuration de développement (un 486DX2 avec 32 mégaoctets de mémoire, le système d'exploitation Linux et X11 pour la visualisation) la fréquence typique pour une chaîne comprenant la génération sur un voisinage de Moore (toujours pour un espace de référence 256<sup>2</sup>) et l'extraction d'un plan mémoire (constitué d'un des deux bits d'état) dans un premier processus, pipé sur un processus dumpant les plans cellulaires vers le serveur X11 (tournant sur la même machine) est supérieure à 10 Hz. Le découplage des différentes opérations de la chaîne permet d'obtenir une fréquence de génération supérieure à 15 Hz (module de génération redirigé sur `/dev/null`) et une fréquence de visualisation monochrome (pour un seul plan de bits cellulaire) supérieure à 30 Hz, (mesuré via un fichier historique de 8 mégaoctets pour 1024 générations). Ces mesures s'entendent en temps écoulé réel, pour un système autrement au repos, les mesures de performance de visualisation ne constituent évidemment pas une référence à notre travail, mais elles permettent de considérer, au regard des mesures de performance du moteur cellulaire, que celui-ci atteint le débit plafond d'un moteur à rendement constant, i.e. la meilleure performance possible est toujours garantie dans le pire des cas (espace cellulaire et portion d'espace-temps saturé).

<sup>14</sup>Tool Command Language.

## Interaction: l'interface humaine

L'interaction se fait soit par l'intermédiaire d'un shell `Tcl`, soit par une interface X11 basé sur `Tk`.

## Visualisation: les outils graphiques

La visualisation se fait par l'intermédiaire de fenêtres X11 présentant des séquences d'espaces monochromes (plans de bits de l'espace cellulaire) ou couleur (tranches de bits de l'espace cellulaire). Les variables macroscopiques (éléments statistiques) sont visualisées par l'intermédiaire d'objets X11 spécialisés (plotters).

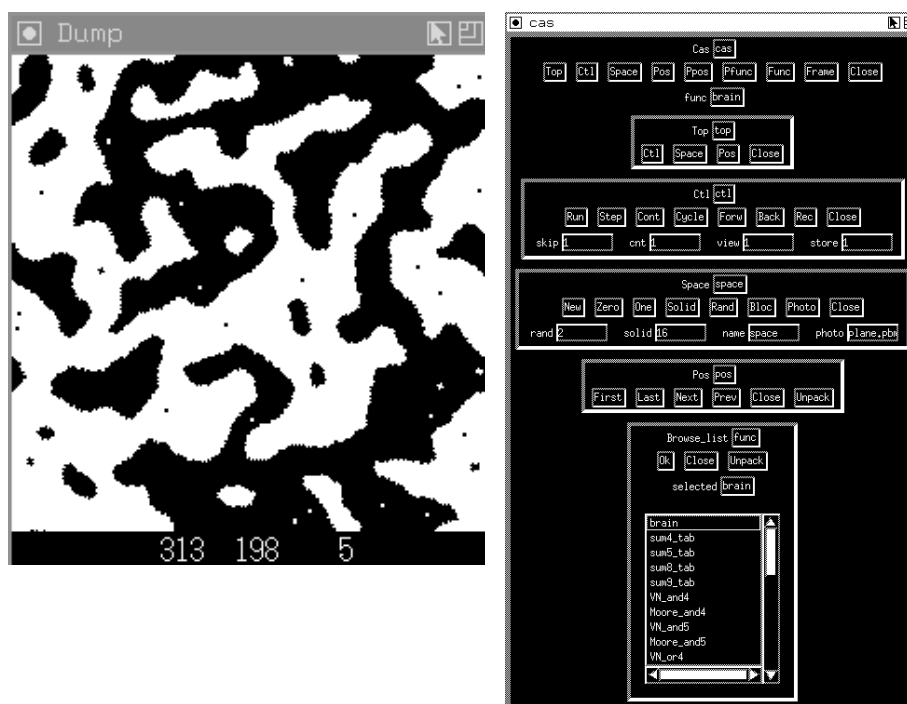
## 1.2 Les outils existants: pas de standard

Une recherche aussi exhaustive que possible des outils publics existants nous a convaincu de la nécessité d'une utilisation critique de certains de ces outils, en particulier dans le domaine de la visualisation, associé à une réalisation expérimentale ad hoc d'éléments d'architecture de calcul cellulaire.

Les outils de développement et de visualisation d'automates cellulaires sont peu nombreux<sup>15</sup>. Le plus connu d'entre eux est sans doute `cellsim`<sup>[1]</sup>; `scamper`<sup>[2]</sup> et `CA LAB`<sup>[3]</sup> sont également des outils disponibles depuis 1989. Il existe aussi des outils spécialisés pour un modèle d'automate cellulaire. `xlife`<sup>[4]</sup> fournit une réalisation d'exécution très rapide du jeu de la vie (Berlekamp et al., 1982), `HODGE-C`<sup>[5]</sup> une réalisation de la machine hodge-podge de Gerhard & Schuster (Dewdney, 1988), une classe d'AC inspirée par les réactions chimiques autocatalytiques, telle que la réaction de Belusov-Zhabotinski. Enfin, `cellang`<sup>[6]</sup>, d'inspiration plus architecturale, propose un langage de description d'AC (Eckart, 1992).

---

<sup>15</sup>Bien que les réalisations de `Life` soit innombrables, et les packages cellulaires plus ou moins généraux très nombreux, leurs réalisations minimales posant peu de problèmes, nous ne prenons en compte que les packages dont il existe des annonces explicites.



Une interface X11 réalisée avec Tk<sup>[19]</sup> (Ousterhout, 1990, 1991). Le dumper présente une image typique de la transformation d'un espace initial aléatoire de densité  $\frac{1}{2}$  par la règle Anneal (Vichniac, 1986; Toffoli et Margolus, 1987)<sup>a</sup> Le dumper présente également différents compteurs liés aux temps propre de la règle, du séquenceur, et au pool d'espaces de l'animation.

<sup>a</sup>Voir § A, page 120.

Figure 1.1: Une interface humaine expérimentale et son dumper

### 1.3 Un nouvel environnement: exploration par le calcul et la visualisation

Nous avons donc réalisé un nouvel environnement<sup>16</sup> de développement et de visualisation d'automates cellulaires.

Nous destinons ce workbench tant à l'exploration de la dynamique des automates cellulaires (AC), qu'à la construction et à l'évaluation d'architectures et d'algorithmes SIMD. De plus nous proposons de considérer la

<sup>16</sup>Ci-après désigné par le terme *workbench*.

réalisation d'algorithmes SIMD sur machines SISD non pas comme un palliatif à l'absence de machines SIMD, mais comme un mode d'utilisation particulier des ressources disponibles sur les machines SISD modernes (50 Mips, 32 mégaoctets).

La programmation des calculateurs massivement parallèles réels et virtuels pose le double problème du pouvoir d'expression et de l'efficacité d'un langage au regard d'une architecture. Ce problème se pose également pour les architectures SISD, mais l'on peut considérer qu'il concerne désormais l'adéquation des langages aux applications plutôt qu'aux architectures. Optant pour une approche résolument bottom-up de la recherche des primitives, nous proposons d'utiliser les AC comme composants élémentaires d'une combinatoire visant à la construction d'objets cellulaires / SIMD arbitrairement complexes. Ces objets se divisent en deux classes épistémologiques. Nous les nommerons objets *analytiques* et objets *synthétiques*. Les objets analytiques sont typiquement issus de l'application itérative d'une fonction simple sur un espace cellulaire initial aléatoire. Ils recoupent intuitivement le champ d'étude traditionnel des AC. Les objets synthétiques sont issus de la sélection d'une fonction localement universelle, et de son application contrôlée (globalement paramétrée) sur un espace initial soigneusement configuré. Ils correspondent aux architectures des machines SIMD. De l'étude des objets analytiques nous espérons tirer des enseignements permettant à terme de considérer les processus morphogénétiques comme des éléments de programmation. De la construction d'un outillage SIMD nous attendons les moyens de contrôle nécessaires à l'étude des objets analytiques. Cet objectif, certes ambitieux, repose précisément sur la réalisation par *lookup-tables* des fonctions de bas niveau manipulables et combinables au sein de l'environnement.

Notre étude est organisée autour de la présentation des structures de données choisies pour le workbench; ces structures de données représentent *l'espace*, *le temps*, et *les lois*: objets élémentaires du workbench. Nous proposons des exemples d'opérations applicables sur ces objets. Nous proposons également de considérer la réalisation de fonctions par lookup-tables comme une structure de données fondamentale, tant pour des raisons d'efficacité, que de généralité.

Nous considérons la visualisation comme un service primordial, qui propose l'usage du workbench en tant que laboratoire de manipulation d'espace cellulaire (l'espace, mais aussi les fonctions et les dynamiques). Nous

présentons plusieurs exemples de visualisations principalement associées à des objets analytiques. Nous montrons comment peut s'effectuer le passage d'une méthode naturaliste à une méthode artefactuelle en proposant une réalisation par lookup-table d'une architecture SIMD spécifique, et une méthode de transformation d'une lookup-table quelconque en séquence de contrôle pour cette machine.

Constatant l'absence d'outils adaptés aux contraintes de la visualisation des espace-temps cellulaires<sup>17</sup>, constatant également l'existence d'espace-temps cellulaires binaires a priori toriques, nous avons conçu, réalisé et testé une bibliothèque de manipulation d'espace<sup>18</sup> cellulaires aux fins de visualisation. Sur cette base, nous avons par exemple pu tester l'idée d'une visualisation relativiste des espace-temps: nous avons projeté le cône de la vitesse de la lumière<sup>19</sup> d'un espace-temps 2D sur un espace 2D, résumant ainsi sur un seul espace 2D la dynamique de la règle visualisée.

La figure 1.2 présente un des usages possible de notre librairie de manipulation d'espaces dans le contexte du workbench: nous y présentons la découverte d'un *glider superluminique*<sup>20</sup>, en déplacement à une vitesse apparente double de la vitesse de la lumière. Nous attendions évidemment de cette expérience des aberrations temporelles, des illusions ou mirages inexplicables dans le contexte d'une projection parallèle à l'axe du temps. Néanmoins les premiers résultats faisaient plutôt apparaître une relative invariance de la perception visuelle subjective associée à la règle. La procédure utilisée produit un déplacement apparent des configurations, mais les objets se comportent par ailleurs, c'est à dire relativement à l'espace apparent pour l'expérimentateur, de manière subluminique ou luminique usuelle. Ce glider, le seul que nous ayons trouvé (dans l'espace-temps englobant la portion présentée) était donc prévisible, puisqu'il correspond à un glider en mouvement corrélé au front de coupe, mais néanmoins totalement surprenant. Beaucoup moins prévisibles nous apparaissent les conséquences générales

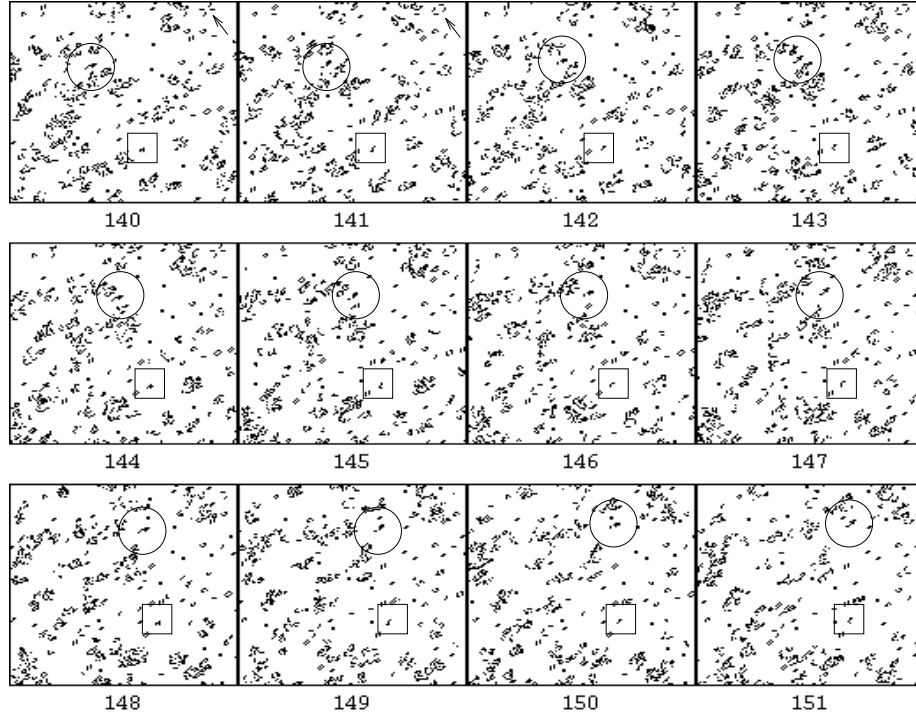
---

<sup>17</sup>La visualisation de volume de données dans un format inadapté conduit à la saturation inutile des capacités mémoire de la machine d'accueil d'un calcul cellulaire.

<sup>18</sup>Coupe, séquences de coupes sur des axes de projection variables, réduction intégrale sur un axe arbitraire pondéré par la distance à la surface de projection pour obtenir des effets de transparence.

<sup>19</sup>La vitesse de la lumière d'un espace-temps cellulaire est le rayon du voisinage de la famille de règle à laquelle appartient la règle génératrice de l'espace-temps.

<sup>20</sup> nous emploierons le terme glider (planeur) de manière générique, i.e. pour désigner des objets cellulaires périodiques, à un décalage spatial près.



Nous présentons dans cette figure la découverte d'un *glider superluminique*. Cette série d'espaces de configurations cellulaires est une séquence de tranches d'espace-temps non alignées sur l'axe du temps. Malgré la distorsion introduite par ce décalage, la règle génératrice de cette portion d'espace-temps est parfaitement identifiable: Life<sup>a</sup>. A première vue, la conséquence de l'utilisation d'un axe non standard pour la projection de la dynamique de Life est d'ordre purement spatial<sup>b</sup>, exception faite du décalage introduit par le désaxage temporel. Le carré sur les photos pointe vers un objet Life de type glider, doté d'une vitesse de déplacement subluminique. L'observation de l'espace-temps contenant la portion présentée au moyen de films à haute fréquence<sup>c</sup> nous a permis d'identifier à la vue un objet paradoxal, pointé par un cercle sur les photos: un glider superluminique.

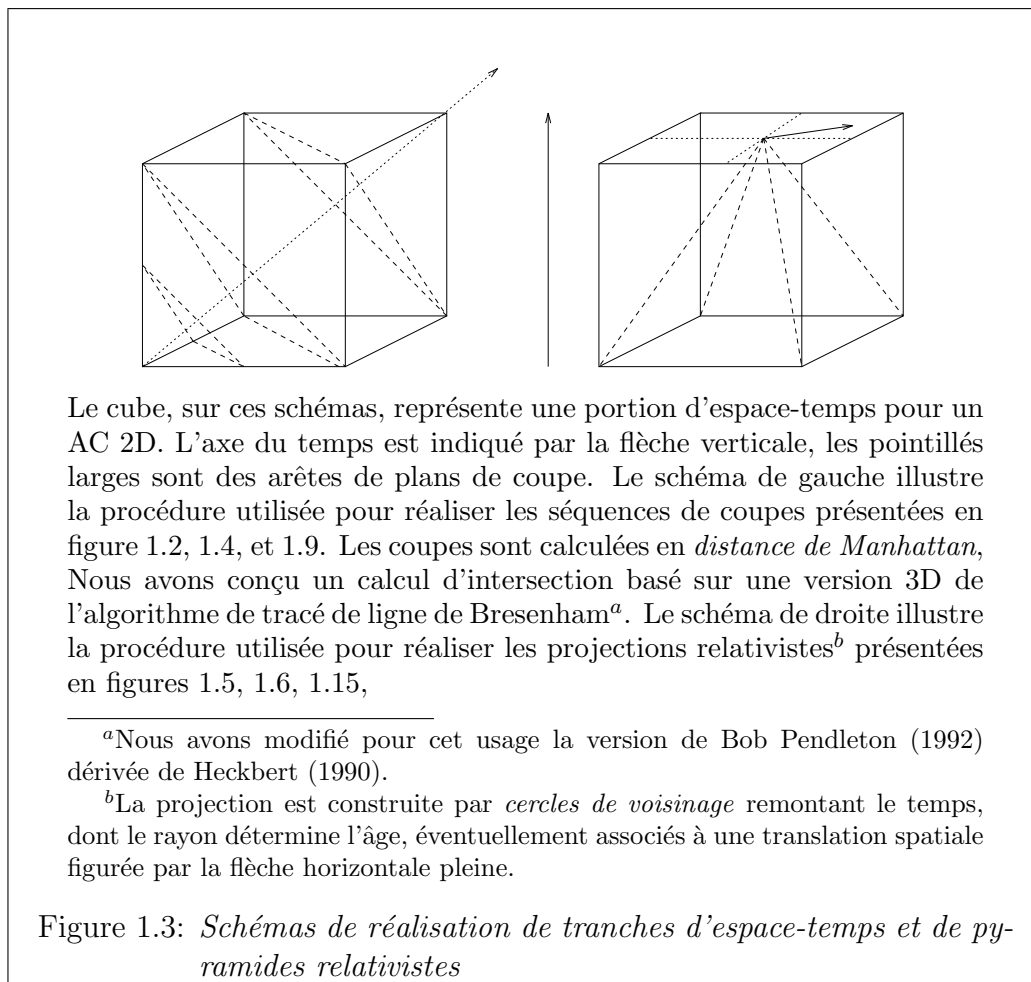
<sup>a</sup>Les flèches en haut à droite des deux premières photos pointent une configuration cyclique de life reconnaissables.

<sup>b</sup>Cette appréciation subjective n'est applicable qu'à l'expérience de la vision du *film*.

<sup>c</sup>Supérieure à la fréquence télévisuelle.

Figure 1.2: *Un glider subluminique et un glider superluminique*

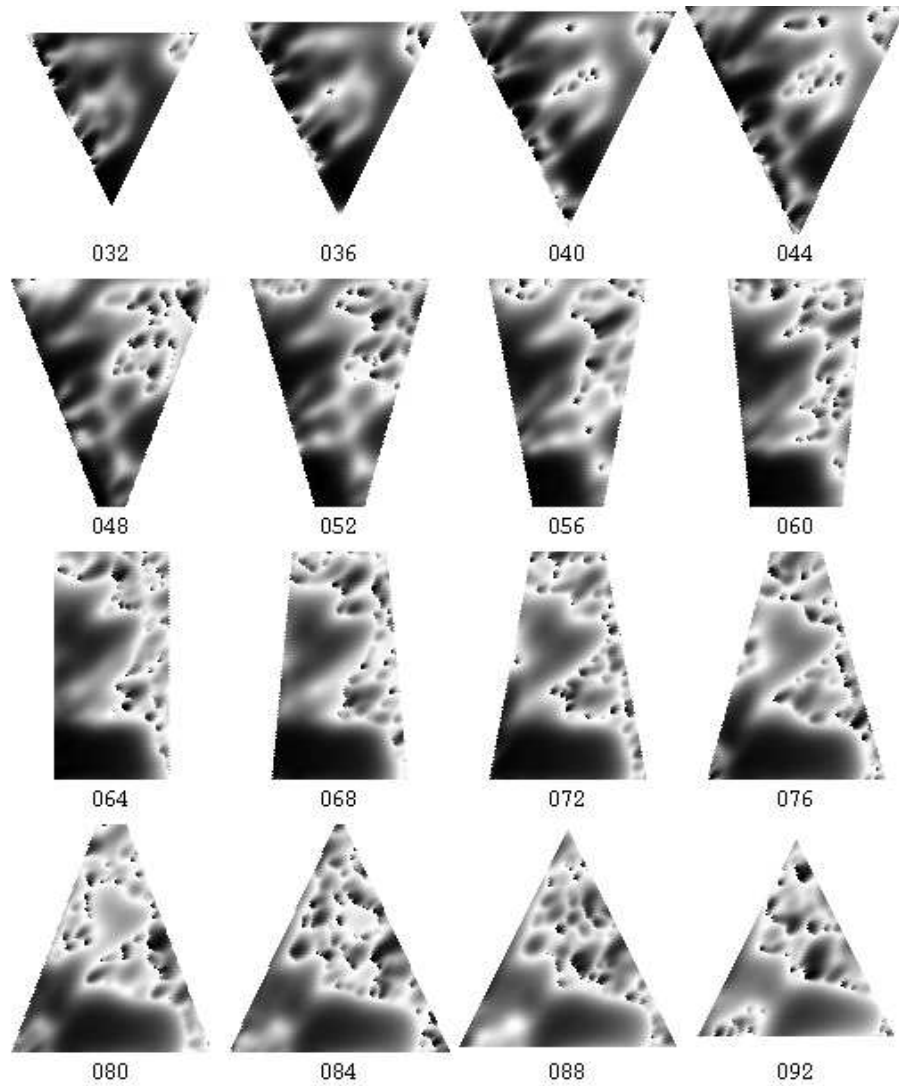
d'observations exploitant systématiquement des projections d'espace-temps non parallèles à l'axe du temps.



L'état actuel du développement du workbench n'a pas encore permis l'expérimentation de constructions complexes d'objets mixtes. La mesure de variables d'états macroscopiques, par exemple, utilise actuellement des outils externes, au lieu d'algorithmes SIMD adaptés à la nature discrète et finie des espaces cellulaires.

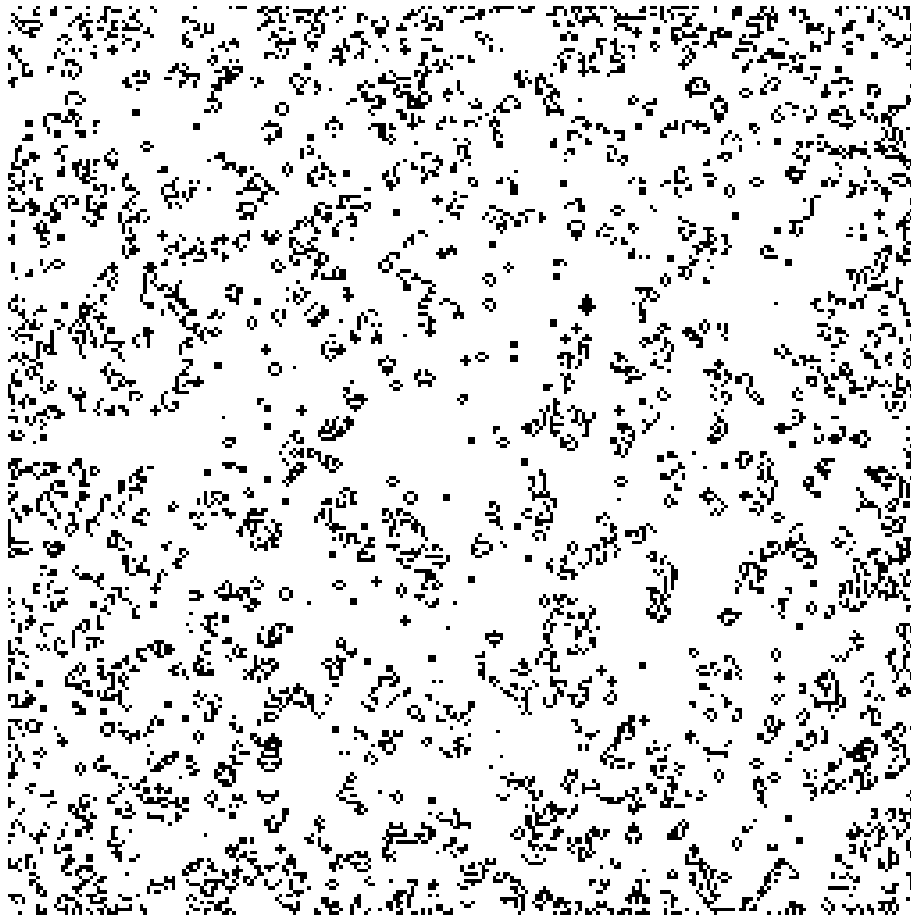
## 1.4 Pourquoi notre nouvel environnement?

Tout d'abord, même si l'histoire de l'étude des AC est presque aussi ancienne que l'histoire des ordinateurs (Metropolis, 1993), les architectures cellulaires



La règle  $Q_{mean}$  est présentée, ainsi que les autres règles utilisées pour la synthèse des images réalisées en § A. La figure 1.3 illustre la procédure de réalisation de cette séquence. D'autres visualisations associées à la règle  $Q_{mean}$  sont présentées par les figures 2.5, 2.6, 2.7 et 2.8.

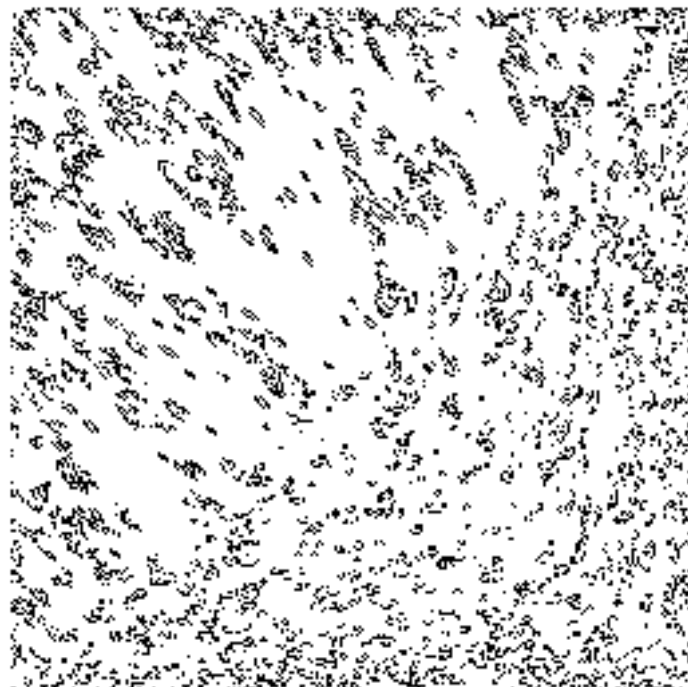
Figure 1.4: Une séquence de tranches d'espace-temps de la règle  $Q_{mean}$



Une projection relativiste (cf. figure 1.3, page 11) d'un espace-temps de life pour une cellule de référence fixe, réalisée sur un espace-temps  $256^3$ , pour une densité initiale  $\frac{1}{2}$ . La cellule de référence est au centre de la projection, celle-ci apparaît donc au sein d'un espace *âgé*, composé en plus grand nombre d'objets point-fixe et de densité moyenne plus faible que l'espace situé à proximité des bords de la projection, de densité moyenne plus élevée et peuplé d'objets plus jeunes<sup>a</sup>. Il est possible d'accentuer l'âge des objets lointains en diminuant la vitesse de la lumière utilisée lors de la projection. Un tel découplage du rayon de voisinage accroît le problème d'interprétation des projections produites.

<sup>a</sup>Que l'on peut cependant qualifier d'archaïques du point de vue de l'origine de la projection.

Figure 1.5: *Une projection relativiste pour une cellule fixe*



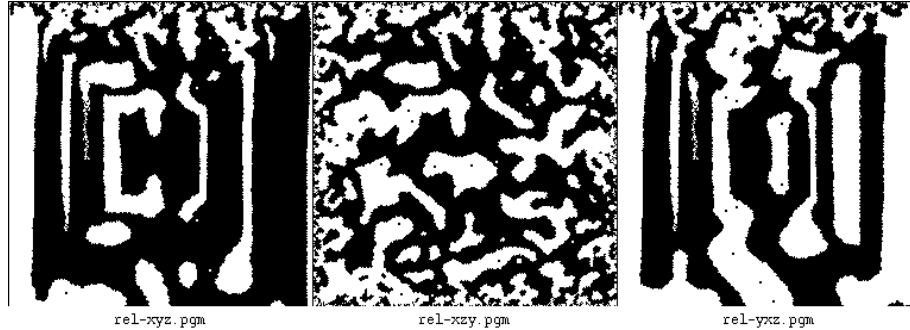
Cette projection diffère de la figure 1.5 par l'ajout d'une translation appliquée à la cellule de référence.

Figure 1.6: *Une projection relativiste à cellule mobile*

construites, bien que non encore industrialisées apparaissent comme les plus prometteuses<sup>21</sup>, et le problème des méthodes de constructions d'architectures et d'algorithmes cellulaires restent largement ouvert. De même, à propos du problème de l'auto-reproduction (von Neumann, 1966), la dualité opposant analytique à synthétique accompagne la naissance des AC<sup>22</sup>. Cette dualité, ou plutôt la capacité offerte par les AC de basculer d'un univers épistémologique à un autre tout en restant au sein d'un modèle unique, doit être incarnée dans l'environnement. Nous souhaitons donc un environnement permettant une approche mixte.

<sup>21</sup>Il est difficile de savoir si ce fait est à mettre au crédit des architectures cellulaires, nous nous bornerons à proposer l'idée que la perception du potentiel d'usage d'une architecture est inversement proportionnelle à sa réalisation industrielle.

<sup>22</sup>Voir Reggia et al. (1993) pour une approche permettant d'accorder plus d'autonomie aux phénomènes d'auto-reproduction. Nous considérons, dans ce contexte, l'autonomie comme la mesure d'une position sur l'axe qui va du construit à l'émergeant.



Cette figure regroupe trois projections relativistes de la règle Anneal réalisées sur une même trajectoire, pour l'axe temporel et les deux axes nord-sud et est-ouest. L'origine du temps est visible dans le haut de la projection pour ces deux axes.

Figure 1.7: *Projection relativiste sur les 3 axes d'un cube d'espace-temps*

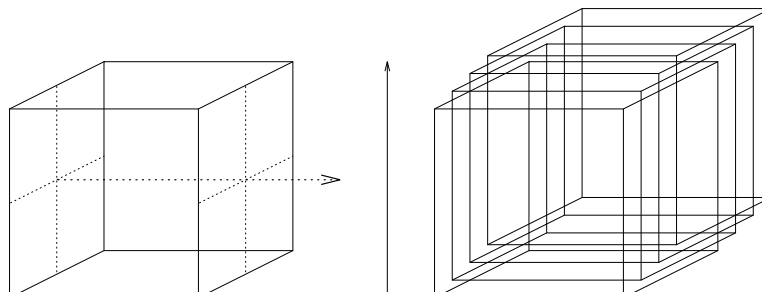
Par ailleurs, aucun des outils que nous avons cités ne répond totalement à un ensemble de contraintes plus pratiques que nous avons fixées comme cadre de développement.

- la nature publique<sup>23</sup> des outils
- la rapidité d'exécution
- la richesse de visualisation
- la souplesse de spécification
- un environnement d'exploitation Unix et X11
- la modularité des constituants en vue de la construction et du séquençement d'expériences cellulaires.

CA LAB n'est pas un outil public et n'est pas disponible dans l'environnement Unix. `Cellsim` de par sa dépendance à `SunView` n'est utilisable que dans un environnement Sun<sup>24</sup>, `scamper` présente une application intégrant un formalisme de spécification des règles basées par pattern, des compteurs

<sup>23</sup>Par *publique* nous entendons accessible sous forme source et utilisable sans licence commerciale.

<sup>24</sup>Felicity George a réalisé en 1992 un portage de `cellsim` pour l'environnement `XView`.



Comme pour la figure 1.3, les cubes représentent une portion d'espace-temps pour un AC 2D. La flèche pointillée illustre une traversée orthogonale non parallèle à l'axe du temps, lors d'une telle traversée on peut soit réaliser un film<sup>a</sup>, soit une réduction intégrale (une somme de la valeur de toutes les cellules rencontrées). Pour les réductions intégrales destinées à la production d'images, la valeur de la cellule est pondérée par la distance parcourue. La figure 1.12 présente une telle image. Pour les transparences non orthogonales, les effets de bord de la méthode utilisée (distance de Manhattan) posent un problème d'interprétation des déformations. Les quatre cubes à droite symbolisent l'opération de séparations d'un espace-temps de cellules à quatre bits en quatre espace-temps de cellules à un bit. La figure 1.11 présente des transparences sur trois axes pour cinq *plans* de bits de Qmean, La figure 1.14 pour l'unique plan d'un espace-temps de life.

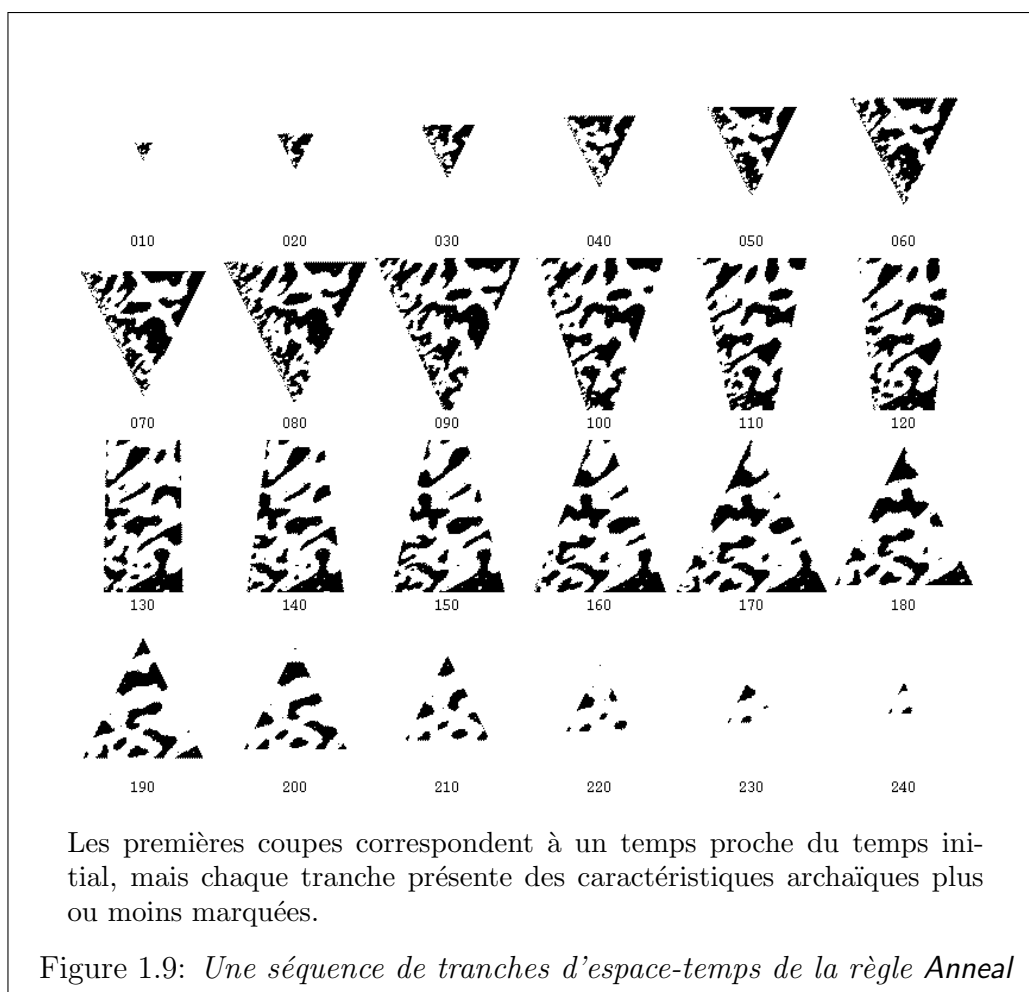
<sup>a</sup>A défaut de films, incompatible avec le support papier, nous présentons des séquences associées à une traversée non orthogonale (cf. figure 1.3, page 11).

Figure 1.8: *Schéma de réalisation de réductions par transparence*

de populations et une interface visuelle basée sur X11, mais il ne permet ni exécution rapide, ni séquençement. **Cellang** propose un langage de description d'AC, mais son compilateur ne produit pas un code permettant de tirer le meilleur parti des ressources disponibles.

## 1.5 Des AC pour l'optimisation d'usage de ressources

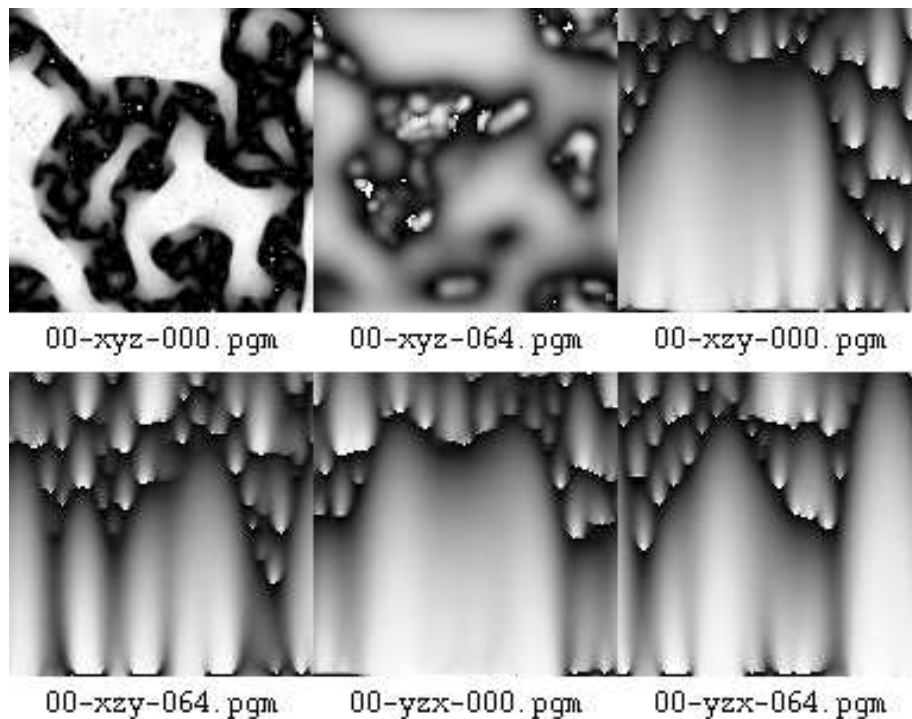
Concernant le parallélisme massif, les tendances de la précédente décennie en architecture des machines ne confirment pas les prédictions avancées à



son propos<sup>25</sup>. Néanmoins, hors considérations architecturales, les ressources accessibles continuent à croître à un rythme soutenu<sup>26</sup>. Hormis le calcul intensif classique, les gains de ressources semblent s'orienter vers le parallélisme MIMD massif pour la gestion et le support multimédia pour le poste de travail. Nous affirmons que les AC constituent un modèle efficace et universel

<sup>25</sup>Machines SIMD massives reposant sur des techniques de fabrication de type wafer scale intégration, associées à des algorithmes de reconfigurations de réseaux d'interconnexion permettant la résistance aux défauts; pyramides de telles architectures pour la vision et plus généralement l'IA, ou architectures diverses pour la physique, la biologie moléculaire, le pattern matching haut débit, la réalisation de film holographique temps réel.

<sup>26</sup>Au point qu'on pourrait mettre en doute l'application du principe des gaz parfaits à la consommation de ressources. Contre cette tendance, nous proposons de considérer qu'il n'est plus temps d'apprendre comment se contenter de ressources restreintes, mais comment tirer profit de ressources disponibles.



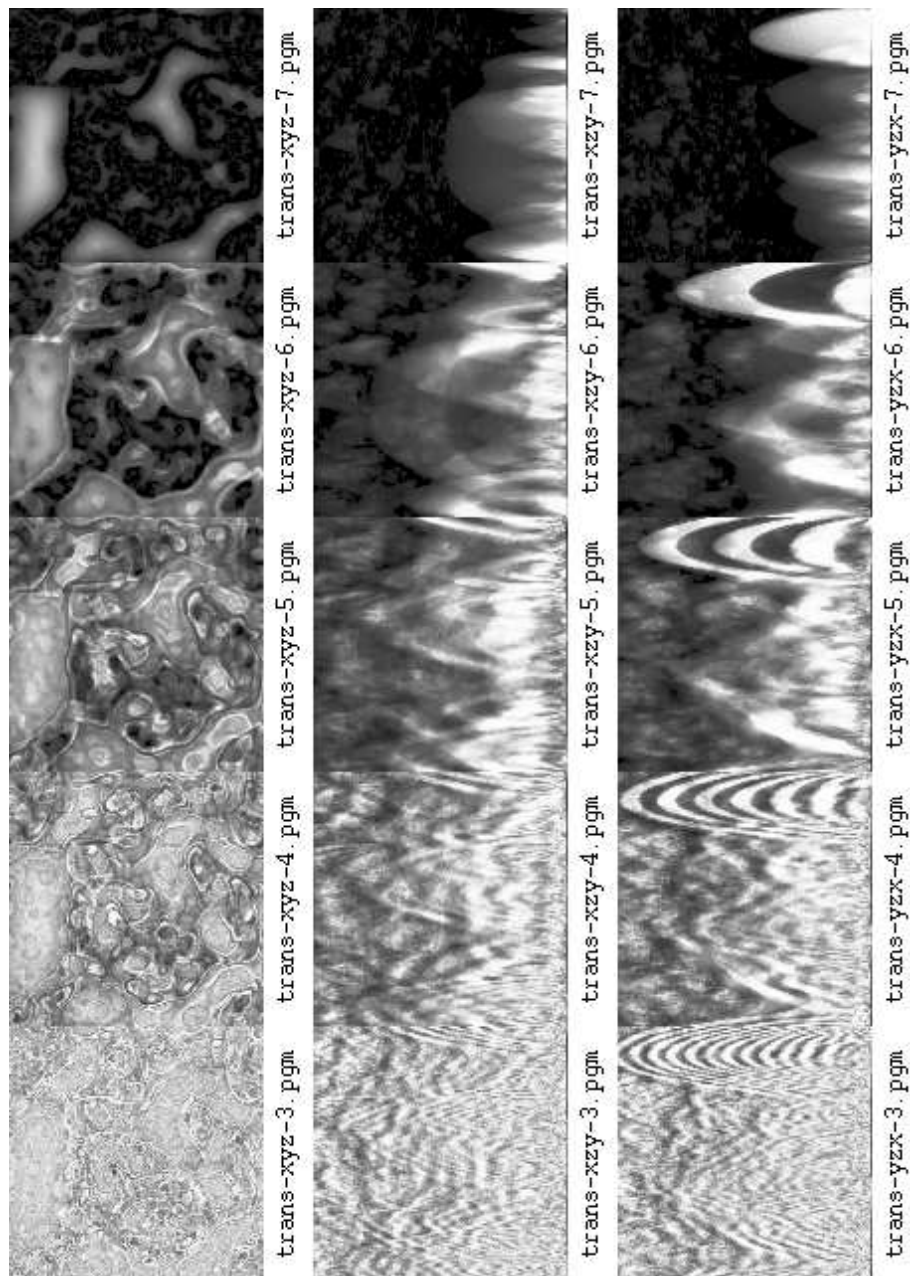
Cette figure présente les six faces d'un cube d'espace-temps de la règle *Qmean*. Les deux faces situées dans l'axe du temps, ainsi que leur âges sont clairement identifiables.

Figure 1.10: *Faces du cube d'espace-temps de Qmean*

pour l'utilisation de ces ressources, y compris par la prise en compte du parallélisme multi-calculateur<sup>27</sup> associé à l'interconnexion par *WAN*<sup>28</sup> de ceux-ci. La régularité structurelle des données et des traitements des AC associés à l'absence de problème algorithmique de synchronisation des éléments du calcul réparti, permet un calcul et une affectation de ressource simple. Une approche bottom-up de la spécification du comportement des éléments cellu-

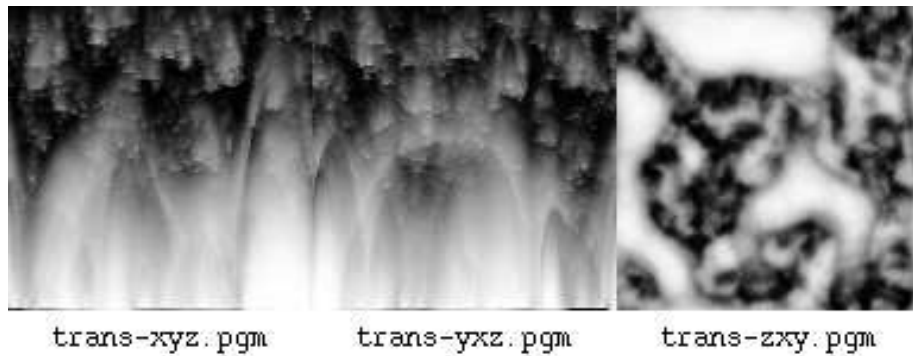
<sup>27</sup>C'est à dire l'utilisation d'un ensemble de machines autonomes (stations de travail ou autres, localement parallèles ou non) reliées par un réseau d'interconnexion (de type bus ou point à point) dotées d'une couche logicielle système ad hoc, comme une ressource parallèle en soi.

<sup>28</sup>Wide Area Network, il s'agit ici de remarquer que les débits disponibles ou annoncés pour les réseaux de télécommunication publics permettent d'envisager la construction virtuelle de machines cellulaires réparties sur une large zone géographique.



Les cinq bits de poids fort de la règle Qmean sont utilisés respectivement pour réaliser une transparence (cf. figure 1.8, page 16) sur les trois axes de l'espace-temps. Ces séquences sont à comparer à la figure 1.12 utilisant la totalité des bits de chaque cellule pour la transparence.

Figure 1.11: *Une transparence éclatée par plan de bits*



Cette transparence fournit une image plus habituelle d'une dynamique cellulaire considérée comme un voxmap<sup>a</sup>. Des expériences antérieures menées avec des packages de visualisation scientifique *classiques*<sup>b</sup> donnent pour ce genre de visualisation des résultats proches en terme d'images produites<sup>c</sup> de ceux présentés ici, mais l'usage de nos fonctions de manipulation d'espaces cellulaires est nécessaire<sup>d</sup> à la production des transparences par plan de bits de la figure 1.11. La comparaison des images produites par les méthodes plan de bits vs. voxel, suggère l'étude des relations entre les deux formulations, et une possible généralisation de la première à l'analyse de voxmaps.

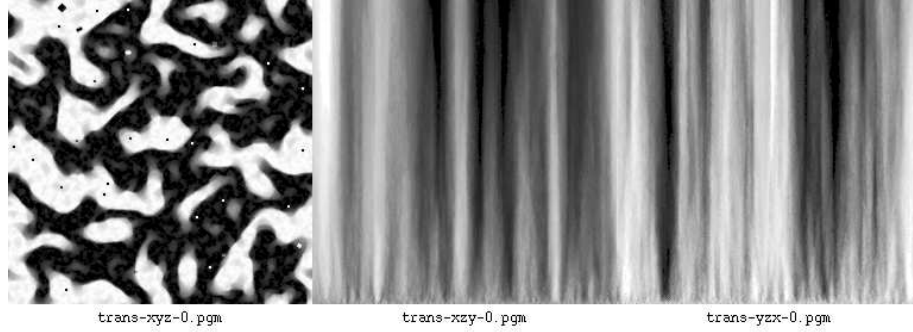
<sup>a</sup>Le voxel est la version 3D du pixel.

<sup>b</sup>Au sens où ils travaillent nécessairement en flottants, ou considèrent des données dont la localisation n'est pas implicite et en tout cas ne sont pas adaptés à un travail au niveau du bit. Nous avons recherché, porté et expérimenté les outils suivants: XDataSlice<sup>[20]</sup> et son format HDF<sup>[22]</sup>, VIS-5D<sup>[23]</sup> (Hibbard et al., 1994), viewit<sup>[24]</sup> et LIC<sup>[25]</sup> (Cabral et Leedom, 1993).

<sup>c</sup>Mais pas en termes de ressources consommées pour la production. Un autre inconvénient de ces packages, étant les erreurs introduites par la conversion flottante intermédiaire lorsque l'espace 2D de transparence produit est destiné à la réalisation de mesures statistiques, autant que d'images.

<sup>d</sup>D'un point de vue pratique, la conversion d'un bitmap tridimensionnel en voxmap de flottants, éventuellement reconverti en voxmap de bytes, est en effet parfaitement inadéquate.

Figure 1.12: Une transparence utilisant tout l'espace de valeur d'une cellule



La méthode de production commentée en figure 1.8 est utilisée pour la règle *Anneal*. Le nombre d'états de *Anneal* étant égal à deux, la méthode éclatée se confond avec la méthode standard.

Figure 1.13: *Une transparence par plan de bits pour Anneal*

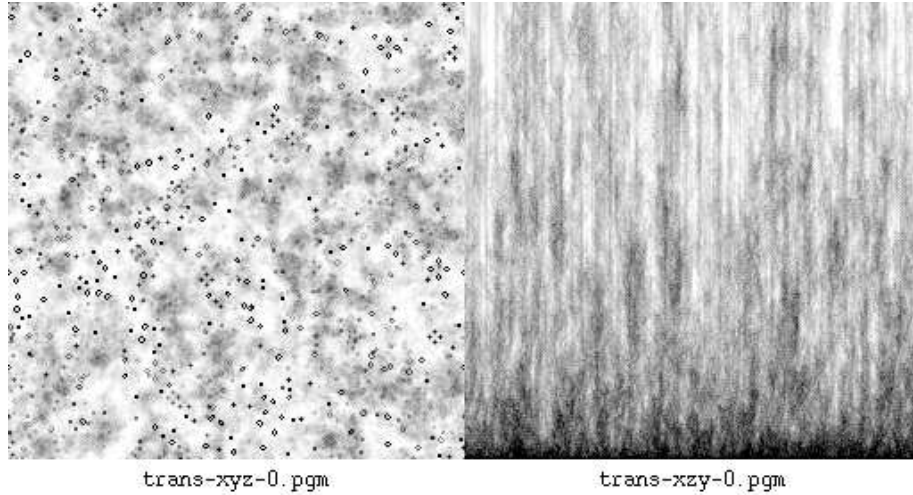
laire permet, de plus, de s'affranchir des problèmes d'architecture CPU, une fonction considérée sous l'angle de sa table de transition est indépendante des architectures de traitement<sup>29</sup>.

A propos de la répartition d'un calcul cellulaire sur réseau de stations de travail<sup>30</sup>, et concernant le problème de l'overhead associé au passage de messages implémentant la synchronisation des bords lorsque le calcul courant réclame un accès à une cellule voisine, indépendamment d'une mesure particulière des débits respectifs du couple cpu / mémoire ( $D_m$ ) et du canal de communication ( $D_c$ ), le rapport des tailles locales / distantes  $N^d / N^{d-1} = N$  garantit l'existence d'une valeur  $k$  de  $N$  telle que l'usage de plusieurs nœuds de calculs sera plus efficace que l'usage d'un seul d'entre eux:  $\frac{D_m}{D_c} k > 1$ .

A défaut de machines cellulaires industrielles, nous espérons pouvoir utiliser les DSP et les accélérateurs *bitblt* (Pike et al., 1984) aujourd'hui largement disponibles comme accélérateurs de nos moteurs cellulaires.

<sup>29</sup>A une permutation de l'ordre des bits sur le mot près, et toujours en considérant que l'unité de traitement réellement disponible reste fondamentalement une architecture de von Neumann dotée d'une mémoire adressable importante.

<sup>30</sup>Algorithme distribué avec recouvrement: calcul des bords, envoi des bords, calcul du centre, réception des bords.



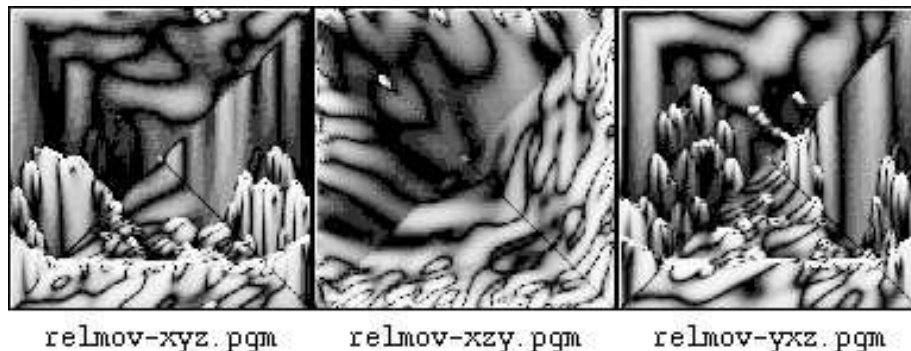
La méthode de production commentée en figure 1.8 est utilisée pour la règle Life

Figure 1.14: *Une transparence par plan de bits pour Life*

## 1.6 Des AC comme outils de réflexion morphogénétique

L'essence de la forme pose un problème, ne serait ce que par l'existence de la généricité du moule. Tout état du monde n'est pas une forme, toutefois une forme est par définition une configuration d'espace cellulaire prise parmi un ensemble fini de configurations. Cette définition réclame un critère de discrimination/réduction de l'ensemble des états. Une première possibilité réside dans l'utilisation d'une fonction arbitraire sur un état déterminant la valeur d'un attribut formel: esthétique, géométrique, statistique, arithmétique. Les formes sont figuratives, abstraites, séduisantes, rondes, carrées, symétriques, rares, uniques<sup>31</sup>, régulières, fractales, denses, colorées, leur numéro d'ordre enfin, peut être argument d'une fonction choi-

<sup>31</sup>Chaque configuration d'espace est évidemment unique, on pourrait néanmoins mesurer, intuitivement ou formellement, l'unicité à partir du produit de l'ensemble des états par l'ensemble des fonctions de qualification formelle disponibles. Les fonctions de qualification sont des fonctions de réduction de l'information (le nombre de bits rendu est inférieur au nombre de bits fournis), le cardinal de l'image d'un ensemble de formes est donc inférieur à celui de l'ensemble de formes, la distribution des redondances de valeurs induites par la différence de cardinalité déterminant les classes d'équivalence de cet attribut.



L'accumulation de transformations renforce le problème de l'interprétation des images produites. Ces figures illustrent les limites rencontrées en combinant une détection de contour une à projection relativiste à cellule de référence mobile, appliquée sur les trois axes de projection d'un cube d'espace-temps de la règle Qmean.

Figure 1.15: *Une détection de contour appliquée sur une projection relativiste*

sie au hasard. Cette méthode pose le problème de l'énumération des fonctions de réduction<sup>32</sup>. Une deuxième méthode consiste à opposer la généticité à la généricité, c'est à dire l'histoire de la forme au moule de la forme. Dans ce contexte la réduction du nombre d'espaces appelés à la forme est déterminée par l'existence d'une trajectoire dans l'espace des états. Cette trajectoire étant elle même déterminée par l'application itérée d'une fonction de l'ensemble des états vers l'ensemble des états. Les trajectoires ne sont pas moins nombreuses que les états, mais si l'on ajoute une contrainte de localité aux fonctions génératrices de trajectoires, une tentative d'énumération devient accessible. Cette contrainte de localité constitue le fondement d'une approche cellulaire de la forme. Cette contrainte introduit un lien entre la vitesse et la forme. Plus généralement, la notion de contrainte s'applique aisément au monde des formes dans une perspective esthétique, le libre choix des contraintes de production de l'objet déterminant la qualité artistique de

Une relation d'ordre pour l'unicité est alors définie par une fonction minimisant le cardinal de l'union des classes d'équivalence des images des formes.

<sup>32</sup>Ainsi que celui du choix à priori des états considérés. Les fonctions de qualification sont implicitement destinées à introduire une réduction du nombre des formes par l'introduction de classes d'équivalence sur l'ensemble des états.

l'œuvre, dont la forme peut être conçue comme une trace d'exécution<sup>33</sup>. De même la *forme* d'un espace cellulaire devient la trace d'un programme<sup>34</sup>.

## 1.7 Des AC comme outils de réflexion sur l'espace, le temps et le calcul

Nous avons déjà mentionné l'existence d'une dualité analytique-synthétique. D'autres projections de cette dualité sont possibles, sur les axes dynamique statique, heuristique algorithmique, parallèle séquentiel ou temporel spatial. La trajectoire d'un AC sur un espace de configuration est un objet dynamique, la structure d'un espace de règles de transition d'AC pour un voisinage donné est un objet statique. A la frontière se trouvent des objets donnés par l'application itérative d'une fonction simple sur un espace cellulaire initial globalement paramétré (l'automate auto-reproducteur de von Neuman, ou une machine de Turing implémentée par un espace de configuration d'un AC tel Life de Conway).

La quantité de parallélisme d'une exécution peut se mesurer au nombre d'instructions différentes rencontrées lors de cette exécution. Il existe une équivalence entre un AC défini par un réseau d'interconnexion et une fonction de transition, et un algorithme cellulaire définit par une séquence d'instructions cellulaires. Si un AC est considéré comme une instruction cellulaire élémentaire, celle-ci possède un coût, qui s'exprime comme la taille de la table de vérité de cette instruction cellulaire. Si un AC est considéré comme une séquence d'instructions cellulaires, celle-ci possède un coût, qui s'exprime comme la somme des temps de réalisation de chaque instruction, et in fine la somme de la taille des tables de vérité de chaque instruction élémentaire. L'implémentation optimale d'un algorithme cellulaire serait obtenu lorsque, pour une occupation maximale de la mémoire disponible pour les tables de vérités (les fonctions de transition) des instructions cellulaires, le nombre de primitives différentes composant l'algorithme est minimum. Le ratio du nombre de primitives employées sur le nombre de primitives existantes détermine l'importance des effets relevant de la dynamique (l'heuristique) de ceux relevant de l'algorithme proprement dit dans la fonction associée au

---

<sup>33</sup>La perception de l'œuvre n'est jamais exempte de la perception de l'histoire de sa fabrication (Gombrich, 1971).

<sup>34</sup>De l'application d'une loi non ergodique sur un espace initial nécessairement aléatoire.

programme cellulaire.

Nous disons que la fonction de transition d'un AC est localement universelle<sup>35</sup> si une seule cellule considérée comme PE se comporte comme une machine universelle sous réserve d'une extension virtuelle du nombre d'états cellulaires. C'est a priori le cas de toutes les architectures SIMD, même si le faible nombre d'états accessibles<sup>36</sup> destine évidemment ces machines à un usage cellulaire.

D'un coté le *calcul séquentiel* pur déroule une séquence d'instructions définie en extension sur un espace initial également défini en extension, de l'autre le *calcul cellulaire pur* itère une loi unique définie en extension sur un espace initial dont seules des propriétés statistiques<sup>37</sup> sont définies. La distinction entre une séquence d'instructions et une *loi* apparaissant comme une instruction unique, ne tient ni à la taille de la loi ni à une propriété telle l'universalité de l'architecture sous-tendant la séquence d'instructions, ou l'universalité locale ou globale de la loi<sup>38</sup> mais à l'opposition entre l'unicité de l'application d'une séquence d'instructions, d'un algorithme et la nécessaire itération sans fin<sup>39</sup> de la loi sur l'espace cellulaire. A la frontière<sup>40</sup> le calcul cellulaire itérant une loi unique sur un espace dont la configuration initiale est une transformation (un mapping) d'une série d'instructions d'un calcul séquentiel, c'est à dire le calcul cellulaire exploitant l'universalité globale d'une loi; ou encore le calcul cellulaire SIMD déroulant une séquence d'instructions définie en extension, c'est à dire le calcul cellulaire utilisant l'universalité locale d'une loi, et sans doute, mais sans qu'il soit possible d'en

---

<sup>35</sup>Nous dirons parfois qu'un AC est globalement universel, pour désigner la propriété usuelle d'universalité par opposition à la propriété d'universalité locale des PE de machines SIMD. L'universalité locale implique évidemment l'universalité globale. Life constitue dans le contexte de ce travail l'archétype d'un AC doté de l'universalité globale, et le processeur GAPP l'archétype d'un AC doté de l'universalité locale.

<sup>36</sup>Ce faible nombre d'états doit néanmoins être exprimé en logarithme, c'est à dire en nombre de bits adresse.

<sup>37</sup>La taille d'un espace cellulaire incarné, nécessairement fini, doit néanmoins être considérée comme tendant vers l'infini; la spécification d'une configuration cellulaire particulière en extension devient alors une barrière à la pleine utilisation des ressources disponibles.

<sup>38</sup>En effet toute séquence d'instruction, tout algorithme peuvent évidemment être transformés en une instruction unique par le calcul d'une lookup-table.

<sup>39</sup>Comme pour l'espace, ressource nécessairement limitée devant néanmoins être considérée comme tendant vers l'infini, le temps cellulaire, le nombre de cycles d'itérations de l'application de la loi sur l'espace doit être considéré comme tendant vers l'infini.

<sup>40</sup>La structure de l'espace englobant le séquentiel et le cellulaire apparaît ici très clairement.

établir simplement la part, l'universalité globale de cette loi.

Si l'on considère que l'encodage d'un algorithme sur une machine séquentielle pose un problème aigu, et encore davantage l'encodage du problème à traiter, il en va bien sûr de même pour les machines SIMD. On continue toutefois dans le contexte SIMD à résoudre des problèmes algorithmiques, à utiliser l'universalité pour la construction de machines algorithmes. Lorsque l'on passe à une machine cellulaire pure, la nature intrinsèquement statistique du processus de calcul<sup>41</sup> change radicalement le point de vue. Le problème à traiter ne pouvant dès lors guère s'exprimer que par analogie à une morphogénèse cellulaire<sup>42</sup>. Il sera toujours possible de plonger une approche SIMD, voire purement séquentielle et distribuée dans un contexte cellulaire pur, mais outre que cela pose des problèmes qui nous apparaissent assez ardues de faisabilité pratique<sup>43</sup>, cela ne tend pas à répondre à la question de la spécificité du calcul cellulaire pur. En effet, si celui-ci ne semble pas, dans son approche la plus stricte, isomorphe à une machine universelle arbitraire, ce pourrait être dû à la nature particulière des morphogénèses cellulaires, lesquelles ne sont peut-être pas réductibles à des algorithmes. Le calcul séquentiel peut être considéré comme une version adimensionnelle (de dimension zéro) d'un calcul cellulaire. Le calcul cellulaire algorithmique<sup>44</sup> correspondant au choix d'un point unique dans l'espace des états cellulaires, permettant par le plongement d'une machine de Turing dans l'espace cellulaire, de réaliser une réduction dimensionnelle de l'espace cellulaire. Le calcul cellulaire SIMD<sup>45</sup> correspondant au choix d'un mapping problème vers configuration cellulaire, associé ou non à l'utilisation des propriétés dynamiques d'une loi ou plusieurs lois pour contrôler une trajectoire dans l'espace des configurations cellulaires. Le calcul cellulaire pur, par la

---

<sup>41</sup>Nous avons jusque là évoqué la seule incertitude statistique associée à la spécification de la configuration initiale, la loi étant considérée comme purement déterministe, on voit bien cependant comment un indéterminisme sur l'espace est de fait indifférentiable d'un indéterminisme sur la loi, laquelle n'est jamais que de l'espace local générateur du temps.

<sup>42</sup>Expression à prendre dans un sens non restrictif à la biologie bien sûr, à l'existence près d'un paysage épigénétique.

<sup>43</sup>Cela pose aussi des problèmes plus fondamentaux, dont le moindre n'est pas d'éclaircir la nature ontologique de l'incertitude statistique qui semble bien devoir être liée à une définition d'un calcul cellulaire réellement massif, même si la dimension technologique de cette incertitude existe également.

<sup>44</sup>C'est à dire la réalisation d'un algorithme par une configuration cellulaire initiale associée à une loi globalement universelle.

<sup>45</sup>C'est à dire la réalisation d'un algorithme sur une configuration cellulaire par une méthode séquentielle locale.

prise en compte de l'incertitude statistique, cherche à dresser un paysage général du calcul, lequel devient une propriété intrinsèque, voire émergente d'un modèle d'univers composé d'atomes d'espace doté d'une loi universelle. Une des propriétés particulièrement intéressante, de notre point de vue, du calcul cellulaire, réside dans les équivalences espace-temps rendues possibles par son architecture. En effet la mesure des quantités élémentaires de cette architecture, ainsi que les transformations mutuelles possibles entre ces quantités sont d'implémentation aisée. Ainsi, en calcul cellulaire pur, le nombre d'états d'une cellule et le nombre de voisins de cette cellule détermine une borne supérieure à la complexité<sup>46</sup> du calcul effectué<sup>47</sup>. La taille de *Life* est de 512 bits, celle de l'automate auto-reproducteur de von Neumann de  $2^{5\lceil\log_2(29)\rceil}\lceil\log_2(29)\rceil = 20\text{mégaoctets}$ . Il serait intéressant de connaître la taille minimale d'une configuration initiale de *Life* réalisant une machine universelle, il n'existe pas à notre connaissance de réalisation effective d'une telle configuration. La taille de la configuration réalisant l'automate auto-reproducteur de von Neumann a été évaluée à plusieurs centaines de milliers de cellules. Jacqueline Signorini a étudié les conditions de réalisation de cet automate sur le GAPP (Signorini, 1992), mais une configuration auto-reproductrice complète n'a encore jamais été menée à bien.

L'implémentation de *Life* sur une machine séquentielle  $M_{\text{SISD}}$  utilise  $a$  cycles de base pour chacune des  $N$  cellules, et  $b = 2$  bits<sup>48</sup> par cellule pour le calcul d'une génération de *Life*, plus  $c \times i$  bits pour le codage des instructions,  $i$  étant la taille moyenne d'une instruction.  $c$  et  $a$  sont typiquement tous les deux de l'ordre de quelques centaines d'instructions / cycle élémentaire d'une machine monoprocesseur idéale pour une version naïve (non optimisée) de l'émulateur de la machine cellulaire. Une implémentation par lookup-table

---

<sup>46</sup>Nous n'avons pas de définition précise de cette complexité, il s'agit a priori d'une mesure statistique de la richesse de comportement, rapportée à une durée. La capacité et la rapidité morphogénétique associées à une règle.

<sup>47</sup>L'analyse de la table de transition permet de préciser une valeur de complexité, en particulier la distance à la borne supérieure. Une table constante, les tables identité ou inverse sur un seul bit du mot de voisinage (translation, et translation alternance de la configuration initiale) sont faciles à identifier. Une généralisation possible de ces notions conduit à la définition de paramètres de contrôle de l'espace des règles tels que le paramètre  $\lambda$  de Langton ou le paramètre d'enclassement.

<sup>48</sup>Un plan *past* et un plan *futur*, implémentant le synchronisme de mise à jour, on peut envisager des optimisations réduisant le nombre de bits nécessaire au plan secondaire, lesquelles se ramènent à une compression de la configuration cellulaire, associée à des méthodes d'accès spécialisées au codage de la compression, renforçant ainsi la tendance des machines séquentielles à l'échange de temps contre de l'espace.

avec collecte pipelinée du voisinage augmente  $c$  et diminue  $a$  d'un ordre de grandeur.

$$(1.1) \quad \frac{\text{Space}}{\text{Time}} = \frac{bN + ci + M_{\text{SISD}}}{aN}$$

Avec:

$$b = 2, \quad i = 32, \quad c \approx a \approx 10^2, \quad M_{\text{SISD}} \approx 10^6$$

La valeur de  $a$  et  $c$  dépendent du nombre  $k$  d'états par cellule et du rayon  $r$  du voisinage cellulaire. La taille du mot de voisinage (9 dans le cas de **Life**) détermine ces valeurs.

L'implémentation de **Life** sur une machine SIMD  $M_{\text{SIMD}}$  utilise  $a'$  cycles de base pour le calcul d'une génération, indépendamment du nombre de cellules, et  $b'$  bits par cellule, les  $c' \times i'$  bits d'instructions étant traditionnellement considérés comme n'appartenant pas stricto sensu à la machine SIMD, mais à une indispensable machine séquentielle, hôte de toute machine SIMD. Les  $a'$  cycles de calcul dépendent de la machine SIMD choisie, une machine SIMD typique est plus difficile à proposer qu'une machine SISD idéale, sur le GAPP, de l'ordre de la centaine pour une implémentation a priori de la fonction de transition<sup>49</sup>, de l'ordre du millier pour une implémentation a posteriori de celle-ci<sup>50</sup>. De l'ordre d'une dizaine de bits pour les  $b'$  bits d'espace.

$$(1.2) \quad \frac{\text{Space}}{\text{Time}} = \frac{(b' + M_{\text{SIMD}})N}{a'}$$

Avec:

$$b \approx 10, \quad a' \approx 10^2, \quad \text{SIMD} \approx 10^2$$

L'implémentation de **Life** sur une machine cellulaire pure  $M_{\text{CA}}$  utilise  $a'' = 1$  cycle et  $b'' = 1$  bit par cellule. Par contre le coût lié à l'architecture s'exprime simplement comme la taille de la table de transition.

$$(1.3) \quad \frac{\text{Space}}{\text{Time}} = \frac{(b'' + M_{\text{CA}})N}{a''}$$

---

<sup>49</sup>L'algorithme arithmétique de **Life**, comptage plus seuil.

<sup>50</sup>Une dérivation algorithmique du DFA de **Life** calculée à partir de la table de transition, laquelle peut être obtenue par observation d'une trajectoire sur l'espace des configurations, associée à une algorithmique SIMD générique interprétant le DFA sur le mot de voisinage.

Avec:

$$a'' = 1, b'' = 1, CA = 512$$

Les valeurs d'espace proposées pour le terme  $M_{\text{SISD}}$  et le facteur  $M_{\text{SIMD}}$  dans les équations 1.1 et 1.2 représentent un nombre de transistors CMOS idéal pour ces architectures. Le facteur  $M_{\text{CA}}$  de l'équation 1.3 n'est pas un nombre de transistors approximé, mais un nombre exact de bits. Cette différence de type entre les objets comptés se trouve considérablement pondérée par les croissances respectives des tailles d'espaces nécessaires à la réalisation d'une fonction de transition lorsque la fonction de transition devient un paramètre des équations précédentes. Sur une machine SISD l'augmentation de la taille du mot de voisinage induit une croissance linéaire de la taille de l'espace et du temps utilisé pour calculer une trajectoire cellulaire. L'augmentation de la taille d'espace ne dépend que de la taille du mot de voisinage, pas de la complexité de la fonction de transition<sup>51</sup>, mais l'augmentation du temps de calcul dépend de la taille de la configuration initiale. Sur une machine SIMD l'augmentation de la taille du mot de voisinage induit également une croissance linéaire de la taille de l'espace et du temps utilisé pour calculer une trajectoire cellulaire. Par contre l'augmentation de la taille d'espace dépend de la taille du mot de voisinage et de la complexité de la fonction de transition, ce facteur de croissance de la taille d'espace est donc supérieur à celui d'une machine SISD. De plus la taille de l'architecture est fonction de la taille de la configuration initiale, ce facteur présentant évidemment la source principale des besoins de ressource spatiale. Par contre, pour prix de cette augmentation linéaire des besoins d'espace par rapport à la version SISD, le temps de calcul n'est plus fonction de la taille de l'espace initial. Sur une machine cellulaire pure l'augmentation de la taille du mot de voisinage induit une croissance exponentielle de la taille d'espace via une croissance exponentielle de la taille de la table elle-même facteur de la taille de la configuration initiale, le temps se réduisant à l'unité.

## 1.8 Plan de lecture

Dans le chapitre 2 nous présentons la structure des objets de notre étude: l'espace, le temps et les lois, ainsi que les méthodes applicables sur ces objets: édition, réduction, transformation, visualisation pour l'espace et le

---

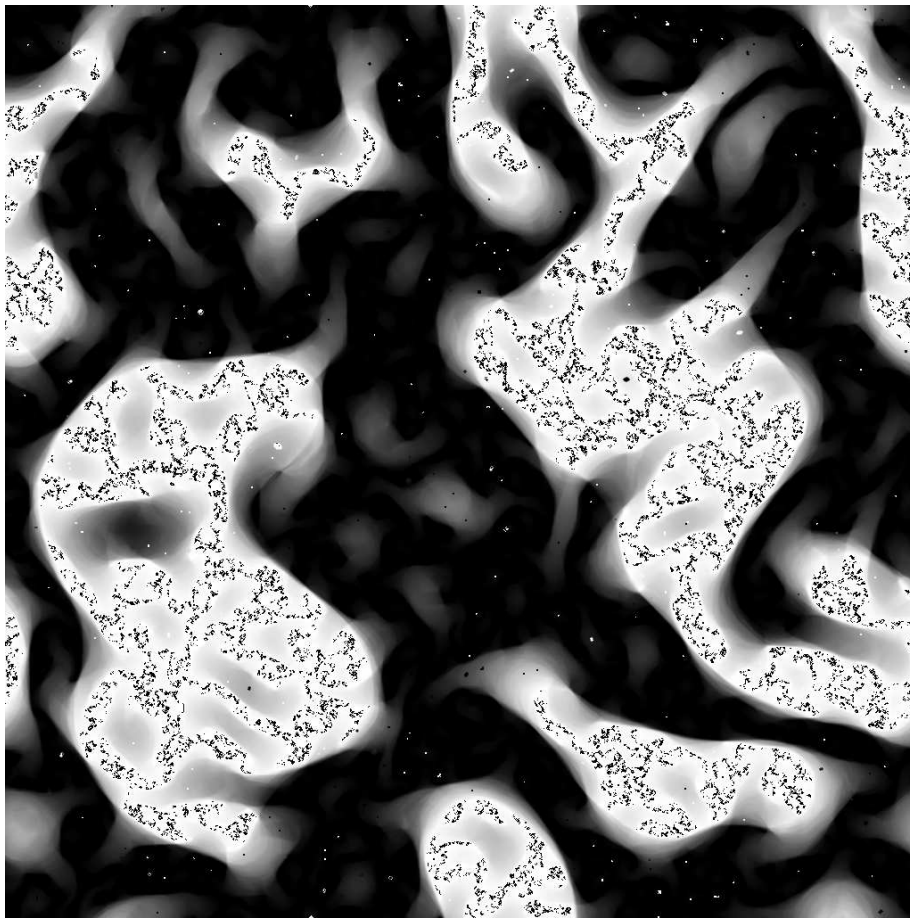
<sup>51</sup>A condition, bien sûr, de ne pas transformer la fonction de transition en lookup-table.

temps, plus construction, altération, composition, mélange et séparation pour les lois. Nous présentons également dans ce chapitre l'architecture de nos moteurs cellulaires.

Dans le chapitre 3 nous présentons les outils conceptuels utilisés pour la quantification de l'espace des règles, et nous proposons un nouveau paramètre de contrôle pour l'exploration et la structuration de cet espace: le paramètre d'enchâssement.

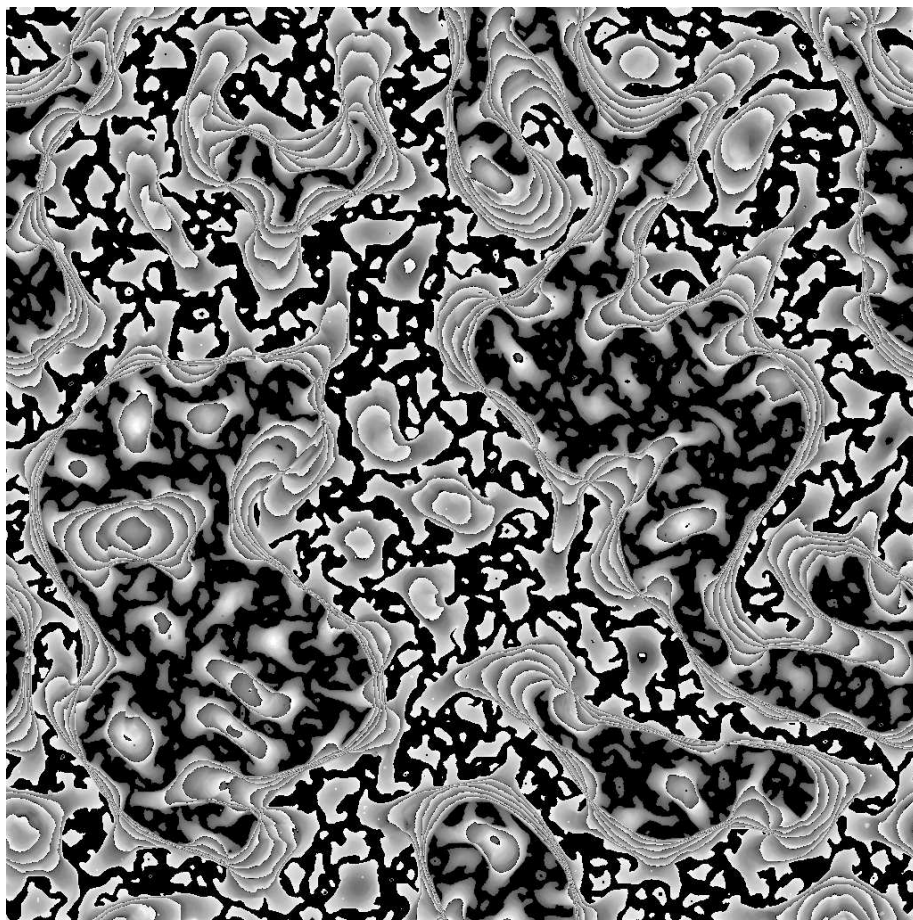
Le chapitre 4 est consacré à une étude expérimentale de la règle **Anneal** basée sur l'exploitation de nos outils. Les expériences réalisées nous amènent à proposer l'existence d'une relative invariance spatio-temporelle des trajectoires associées à une transition de phase dans l'espace des configurations initiales.

La conclusion introduit les limites de notre approche qui à associé un constructivisme bottom-up à une *gestion de tables généralisée*. Nous évoquons pour finir une approche alternative de réalisation cellulaire purement algorithmique.



Cette image est obtenue par la superposition de la réduction intégrale  $\sum_{i=0}^n t_i xy$  des  $2^{10}$  et  $2^{13}$  premières générations d'un espace initial de taille  $1024 \times 1024$  et de densité  $\frac{1}{2}$ .

Figure 1.16: *Une superposition de réduction intégrale de Anneal*



Une colormap en bandes. Pour cette image, comme pour toutes les images en niveaux de gris que nous avons présentées, un périphérique de sortie capable de restitution nuancée nous fait défaut.

Figure 1.17: *Une autre colormap pour la figure 1.16*

## Chapitre 2

# Vers des objets cellulaires: espace, temps, lois

Nous soutenons qu'un environnement de développement d'AC doit reposer sur un ensemble d'objets permettant la construction d'expériences. Ces objets doivent correspondre aux ressources utilisables par une simulation et fournir les opérations et les fonctions de réduction d'information utilisées lors de la description de l'expérience projetée.

Les objets de base d'une simulation sont l'*espace*, le *temps* et la *fonction de transition*. La fonction de transition se compose d'une structure d'interconnexion cellulaire associable à un espace, d'une fonction de transition locale<sup>1</sup>, applicable de manière synchrone sur l'ensemble des cellules d'un espace doté d'un réseau d'interconnexion, et d'une fonction d'application utilisant les composants de la fonction de transition pour générer le nouvel état d'un l'espace. La fonction de transition réalise les lois de la dynamique de l'univers cellulaire simulé. Elle est génératrice du temps par application sur l'espace.

Des instances d'objets de base sont regroupées pour la réalisation d'une simulation au sein d'un objet de type séquenceur. Des manipulateurs et des observateurs peuvent être associés aux objets de la simulation.

---

<sup>1</sup>L'expression *fonction de transition* désigne, suivant le contexte, soit l'objet regroupant le réseau d'interconnexion et la fonction de transition locale, soit la fonction de transition locale elle-même. La fonction de transition locale sera par ailleurs également nommée *règle* ou *table de transition*, et l'objet fonction de transition sera à l'occasion nommé loi, ou AC.

---

## 2.1 L'espace: la mémoire

L'espace est un ensemble de cellules ordonnées conformément à la structure d'interconnexion de la fonction. La structure de l'espace peut théoriquement être quelconque pourvu qu'elle soit homogène. En pratique nous limitons initialement les structures disponibles aux espaces de dimension  $d$  1, 2 ou 3. Le nombre  $r$  de cellules distantes de  $n$  unités au plus d'une cellule quelconque de l'espace est  $(2n+1)^d$ . La structure d'interconnexion sera un sous-ensemble de l'ensemble des cellules distantes de  $n$  unités au plus, avec  $n$  typiquement limité à 1 ou 2 unités. Nous appellerons *cube d'interconnexion* de dimension  $d$  une structure d'interconnexion complète pour la dimension  $d$  et *croix d'interconnexion* de dimension  $d$  l'ensemble des cellules distantes de  $n$  unités au plus sur une dimension de l'espace au plus. Le cube d'interconnexion de dimension 2 et distance 1 correspond au voisinage de Moore, et la croix d'interconnexion de dimension 2 et distance 1 au voisinage de von Neumann. Le nombre d'éléments d'une croix d'interconnexion est  $2nd + 1$ .

| $2^{(2n+1)^d-3}$ |   | $n$         |              |              |               |
|------------------|---|-------------|--------------|--------------|---------------|
|                  |   | 1           | 2            | 3            | 4             |
| $d$              | 1 | <b>1</b>    | <b>4</b>     | <b>16</b>    | <b>64</b>     |
|                  | 2 | <b>64</b>   | <b>4M</b>    | $64T$        | $> 10^{23}$   |
|                  | 3 | <b>16M</b>  | $> 10^{36}$  | $> 10^{102}$ | $> 10^{218}$  |
|                  | 4 | $> 10^{23}$ | $> 10^{187}$ | $> 10^{721}$ | $> 10^{1974}$ |

Taille en octets des tables de transitions de quelques cubes d'interconnexions de cellules binaires.

Tableau 2.1: *Tailles induites par des cubes d'interconnexions*

Le tableau 2.1 donne la taille en bytes des tables de transitions des cubes d'interconnexions de cellules binaires pour les 4 premières dimensions d'espace et les 4 premières distances. Les entrées en caractères gras sont réalisables par lookup-table.

Le tableau 2.2 suggère l'intérêt de voisinages mixtes

$$\text{croix}(d, n) \cup \text{cube}(d, n - 1)$$

Mais la structure d'interconnexion régulière offrant le meilleur compromis distance / volume / isotropie est une *sphère de Manhattan*, une hyper-sphère

de dimension  $d$  dont le rayon est une distance de Manhattan<sup>2</sup>. Une sphère de Manhattan de rayon 1 définit la même structure d'interconnexion qu'une croix de rayon 1. Le voisinage de von Neumann est donc une sphère de Manhattan, mais pas un voisinage de Moore. Avec des cellules binaires, en dimension 2 les sphères de rayon 2 et 3 sont accessibles<sup>3</sup>, en dimension 3 la sphère de rayon 2 est accessible<sup>4</sup>.

L'espace supporte des fonctions d'entrée / sortie, d'édition, de réduction et de transformation.

| $2^{2nd-2}$ |   | $n$       |            |            |            |
|-------------|---|-----------|------------|------------|------------|
|             |   | 1         | 2          | 3          | 4          |
| $d$         | 1 | <b>1</b>  | <b>4</b>   | <b>16</b>  | <b>64</b>  |
|             | 2 | <b>4</b>  | <b>64</b>  | <b>1K</b>  | <b>16K</b> |
|             | 3 | <b>16</b> | <b>1K</b>  | <b>64K</b> | <b>4M</b>  |
|             | 4 | <b>64</b> | <b>16K</b> | <b>4M</b>  | <b>1G</b>  |

La taille en bytes des tables de transitions induite par des croix d'interconnexions de cellules binaires permet l'exploration de voisinages utilisant de plus grandes valeurs pour les paramètres dimension et rayon, au détriment du rapport  $\frac{\text{volume}}{\text{rayon}}$ , donc de l'isotropie.

Tableau 2.2: *Tailles induites par des croix d'interconnexions*

### 2.1.1 Structure de l'espace: dimension et voisinage

L'espace apparaît comme une ligne, un plan, ou un volume de cellules adressables globalement par leurs coordonnées euclidiennes. Nous utiliserons un espace de dimension 2 par défaut pour exposer les opérations applicables sur les espaces. Cet espace peut être vu et manipulé soit comme un plan de cellules, soit comme un vecteur de plans de bits correspondant à une tranche cellulaire. Les opérations sur un espace sont toutes les opérations possibles

<sup>2</sup>Le volume d'une sphère de Manhattan est  $S(d, r) = S(d-1, r) + 2 \sum_{i=0}^{r-1} S(d-1, i)$  avec  $S(1, r) = 2r + 1$ . Les définitions que nous proposons pour les cubes, croix et sphère de connexions considèrent le centre comme exclu du rayon. L'inclusion du centre dans le rayon permet la définition de cubes et de sphères privés d'un centre ponctuel. Le voisinage de Margolus (Toffoli et Margolus, 1987) par exemple utilise un carré de 2 cellules de côté.

<sup>3</sup>1K et 4M

<sup>4</sup>4M

| $2^{S(d,n)-3}$ |   | $n$       |             |             |             |
|----------------|---|-----------|-------------|-------------|-------------|
|                |   | 1         | 2           | 3           | 4           |
| $d$            | 1 | <b>1</b>  | <b>4</b>    | <b>16</b>   | <b>64</b>   |
|                | 2 | <b>4</b>  | <b>1K</b>   | <b>4M</b>   | <b>256G</b> |
|                | 3 | <b>16</b> | <b>4M</b>   | $> 10^{18}$ | $> 10^{37}$ |
|                | 4 | <b>64</b> | <b>256G</b> | $> 10^{37}$ | $> 10^{95}$ |

La taille en bytes des tables de transitions induite par des sphères d'interconnexions de cellules binaires, est particulièrement intéressante pour les valeurs  $d = 2, n = 3$  et  $d = 3, n = 2$  qui sont inaccessibles au voisinage cube (cf. tableau 2.1, page 34).

Tableau 2.3: *Tailles induites par des sphères d'interconnexions*

sur une mémoire organisée selon une dualité vecteur de plans de bits, plan de vecteurs de bits.

Idéalement, en association avec une structure d'interconnexion, le vecteur de bits de la mémoire cellulaire est associé à une structure de nommage. Les noms des zones de mémoire sont utilisables en paramètres des opérations portant sur les tranches d'espaces cellulaires (édition, visualisation, réduction) et sur les variables cellulaires (définition de la fonction de transition).

### 2.1.2 Édition d'espace: les configurations cellulaires

Une première famille d'opérations est fournie par un éditeur de *pixmaps*<sup>5</sup> permettant la construction des configurations d'espaces au temps zéro d'une simulation. L'éditeur permet également la modification du point d'espace courant dans le temps entre chaque application de la fonction de transition. Il est souhaitable de pouvoir utiliser aussi d'autres outils que les nôtres (que nous nommons outils externes) pour manipuler les espaces. Le format de stockage externe doit permettre l'application *off-line*<sup>6</sup> de nombreux outils sur les espaces. Ceci permet une intégration étroite de ces outils à l'environnement de développement et / ou d'interaction, mais implique l'existence de plusieurs classes de débit pour les opérations applicables sur les objets. Cette contrainte est quasiment sans effet pour l'utilisation d'éditeurs interactifs de

<sup>5</sup>Une mémoire de pixels.

<sup>6</sup>Le terme off-line désigne le passage des objets de la simulation à d'autres processus.

Nous avons choisi PBM<sup>[7]</sup> comme format<sup>a</sup> pivot de bitmap / pixmap. Cette figure montre l'application de la règle xor9 (cf. § A, page 119) sur un espace obtenu à partir du tableau 2.1. Pour ce faire `groff`<sup>[8]</sup> est utilisé pour produire une représentation PostScript de la page, puis `ghostscript`<sup>[9]</sup> est utilisé pour produire une représentation PBM de la page, divers outils comprenant le format PBM tel `xv`<sup>[10]</sup> peuvent être utilisés à ce stade pour retravailler le bitmap, enfin le bitmap est utilisé par le `workbench` qui fournit le bitmap transformé par l'application itérative d'une règle, lequel est retransformé en PostScript pour insertion dans le document final.

<sup>a</sup>Les numéros en indice supérieur entre crochets renvoient au répertoire des outils (cf. § D, page 131).

Figure 2.1: *Édition d'espace; génération 16 de la règle Xor9 sur le tableau 2.1*

type *painter*<sup>7</sup> ou *drawer*<sup>8</sup> sur un espace. L'éditeur devrait également fournir une famille de fonctions de remplissage aléatoire de l'espace aussi riche que possible.

### 2.1.3 Réduction d'espace: taille et contenu

La deuxième famille d'opérations fournit des fonctions de réduction de l'information contenu dans un espace. Il s'agit de réductions arithmétiques élémentaires (somme, histogramme, parité) de réductions statistiques stan-

<sup>7</sup>Un outil métaphore de la palette et du pinceau, lorsque ceux-ci sont utilisés pour peindre une surface

<sup>8</sup>Un outil métaphore du crayon lorsqu'il est utilisé pour dessiner.

dard (minimum, maximum, moyenne, entropie, dimension fractale) ou spécialisées comme le comptage de patterns propres à une simulation. On peut ajouter les outils standards de compressions de données qui, outre l'archivage des espaces, fournissent des quantités informationnelles.

Ici encore l'utilisation d'un format pivot standard et doté de nombreux outils de conversion de format permet l'utilisation d'outils statistiques off-line.<sup>9</sup> De plus les outils externes se présentant sous la forme de librairie sont également intégrables à l'environnement de développement et permettent une utilisation directe qui ne nécessite pas l'emploi de fichiers temporaires.

### 2.1.4 Transformation d'espace: d'une forme à l'autre



L'espace original est présenté par la figure 2.5. Ces transformations ont été réalisées avec les outils PBM. Le *dithering* permet la réduction d'un greymap à un bitmap.

Figure 2.2: *Transformation d'espace; un dithering et deux détections de contours*

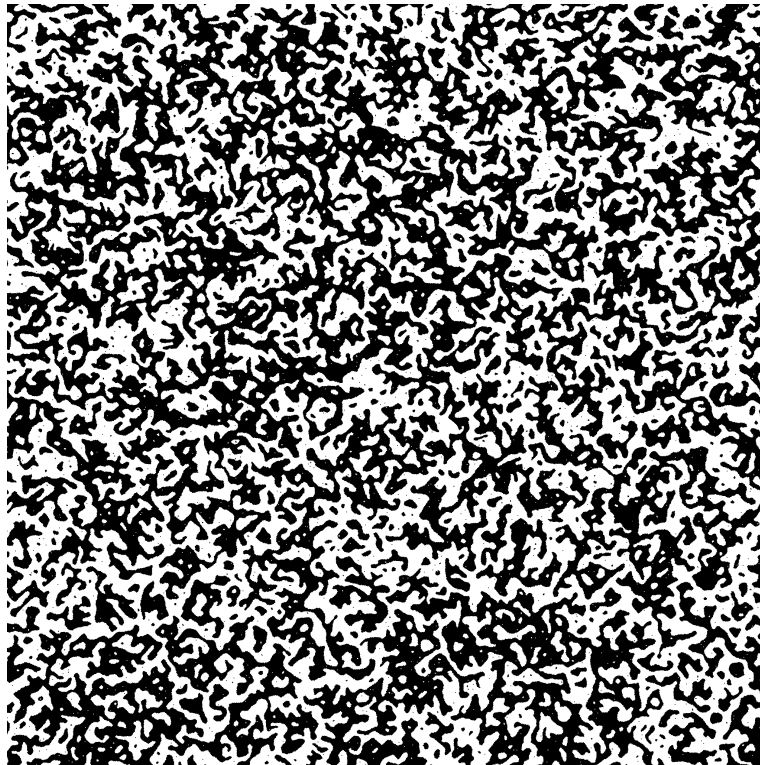
Enfin, des transformations globales arbitraires sont applicables sur les espaces. Il s'agit par exemple des fonctions de traitement du signal en général, et de traitement d'images tel que la reconnaissance des formes et l'extraction de contour. La figure 2.2 montre une détection de contours utilisant des outils fournis par PBM. Voir aussi figure 2.5, page 42.

Certaines de ces fonctions sont elles mêmes basées sur des algorithmes cellulaires, elle doivent alors pouvoir être réalisées à l'aide de l'environnement de développement, les fonctions de séquençement permettant leur application non destructive à la volée sur le point d'espace courant d'une séquence.

<sup>9</sup>Voir tableau 2.7, page 56.

### 2.1.5 Visualisation d'espace: comprendre les lois

L'espace constitue un objet de visualisation élémentaire a priori. Il peut être vu par l'intermédiaire de fenêtres X11 associées à un bitmap, ou à une séquence de plans de bits (pixmap) associée à une *colormap*.



Génération 128 de la règle Anneal, sur une espace initial de densité  $\frac{1}{2}$ , et de taille  $2048 \times 2048$ .

Figure 2.3: *Un bitmap point-bit  $2048 \times 2048$*

La figure 2.3 montre la visualisation par bitmap de l'espace de la génération 128 de la règle Anneal<sup>10</sup>. L'espace est de taille  $2048 \times 2048$ . L'espace initial a une densité aléatoire  $\frac{1}{2}$ . La taille de la cellule étant de 1 bit, le bitmap permet de visualiser l'intégralité des états cellulaires. Chaque point de l'image correspond à un bit.

La figure 2.4 montre 8 séquences de 8 espaces chacune. Il s'agit de séquences de la règle Tube-Worm<sup>11</sup> appliquée sur le même espace initial, les variations

<sup>10</sup>Voir § A, page 120.

<sup>11</sup>Voir § A, page 121.

portant sur les paramètres statiques<sup>12</sup> de la règle. Le nombre de générations entre deux espaces dépend d'un des paramètres statiques de la règle. Il est de 32 intervalles du temps propre à cette règle.

La figure 2.5 montre la visualisation par pixmap de 2 générations de la règle **Qmean** (cf. § A, page 119). La taille de la cellule étant de 8 bits, 256 couleurs sont nécessaires pour visualiser tous les états possibles<sup>13</sup>.

La figure 2.6 montre la visualisation des générations  $2^3$  à  $2^{10}$  (de haut en bas) de la règle **Qmean**. L'espace est de taille  $128 \times 128$ . L'espace initial a une densité aléatoire  $\frac{1}{2}$ . Les 256 états possibles de chaque cellule sont visualisés par 8 bitmaps au lieu d'un pixmap. Les bitmaps sont présentés de gauche à droite pour les bits 0 à 7.

Les variables de réductions d'espace sont présentées sous forme ASCII, associées aux fenêtres pixmap, ou éventuellement par des *viewers*<sup>14</sup> de groupe de variables scalaires (bargraph par exemple). Ce type de visualisation (bitmaps + scalaires) est assez rapide pour permettre la réalisation de films de séquences d'espaces à une fréquence supérieure à 1 Hz.

D'autres types de visualisations dédiées à une simulation particulière ou plus consommatrice en ressources sont possibles:

- Une version iconique de l'état d'une cellule.
- Une vision tridimensionnelle, réalisée par *plotter*<sup>15</sup> (cf. figure 2.9, page 50) ou *renderer*<sup>16</sup> (cf. figure 2.7, page 44) d'un espace considéré comme surface d'élévation, ou dont une des variables d'état de la cellule est considérée comme donnée d'élévation, et une autre comme donnée de projection.
- Pour les espaces de dimension 3, une vision tridimensionnelle par seuillage, épluchage ou transparence.

Les visualisations coûteuses en ressources, ne permettant pas la réalisation de films temps réel, servent à la réalisation d'instantanés, la réalisation de films de séquences d'instantanés restant possibles en temps différé.

---

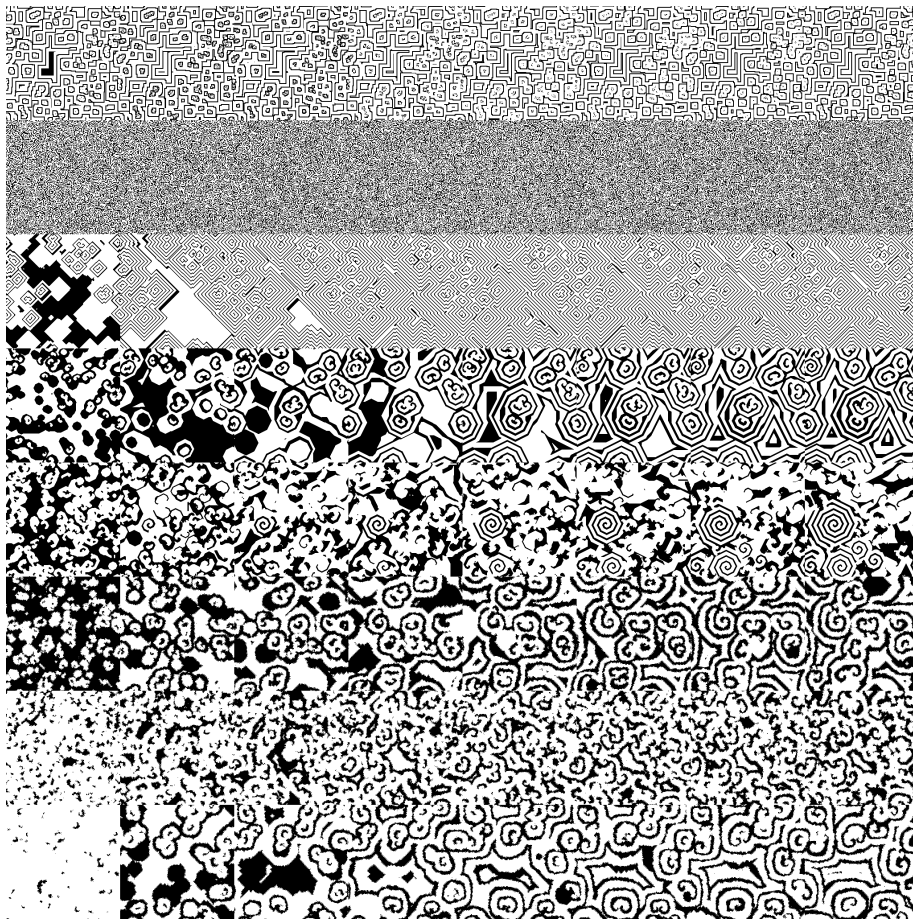
<sup>12</sup>Voir § 2.3.5, page 72.

<sup>13</sup>Bien entendu, en termes purement informationnel l'espace est comme pour la figure 2.3  $2048 \times 2048$ .

<sup>14</sup>visualiseurs.

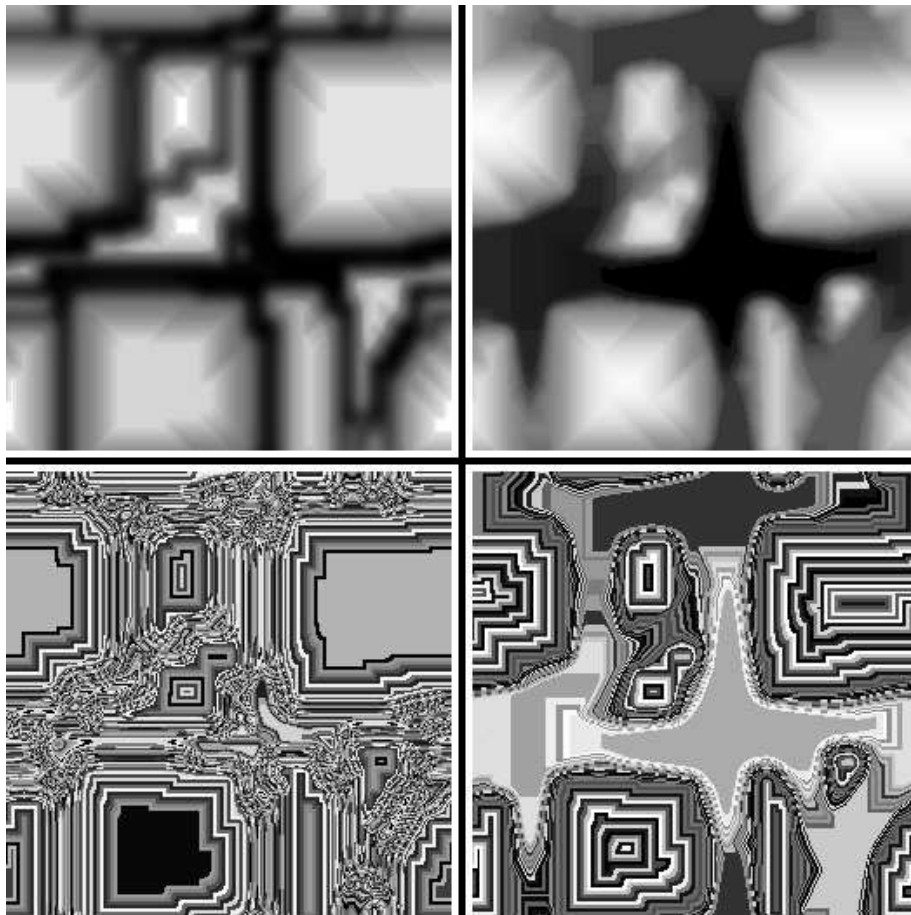
<sup>15</sup>Traceur. Ce type de visualisation repose évidemment sur le transfert d'un espace à un outil externe spécialisé. Nous avons utilisé *gnuplot*<sup>[11]</sup> et *Khoros*<sup>[12]</sup>.

<sup>16</sup>Générateur de rendu. Nous avons utilisé *qrt*<sup>[14]</sup> et la toolbox *vart*<sup>[15]</sup> sous *Khoros*.



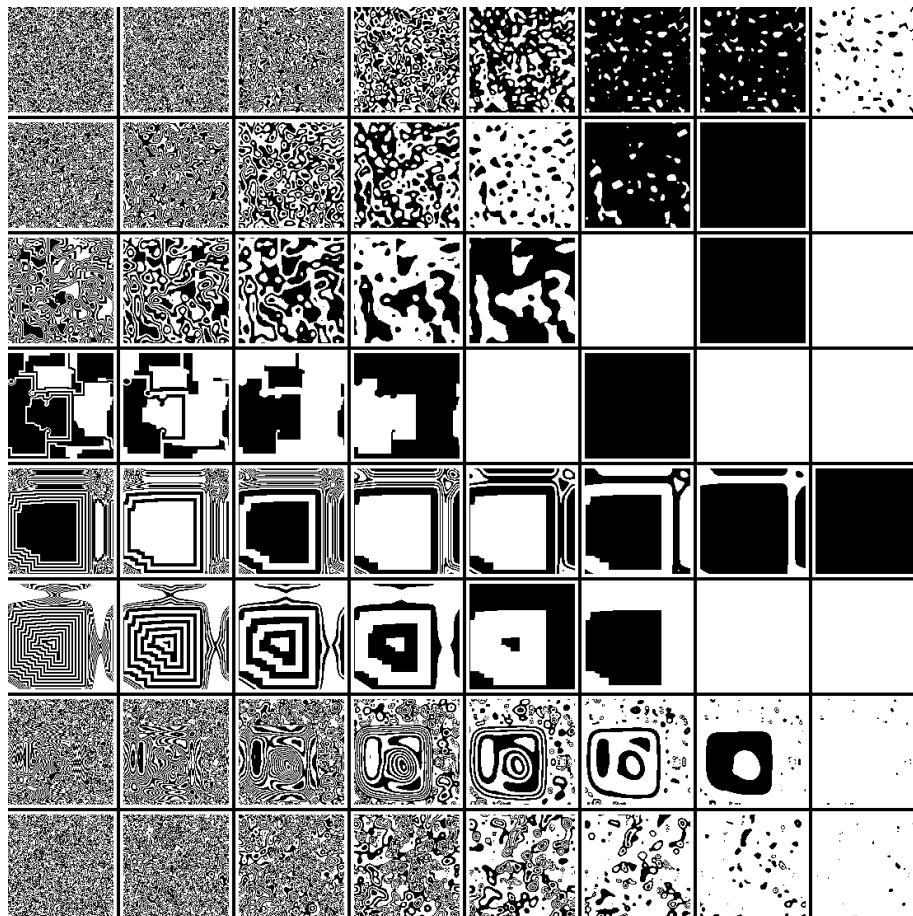
Une composition construite avec 8 séquences (de haut en bas), de 8 espaces chacune (de gauche à droite) de la règle Tube-Worm appliquée sur le même espace initial

Figure 2.4: *Un bitmap constitué de 64 bitmaps  $256 \times 256$*



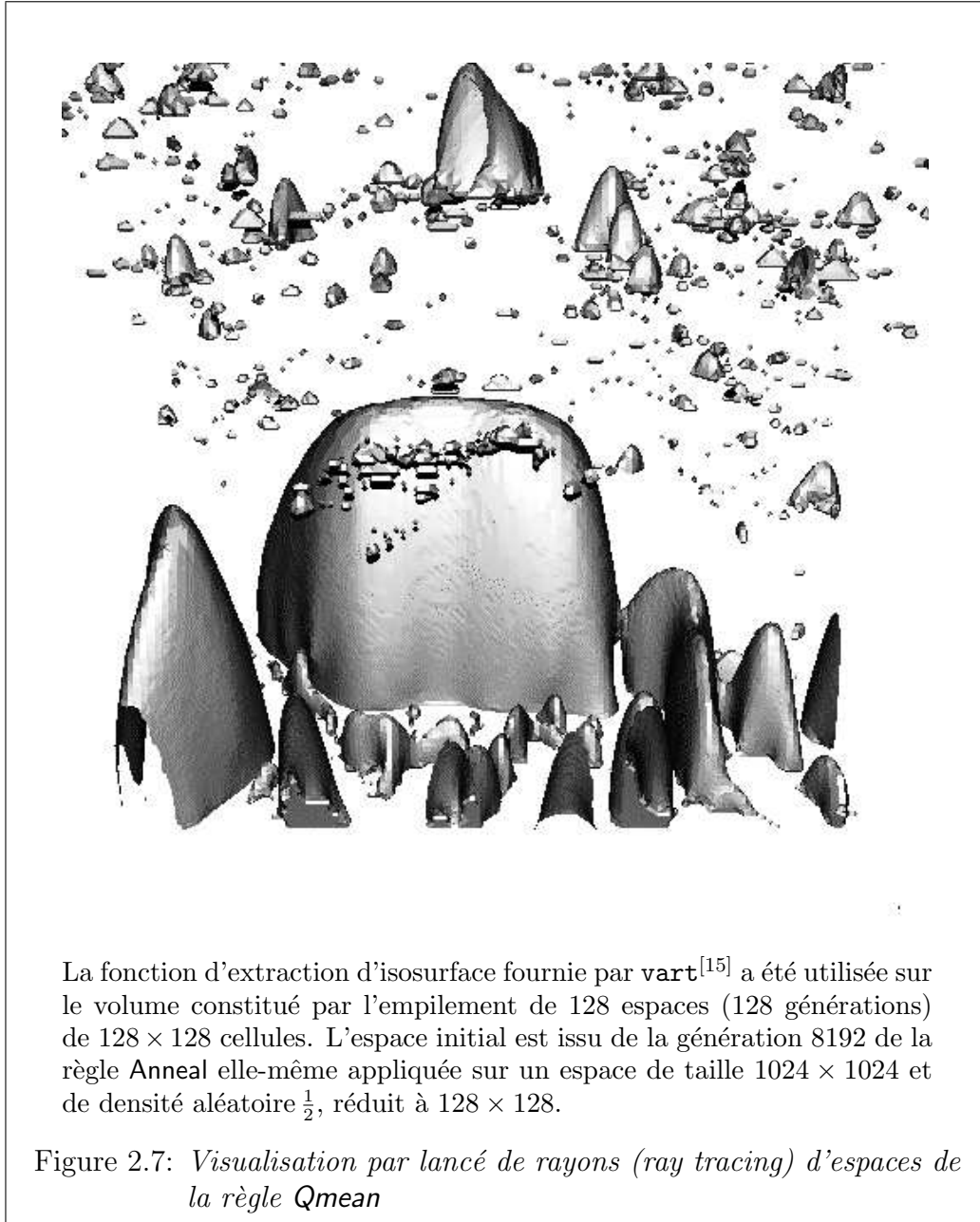
Les génération 128 et 256 (de gauche à droite) de la règle *Qmean* utilisant deux types de colormaps (de haut en bas). L'espace est de taille  $256 \times 256$ . L'espace initial a une densité aléatoire  $\frac{1}{2}$ . Les colormaps sont ici des greymaps. Les deux pixmaps supérieurs utilisent une greymap dont l'échelle des valeurs est progressive. Les deux pixmaps inférieurs utilisent une greymap dont l'échelle des valeurs est aléatoire, masquant la répartition générale des intensités, mais révélant les détails de l'alternance de valeurs.

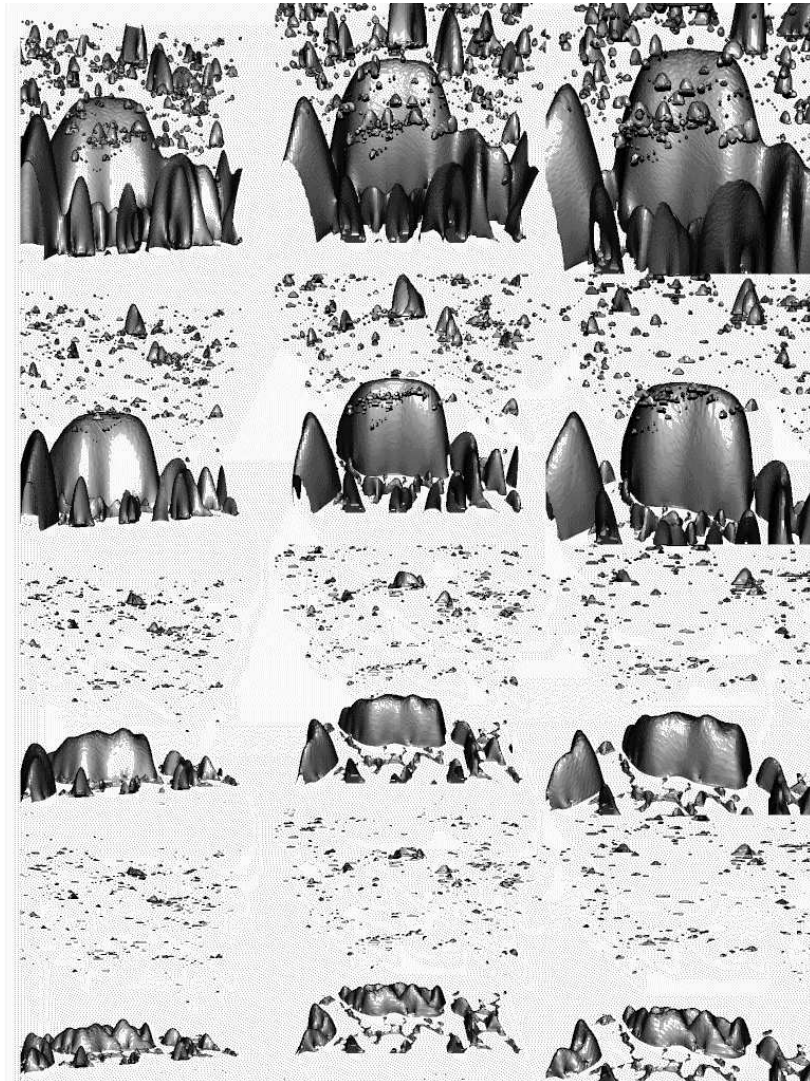
Figure 2.5: *Visualisation par pixmap de 2 générations de la règle Qmean*



Les générations  $2^3$  à  $2^{10}$  (de haut en bas) de la règle Qmean. L'espace est de taille  $128 \times 128$ . L'espace initial a une densité aléatoire  $\frac{1}{2}$ . Les 256 états possibles de chaque cellule sont visualisés par 8 bitmaps au lieu d'un pixmap. Les bitmaps sont présentés de gauche à droite pour les bits 0 à 7.

Figure 2.6: *Visualisation par bitmap de 8 plans de 8 générations de la règle Qmean*





Par rapport à la figure 2.7, les variations de paramètres sur l'axe horizontal porte sur la prise de vues; les variations de paramètres sur l'axe vertical porte sur la valeur de seuillage utilisée. La valeur de seuil varie de 0.3 à 0.8.

Figure 2.8: *Visualisation par films de vues ray-tracées d'espaces de la règle  $Q_{mean}$*

### 2.1.6 Ressources associées à l'espace: des limites à choisir

La consommation mémoire d'un espace d'arête  $a$  et de dimension  $d$  est  $a^d$ . Pour des raisons détaillées lors de la présentation de la fonction de transition<sup>17</sup>, l'organisation mémoire de l'espace privilégie le plan de vecteurs de bits, d'une taille multiple de l'unité adressable, c'est à dire que les cellules seront constituées au minimum d'un byte.

| $a^d$ |   | $a$        |            |             |             |            |             |           |
|-------|---|------------|------------|-------------|-------------|------------|-------------|-----------|
|       |   | 16         | 32         | 64          | 128         | 256        | 512         | 1K        |
| $d$   | 1 | <b>16</b>  | <b>32</b>  | <b>64</b>   | <b>128</b>  | <b>256</b> | <b>512</b>  | <b>1K</b> |
|       | 2 | <b>256</b> | <b>1K</b>  | <b>4K</b>   | <b>16K</b>  | <b>64K</b> | <b>256K</b> | <b>1M</b> |
|       | 3 | <b>4K</b>  | <b>32K</b> | <b>256K</b> | <b>2M</b>   | <b>16M</b> | <i>128M</i> | <i>1G</i> |
|       | 4 | <b>64K</b> | <b>1M</b>  | <b>16M</b>  | <i>256M</i> | <i>4G</i>  | <i>64G</i>  | <i>1T</i> |

Ce tableau donne les tailles en byte, fonction des paramètres dimension et arête, d'espaces accessibles à l'expérimentation.

Tableau 2.4: *Tailles d'espaces pour quelques tailles d'arête des premières dimensions*

Le tableau 2.4 donne un aperçu des tailles d'espaces maximum accessibles pour les 4 premières dimensions d'espace. On peut voir que si les espaces de dimension 2 ne posent pas de problèmes, les espaces de dimension 3 seront avantageusement ramenés à des parties de cube. Par exemple, les 256K d'un espace  $128 \times 64 \times 32$  sont probablement plus avantageux que les 256K d'un espace  $64 \times 64 \times 64$ . La consommation CPU de la fonction de transition, croissant linéairement en fonction de la taille de l'espace, on peut également avoir une idée des temps de calculs associés à la taille des espaces. Celui-ci est  $\frac{Ga^d}{D}$ , avec  $G$  pour le nombre de générations,  $d$  pour la dimension de l'espace et  $D$  pour le débit de la fonction d'application.

Le tableau 2.5 montre les temps associés au calcul de  $1K^{18}$  générations d'espaces du tableau 2.4 avec une fonction d'application à 100 KCPS<sup>19</sup>, (consommant 100 instructions par cellule sur une machine 10 MIPS, par exemple). Seule l'unité de temps la plus significative est indiquée. On pourrait obtenir

<sup>17</sup>Voir § 2.3, page 64.

<sup>18</sup>Les débits sont en puissance de 10:

1 kilocell = 1024 cell, 1 kilo génération = 1000 générations.

<sup>19</sup>Kilo Cell update Per Second.

|     |   | $\frac{Ga^d}{D}$ |       |       |     |     |     |      |
|-----|---|------------------|-------|-------|-----|-----|-----|------|
|     |   | 16               | 32    | 64    | 128 | 256 | 512 | 1K   |
| $d$ | 1 | 160ms            | 320ms | 640ms | 1s  | 2s  | 5s  | 10s  |
|     | 2 | 2s               | 10s   | 40s   | 2m  | 10m | 43m | 2h   |
|     | 3 | 40s              | 5m    | 43m   | 5h  | 1j  | 15j | 124j |
|     | 4 | 10m              | 2h    | 1j    | 31j | 1a  | 21a | 348a |

Ce tableau donne un aperçu des expériences réalisables fonction des tailles d'espace et de la puissance de calcul du moteur cellulaire. Les paramètres nombre de générations et puissance de calcul sont fixés respectivement à 1000 générations et 100 KCPS. Un temps de calcul d'une journée pour 1000 générations est considéré comme la limite praticable. Un débit à 100 KCPS constitue une hypothèse basse. Notre environnement fournit actuellement un débit supérieur d'un ordre de grandeur.

Tableau 2.5: *Temps de calcul sur les espaces du tableau 2.4 fonction du nombre de génération et de la puissance de calcul*

|     |   | $\frac{Ga^d}{D}$ |       |      |      |      |       |
|-----|---|------------------|-------|------|------|------|-------|
|     |   | 32               | 64    | 128  | 256  | 512  | 1K    |
| $d$ | 1 | 3ms              | 6ms   | 12ms | 25ms | 51ms | 102ms |
|     | 2 | 102ms            | 409ms | 1s   | 6s   | 26s  | 1m    |
|     | 3 | 3s               | 26s   | 3m   | 27m  | 3h   | 1j    |
|     | 4 | 1m               | 27m   | 7h   | 4j   | 79j  | 3a    |

Le gain d'un ordre de grandeur, par rapport au performance de notre implémentation, ouvrirait l'exploration des espaces de dimension 3 (nous considérons empiriquement qu'une arête de taille 256 constitue une base suffisante à l'exploration de ces espaces). Ce gain pourrait être obtenu en partie par une implémentation répartie du moteur cellulaire.

Tableau 2.6: *Variations des temps de calculs du tableau 2.5 résultant d'une augmentation de deux ordre de grandeur de la puissance de calcul*

un débit de 10 MCPS avec une fonction d'application consommant 20 instructions par cellule, et quatre processeurs à 50 MIPS, lesquels ne partagent qu'une faible partie de la mémoire totale<sup>20</sup>. Le tableau 2.6 reprend les calculs du tableau 2.6 pour  $G = 10\text{MCPS}$ .

---

<sup>20</sup>Voir § 1.5, page 21.

## 2.2 Le temps: les cycles

Le temps  $t$  est un ensemble de  $N$  espaces ordonnés. Il supporte toutes les opérations sur les séquences d'espaces. Ces opérations sont plus difficiles à définir que celles portant sur les espaces. Leur mise en oeuvre nécessite l'enchâssement du temps dans un séquenceur. C'est précisément le choix des primitives appartenant en propre au temps, par opposition à celles qui doivent être associées à un séquenceur qui pose problème. Nous espérons que ces problèmes seront peu à peu levés à l'usage. Nous tenterons néanmoins de définir, en faisant momentanément abstraction de la structure particulière d'un séquenceur, une liste d'opérations possibles.

Le temps est a priori une séquence d'espaces générée par l'application itérative d'une fonction de transition sur un espace initial. Si l'on utilise une métaphore cinématographique, il permet l'enregistrement, le tirage et le montage de films. Il permet également des réductions d'informations différentielles sur les espaces, des réductions de séquences de réductions d'espaces, et des transformations arbitraires.

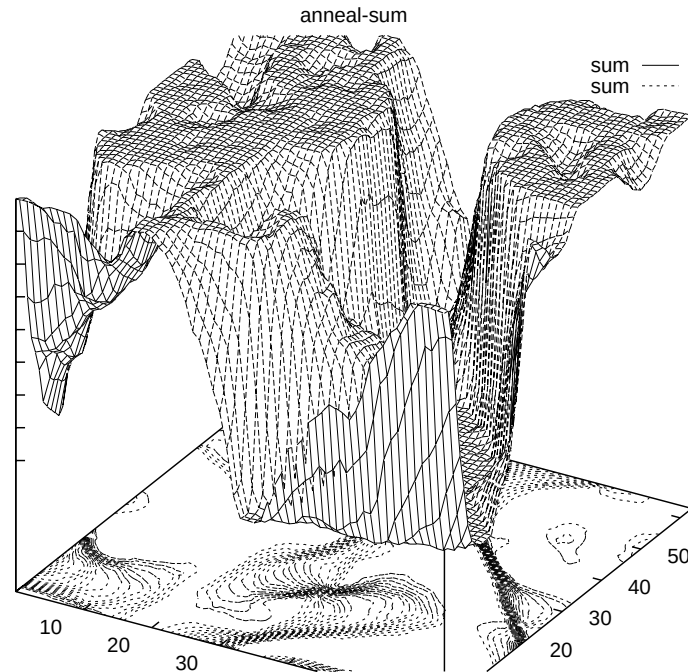
### 2.2.1 Structure du temps: un axe d'organisation privilégié

Le temps est un vecteur d'espaces de taille fixe associé à divers compteurs et pointeurs. La construction des opérateurs complexes sur le temps repose sur ces éléments, devant permettre un accès plus ou moins structuré à la mémoire temps.

Lorsque le système est réalisé sur une machine séquentielle, le temps fournit les espaces temporaires (accumulateurs) nécessaires à la réalisation de la fonction d'application synchrone de la fonction de transition sur l'ensemble des cellules de l'espace courant. La notion d'accumulateur recevant le résultat de l'application d'opérateurs sur les espaces est sans doute nécessaire dans tous les contextes, mais sa réalisation sur une machine parallèle arbitraire, ou dédiée AC pourrait s'avérer délicate<sup>21</sup>.

---

<sup>21</sup>Sur une architecture SIMD privilégiant le nombre de PE (Processeur Élémentaire) au détriment de la mémoire locale à chaque PE (de type GAPP), où chaque PE est associé à une cellule, et la mémoire locale de chaque PE aux états de chaque cellule, la source et la cible de l'application d'une fonction de transition sont potentiellement la mémoire totale de la machine.



Cette visualisation par surface d'élévation d'une réduction intégrale du temps résultant de la somme de 400 générations de la règle **Anneal** à été réalisée avec **gnuplot**, elle illustre l'intérêt d'une chaîne de traitement combinant génération, réduction/transformation et visualisation d'espaces cellulaires. D'autres d'images résultant de manipulations d'espaces générés par la règle **Anneal** sont présentées dans les figures 4.1, 2.10, 2.15, 2.16, 1.16, 1.17, 2.3 et en § 4

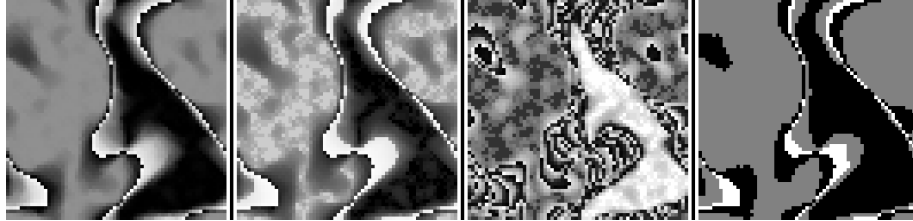
Figure 2.9: *Somme de 400 générations de la règle Anneal*

### 2.2.2 Édition du temps: une métaphore cinématographique

Voici les opérateurs d'édition du temps:

- Remise à zéro et incrémentation d'un compteur d'application d'une fonction de transition<sup>22</sup>.

<sup>22</sup>Ce compteur doit être ensuite associé à un espace. En règle générale l'inclusion d'un espace dans un objet de plus haut niveau implique la création de champs nouveaux servant à mémoriser le contexte d'usage de cet espace. Par exemple, la photo d'un espace doit si possible contenir des annotations sur les conditions expérimentales de sa création.



Le processus d'érosion évoqué par cette visualisation (et celle de la figure 2.9) est révélé par une réduction intégrale.

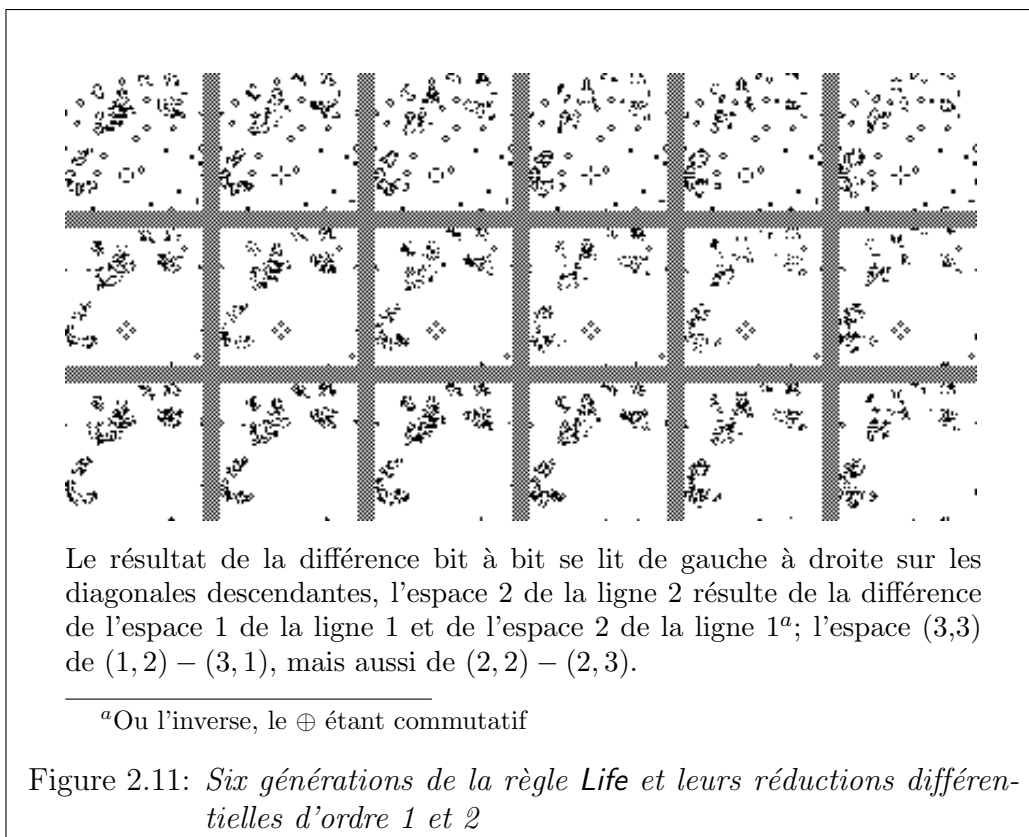
Figure 2.10: *D'autres vues sur la réduction de la figure 2.9 par pixmaps utilisant des colormaps différentes*

- Positionnement absolu et relatif du point d'espace courant.
- Test du point courant, relatif aux bornes.
- Accès au point d'espace passé.
- Application d'une fonction de transition de l'espace courant sur l'accumulateur.
- Échange du point d'espace courant et de l'accumulateur.

Ces primitives permettent la construction de procédures composites. Par exemple, pour enregistrer une séquence d'espaces d'indice de temps multiple de  $n$ , il faut, après initialisation de l'espace d'indice zéro et d'un *timer* à  $n$ :

- Appliquer la fonction transition du point courant sur l'accumulateur,
- Décrémenter le timer, et si zéro incrémenter le point courant,
- Échanger le point courant et l'accumulateur. Optionnellement, estamper le point courant avec le compteur d'application de la fonction de transition,
- Itérer tant que le point courant n'est pas le dernier,

Ces procédures doivent pouvoir être construites par l'utilisateur aussi simplement que possible, par l'intermédiaire d'un interpréteur.



### 2.2.3 Réductions du temps: sommes et différences

Il existe plusieurs types d'opération de réductions sur le temps. Les réductions des séquences de réductions d'espaces constituent un deuxième niveau de réduction arithmétique et statistique. On peut par exemple calculer la dimension fractale de l'évolution de n'importe quelle quantité associée à un espace.

Les réductions d'information intégrale<sup>23</sup> sur les espaces permettent de pas-

<sup>23</sup>La relation entre les notions classiques de dérivée et d'intégrale et le contenu informationnel nous pose problème. Dans notre contexte de production d'objets discrets et finis, les dérivés et les intégrales ne sont pas des valeurs limites mais des chaînes de bits. Le contenu informationnel est la taille de cette chaîne de bit, il ne se confond pas avec la quantité d'information. Une opération d'intégration discrète définie comme une somme sur une direction de l'espace temps cellulaire produit un espace cellulaire dont la taille en bits est inférieure à la taille de l'espace initial. Par extension, nous considérons toute opération de réduction du contenu informationnel par diminution du nombre de dimensions de l'espace cellulaire comme une intégration.

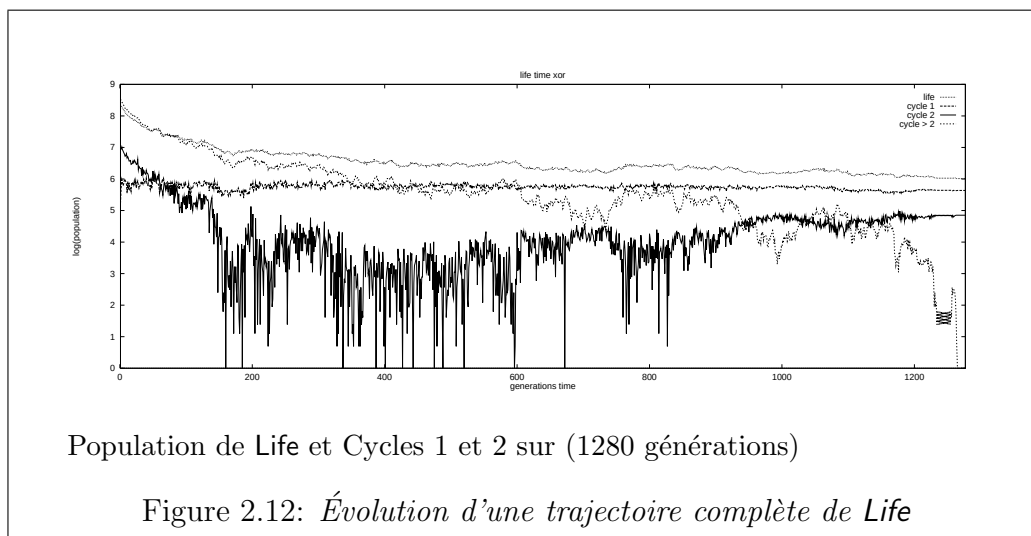
Une transformation différentielle (dont le prototype est le  $\oplus$  (xor) bit à bit ou la distance de Hamming chaîne à chaîne) ne réduit pas le contenu informationnel, mais le but

ser d'un ensemble d'espaces à un espace unique<sup>24</sup>. Par exemple un espace dont chaque cellule est la somme  $\sum_{i=0}^{i=n} t_i xy$  des valeurs de l'ensemble des cellules de coordonnées spatiales  $xy$  identiques, sur une séquence  $t$  de  $n$  espaces. La figure 2.9 montre une vue perspective d'un espace somme de 400 générations de la règle **Anneal**. L'espace est de taille  $64 \times 64$ . L'espace initial a une densité aléatoire  $\frac{1}{2}$ .

La figure 2.10 montre quatre visions pixmap du même espace utilisant 4 grey-map différentes.

Une autre réduction simple est le  $\oplus$  plan de bits à plan de bits de deux espaces, qui identifie les points d'espace modifiés d'un point de temps à son successeur. Appliqué à un AC tel que le jeu de la vie qui produit des patterns statiques ou de cycle 2 en nombres, cet opérateur permet d'éliminer les points d'espace de cycle 1 ( $t_{n-1} \oplus t_n$ ), ou 1 et 2 ( $t_{n-2} \oplus t_n$ ).

La première ligne de la figure 2.11 montre 6 générations de Life (de 100 à 106) pour un espace de  $64 \times 64$  cellules et un espace initial aléatoire de densité  $\frac{1}{2}$ , la deuxième ligne le résultat de l'élimination de cellules de cycle 1 et la troisième de l'élimination des cycles 1 et 2. Bien sûr, ces éliminations ne laissent pas intactes les zones non éliminées.



La figure 2.12 montre l'évolution sur 1280 générations des populations associées aux cycles 1 et 2, pour un espace de  $128 \times 128$  cellules (densité aléatoire initiale  $\frac{1}{2}$ )<sup>25</sup>.

poursuivi est a priori une réduction de la quantité d'information, ce qui nous amène à parler d'opération de réduction différentielle.

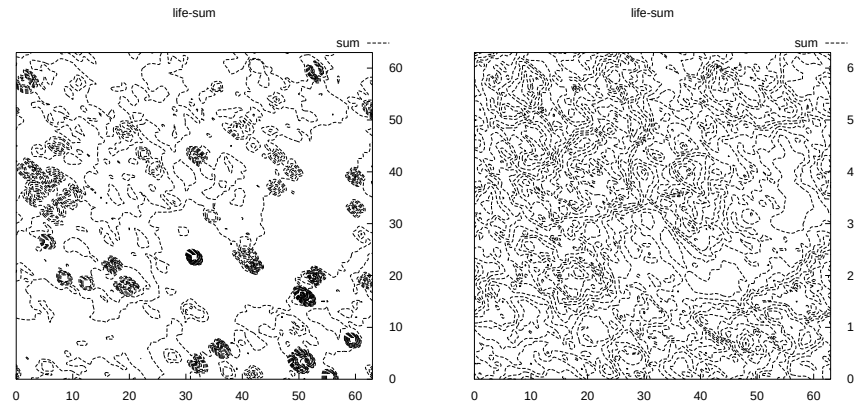
<sup>24</sup>La taille en bit des cellules de l'espace résultat peut être supérieure à celle des cellules des espaces arguments.

<sup>25</sup>Dans ce qui suit nous utilisons le logarithme pour normaliser les résultats

$\text{life} = \log(P_0)$ ,  $\text{cycle}^1 = \log(P_0 - P_1)$ ,  $\text{cycle}^2 = \log(|P_1 - P_2|)$  et  $\text{cycle}^{>2} = \log(P_2)$ .

Avec  $P_0$  pour la population standard,  $P_1$  pour la population moins les points d'espace de  $\text{cycle}^1 = t_{n-1} \oplus t_n$ , et  $P_2$  pour la population moins les points d'espace de  $\text{cycle}^{<3} = t_{n-2} \oplus t_n$ .

A la génération 1280, il ne reste plus que des cellules engagées dans un cycle 1 ou 2.



Une réduction intégrale du temps sur une séquence de 512 générations de Life (à gauche), et sur la séquence obtenu par réduction différentielle d'espace avec l'opérateur  $\oplus$ . Cette comparaison suggère l'existence pour Life d'un invariant d'échelle pour la densité d'occupation de l'espace cellulaire, après filtrage. Nous considérons cette question, ainsi que la relation entre cette question et les expériences de Bak (1993) sur l'auto-organisation critique de Life<sup>a</sup>, comme une source d'expériences futures.

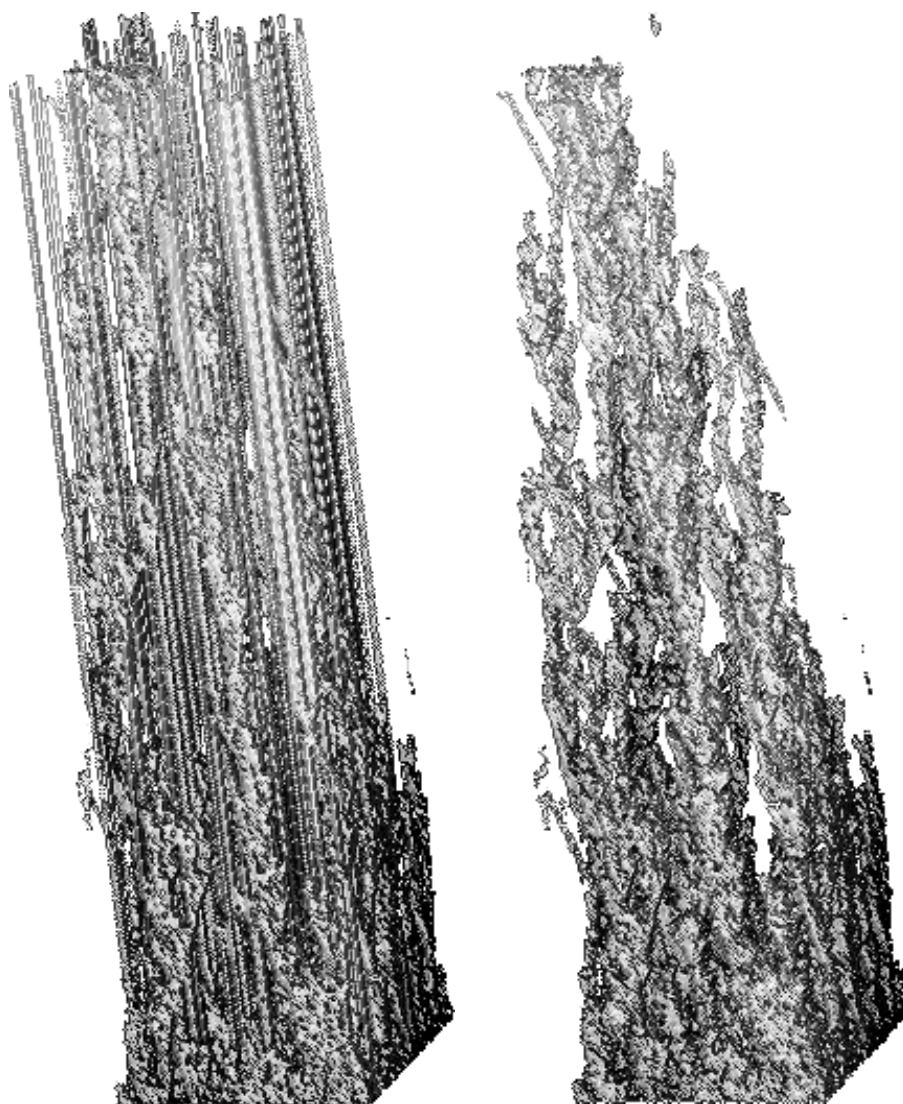
<sup>a</sup>Celle-ci portent sur la taille des avalanches (le nombre d'événements cellulaires élémentaire, i.e. de bit flip) résultant d'une perturbation aléatoire d'un cycle limite.

Figure 2.13: *Somme de 512 générations de Life et Life filtré*

La figure 2.13 montre les contours de la somme de 512 générations de Life (à gauche) et Life moins les points d'espace de cycle 1 et 2 (à droite).

Les tableaux 2.7 et 2.8 montrent le résultat de la commande `pgmtexture`<sup>[16]</sup> sur deux espaces somme de la figure 2.13.

La réalisation d'une procédure de mesure de la proportion des cellules actives engagées dans un point fixe ou un cycle de longueur 2 ou 3, au top level du Workbench, par combinaison d'objets et de primitives du noyau ou



Une visualisation 3D du temps de Life en partie gauche, et d'un temps de réduction différentielle d'espace de Life (cf. figure 2.13, page 54).

Figure 2.14: *Une autre vue de la figure 2.13*

| Angle                 | 0          | 45         | 90         | 135        | Avg        |
|-----------------------|------------|------------|------------|------------|------------|
| Angular Second Moment | 9.484e-04  | 8.288e-04  | 9.534e-04  | 8.279e-04  | 8.896e-04  |
| Contrast              | 5.979e+02  | 6.454e+02  | 6.021e+02  | 6.430e+02  | 6.221e+02  |
| Correlation           | 4.875e-01  | 4.476e-01  | 4.854e-01  | 4.497e-01  | 4.675e-01  |
| Variance              | 5.843e+02  | 5.852e+02  | 5.860e+02  | 5.852e+02  | 5.852e+02  |
| Inverse Diff Moment   | 1.685e-01  | 1.489e-01  | 1.684e-01  | 1.488e-01  | 1.587e-01  |
| Sum Average           | 5.204e+01  | 5.204e+01  | 5.204e+01  | 5.204e+01  | 5.204e+01  |
| Sum Variance          | 4.229e+03  | 4.186e+03  | 4.231e+03  | 4.188e+03  | 4.209e+03  |
| Sum Entropy           | 2.104e+00  | 2.101e+00  | 2.104e+00  | 2.101e+00  | 2.103e+00  |
| Entropy               | 3.324e+00  | 3.368e+00  | 3.322e+00  | 3.369e+00  | 3.346e+00  |
| Difference Variance   | -5.557e-06 | -4.955e-06 | -5.580e-06 | -4.941e-06 | -5.258e-06 |
| Difference Entropy    | 1.398e+00  | 1.438e+00  | 1.395e+00  | 1.438e+00  | 1.417e+00  |
| Meas of Correlation-1 | -1.626e-01 | -1.383e-01 | -1.640e-01 | -1.379e-01 | -1.507e-01 |
| Meas of Correlation-2 | 6.666e-01  | 6.273e-01  | 6.687e-01  | 6.266e-01  | 6.473e-01  |

Caractéristiques texturales de l'espace de la figure 2.13 (version non filtrée): exemple de réductions statistiques sur un espace obtenu par réduction intégrale du temps.

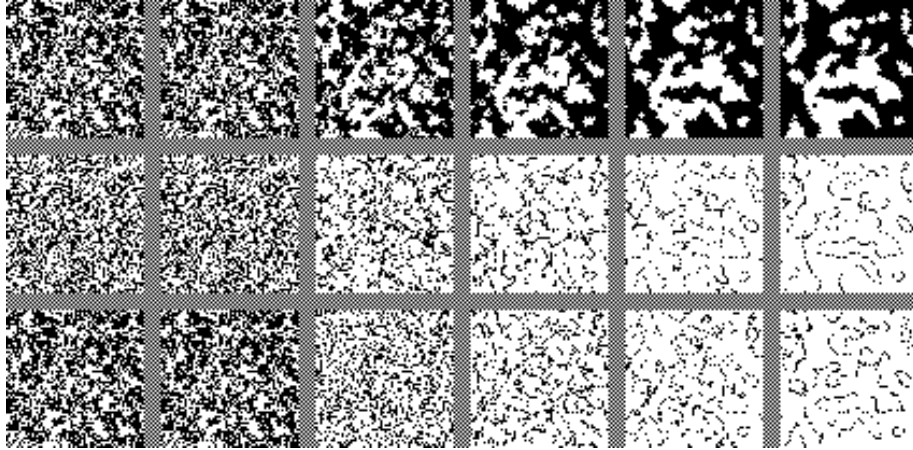
Tableau 2.7: *pgmtexture*<sup>[16]</sup> sur 2.13 (gauche)

| Angle                 | 0          | 45         | 90         | 135        | Avg        |
|-----------------------|------------|------------|------------|------------|------------|
| Angular Second Moment | 1.461e-03  | 1.311e-03  | 1.486e-03  | 1.307e-03  | 1.391e-03  |
| Contrast              | 9.626e+01  | 1.047e+02  | 9.486e+01  | 1.051e+02  | 1.002e+02  |
| Correlation           | 3.639e-01  | 3.078e-01  | 3.733e-01  | 3.055e-01  | 3.376e-01  |
| Variance              | 7.666e+01  | 7.666e+01  | 7.668e+01  | 7.666e+01  | 7.666e+01  |
| Inverse Diff Moment   | 1.784e-01  | 1.600e-01  | 1.816e-01  | 1.589e-01  | 1.697e-01  |
| Sum Average           | 3.110e+01  | 3.110e+01  | 3.110e+01  | 3.110e+01  | 3.110e+01  |
| Sum Variance          | 1.068e+03  | 1.059e+03  | 1.069e+03  | 1.059e+03  | 1.064e+03  |
| Sum Entropy           | 1.752e+00  | 1.746e+00  | 1.753e+00  | 1.745e+00  | 1.749e+00  |
| Entropy               | 2.911e+00  | 2.942e+00  | 2.906e+00  | 2.943e+00  | 2.926e+00  |
| Difference Variance   | 6.846e-05  | 6.080e-05  | 6.999e-05  | 6.032e-05  | 6.489e-05  |
| Difference Entropy    | 1.274e+00  | 1.311e+00  | 1.267e+00  | 1.312e+00  | 1.291e+00  |
| Meas of Correlation-1 | -5.828e-02 | -3.774e-02 | -6.176e-02 | -3.735e-02 | -4.878e-02 |
| Meas of Correlation-2 | 4.004e-01  | 3.271e-01  | 4.111e-01  | 3.255e-01  | 3.661e-01  |
| Max Correlation Coeff | 2.764e-03  | 7.766e-03  | 7.918e-03  | 2.390e-03  | 5.210e-03  |

Caractéristiques texturales de l'espace de la figure 2.13 (version filtrée): à partir d'une réduction intégrale d'une séquence temporelle de réduction différentielle d'espaces.

Tableau 2.8: *pgmtexture*<sup>[16]</sup> sur 2.13 (droite)

de la librairie d'application, constitue un objectif exemplaire de la souplesse d'usage recherchée.



Le mode de production de cette séquence est identique à celui de la figure 2.11, son observation<sup>a</sup> suggère comme pour Life, la recherche d'invariants d'échelle<sup>b</sup>

<sup>a</sup>Plus exactement, l'observation interactive de nombreuses séquences pour une taille d'espace et un nombre de générations plus important.

<sup>b</sup>Voir § 4, page 99.

Figure 2.15: Six générations de la règle *Anneal* et leurs réductions différentielles d'ordre 1 et 2

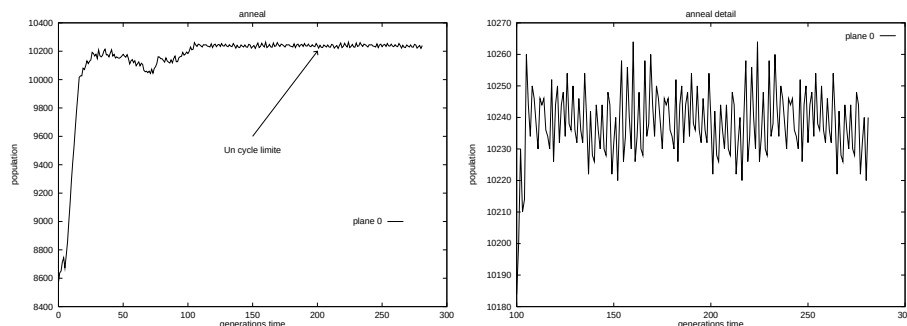
La figure 2.15 montre le même traitement appliqué aux générations 2 à 12, par pas de 2, sur la fonction *anneal* dans les mêmes conditions d'espace.

Nous avons expérimenté des réductions différentielles du temps sur des séquences de la réduction intégrale élémentaire d'espace que constitue le nombre de bits d'un plan cellulaire. Ces réductions sont définies par une différence locale des populations:

$$\text{Pop}_i = \text{Pop}(\text{Space}_i) - \text{Pop}(\text{Space}_{i-1})$$

Nous cherchons à établir la relation entre de telles séquences et des séquences de réductions intégrales (population) d'espaces; ces espaces étant obtenus par la réduction différentielle spatiale élémentaire résultant de l'application du  $\oplus$  plan à plan:

$$\text{Pop}_i = \text{Pop}(\text{Space}_i \oplus \text{Space}_{i-1})$$



Le plot de gauche est un agrandissement sur le cycle limite atteint par cette trajectoire de *anneal*. La découverte de l'existence de cycles limite de grande taille<sup>a</sup> fut une surprise<sup>b</sup>. Voir § 4, page 99.

<sup>a</sup>Du même ordre de grandeur que l'arête de l'espace, ici 127.

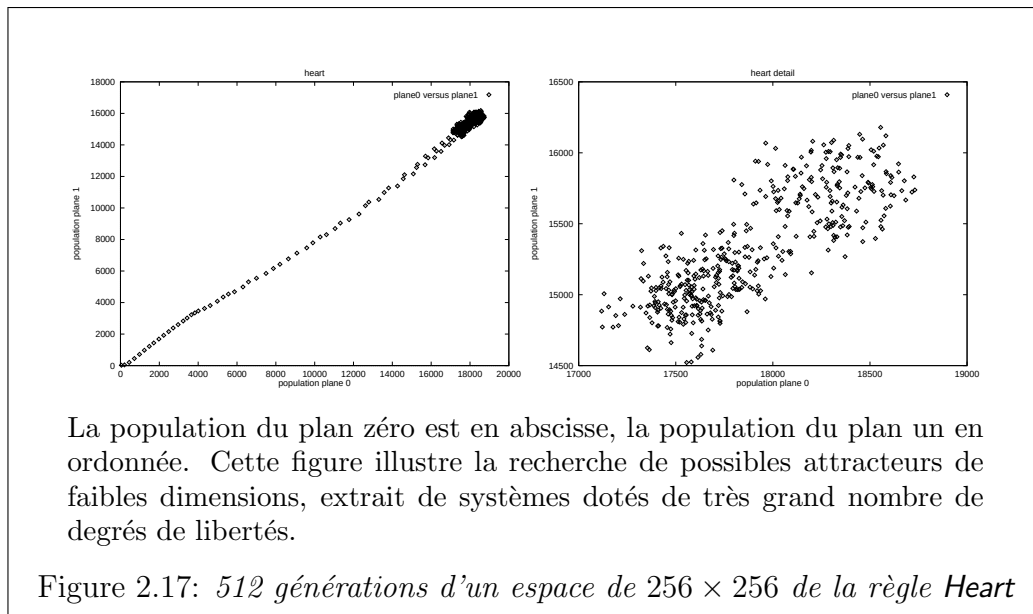
<sup>b</sup>Ni la littérature mentionnant la règle: **M46789** pour Vichniac (1986), **Lava Lamp** (une version 3D) pour Pickover (1992), **Anneal** pour Toffoli et Margolus (1987); ni une expérimentation restreinte ne permettent d'attendre autre chose qu'un point fixe ou un cycle limite court (i.e. d'une taille indépendante de la taille de l'espace).

Figure 2.16: *La population de 5640 générations de la règle Anneal*

Les réductions différentielles associatives sont réalisables par itération cumulée d'un opérateur à nombre d'arguments fixe sur une fenêtre de temps. On peut alors les considérer comme l'application d'une fonction de transition sur un espace composite.

Par exemple, comment transformer une séquence de plan de bits d'un espace (ou copier cette transformation en conservant la séquence initiale), en séquence des différences plan à plan en utilisant l'opérateur  $\oplus$ . La fonction de transition **Xor** a deux arguments d'un bit, et un bit de résultat. Ses paramètres formels sont liés aux variables de connexion *past* et *present* (au lieu de *east* et *west* par exemple), et son résultat est lié à la variable de connexion *present* (au lieu de *center*). Lors de l'application de la fonction de transition l'espace résultant est stocké dans l'accumulateur. Un séquenceur peut alors soit copier l'accumulateur dans le point d'espace courant d'un temps accumulateur de la transformation du temps courant, soit dans le point d'espace passé du temps courant.

Plus généralement, si l'on considère le temps comme un espace de dimension supérieure, on peut réaliser des projections, ou extraire des coupes de cet espace de dimension supérieure sur un espace de dimension inférieure. Par



exemple, une ligne d'espace de dimension 2 permet d'extraire une tranche (un espace de dimension 2) d'un temps d'espace de dimension 2.

#### 2.2.4 Transformations du temps: l'espace-temps

Les transformations sur le temps résultent soit d'une combinaison extraction/construction qui permet de réaliser des films de coupes, soit de l'application sur chaque point d'espace du temps d'une transformation d'espace, soit de l'application d'une transformation d'espace sur un temps considéré comme un espace de dimension supérieure.

#### 2.2.5 Visualisation du temps: séquences de réductions et projections de plongements

Les visualisations du temps utilisent les techniques évoquées à propos de la visualisation des espaces. Celles-ci sont pilotées par un séquenceur qui combine les primitives de manipulation du temps et les primitives de visualisation de l'espace. Les mêmes outils sont utilisés pour visualiser les espaces résultant de réduction du temps.

La visualisation des séquences de réductions d'espace utilise des ploters pour présenter des graphes temps/variable, ou multi-variables (population plan 1, population plan 2 e.g.), ou toutes combinaisons de variables, de réductions et d'expressions autorisées par les outils off-line utilisables.

La figure 2.16 (anneal) présente l'évolution de la population de la règle **Anneal** pendant 5640 générations échantillonnées chaque 20 générations. La taille de l'espace est  $127 \times 127$ . On peut y voir un cycle limite.

La figure 2.16 (anneal détail) présente un agrandissement sur le cycle. La détection et la mesure de cycles limites sur les populations sont des exemples de réductions appliquées sur une séquence de réduction.

La figure 2.17 (heart) présente l'évolution de la population du plan 0 contre l'évolution de la population du plan 1 de la règle **Heart** pendant 512 générations d'un espace de  $256 \times 256$ .

On peut y voir le découplage des deux variables. La figure 2.17 (heart détail) est un agrandissement sur le découplage.

## 2.3 Les lois: la fonction de transition

La *fonction de transition* est l'élément central de notre problématique des environnements de programmation pour automates cellulaires. Les objets présentés précédemment ne sont pas particulièrement spécifiques d'un environnement de développement d'AC. Ils pourraient convenir à un environnement de production et de traitement d'images animées quelconques. Ils constituent des extensions plus ou moins évidentes d'outils existants<sup>26</sup> ou sont basés sur des impératifs d'architectures et de gestion de ressources totalement inévitables.

La fonction de transition concentre les problèmes de gestion de ressources en mémoire et en CPU, et le problème du pouvoir expressif de la spécification du réseau d'interconnexion cellulaire et de la table de transition.

### 2.3.1 Structure des lois: voisinage et lookup-table

La fonction de transition est la structure de contrôle de l'AC. Une fonction de transition locale reçoit en argument un fragment d'espace (un ensemble de cellules ordonnées)<sup>27</sup>. Il existe un tel fragment pour chaque cellule de l'espace. La fonction retourne le nouvel état de la cellule associée à ce fragment (la cellule cible). En général, la cellule cible est un des éléments du fragment. Si le fragment n'est pas réduit à la cellule cible, les fragments ne sont pas disjoints et l'information peut circuler de cellule en cellule.

Sur une machine séquentielle, un espace temporaire stocke le nouvel état des cellules cibles et l'espace temporaire devient le nouvel état d'espace après l'application de la fonction de transition locale à chaque cellule de l'espace courant<sup>28</sup>. Ce mécanisme de simulation d'une architecture SIMD dotée d'un réseau d'interconnexion homogène ne présente pas de difficulté algorithmique mais l'efficacité et la souplesse du système dépendent de l'architecture choisie pour sa réalisation.

Il faut une méthode de *balayage* et de *collecte* des fragments d'espace pour chaque cellule cible, d'*association*<sup>29</sup> des paramètres formels de la fonction de transition aux cellules du fragment et d'*invocation* de la fonction. La fonction qui reçoit en argument un ensemble d'espaces, un réseau d'interconnexion et

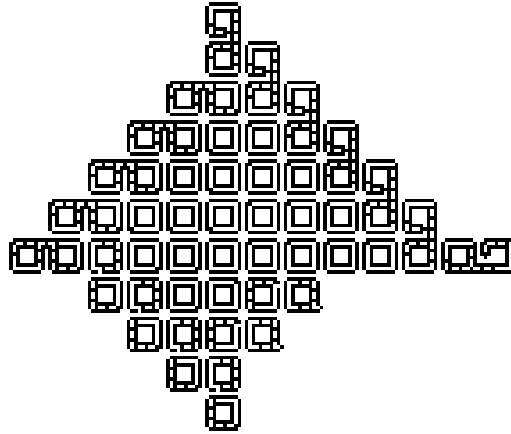
---

<sup>26</sup>Initialement nous pensions même pouvoir utiliser un des outils existants (dans le domaine de l'animation interactive d'images), auquel des fonctions spécifiques d'application de fonctions de transitions auraient pu être ajoutées.

<sup>27</sup>Encore appelée voisinage.

<sup>28</sup>Avec la contribution du séquenceur.

<sup>29</sup>Encore appelé liaison ou *binding*.



la table de la règle `loop` qui a produit cette image vient de `cellsim`. Le source servant à produire la table ne faisant pas partie de `cellsim`<sup>a</sup> Un examen sommaire du code source de `cellsim` permet de déterminer l'ordre choisi pour l'index, et un programme de permutation des entrées de la table en fonction de cet ordre permet la conversion de la table au format de notre moteur.

<sup>a</sup>La reconstruction d'un tel source ne pose bien sûr pas de problème, mais nous souhaitons user des propriétés des tables.

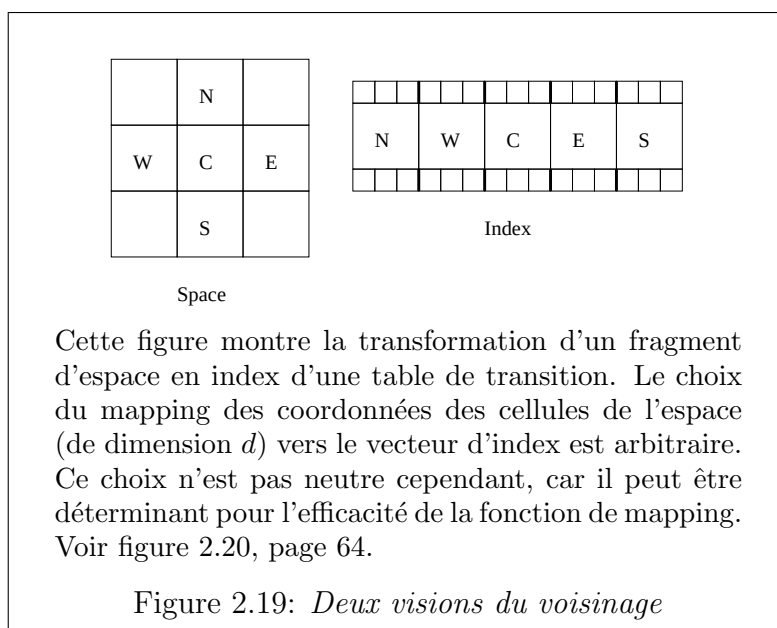
Figure 2.18: *Une table importée pour la génération 1024 de l'automate auto-reproducteur de Langton*

une fonction de transition, est la *fonction d'application*<sup>30</sup>.

Le choix du mécanisme d'invocation conditionne la méthode de liaison. La *méthode* la plus *rapide* d'*invocation* de *fonction* est l'accès par *table*. On construit une table des valeurs de la fonction pour l'ensemble des valeurs possibles de ses arguments et chaque élément de l'ensemble des valeurs est considéré comme un indice d'accès à la table. La liaison des valeurs d'appel de la fonction se ramène à la concaténation des valeurs des cellules du fragment d'espace et le calcul du nouvel état de la cellule cible à un accès mémoire indirect.

Les avantages sont évidents. Les fonctions de transitions peuvent être réalisées dans n'importe quel langage, sans souci de performance. L'opération de mise à jour d'une cellule est réduite à un nombre minimum d'instructions. Les inconvénients sont également évidents. Les fonctions réalisables sont limitées par la taille des tables (en nombre d'entrées) qui est une fonction

<sup>30</sup>Encore appelée moteur cellulaire.



exponentielle de la taille des arguments (produit du nombre de bits  $k = \log_2(K)$  codant le nombre  $K$  d'états par cellules et du nombre  $r$  de cellules du fragment d'espace). La taille  $T$  des tables exprimée en bytes dépend de  $k$  exprimé en bytes,  $T = \text{byte}(k)2^{kr}$ . Le rendement de la contraction de la table au strict nombre de bits nécessaires est proportionnel au rapport du nombre de voisins et du nombre d'états. Un espace de dimension  $d = 3$  doté du voisinage complet des  $r = 26$  cellules distantes de 1 unité de la cellule cible (cf. § 2.1, page 34) et de  $k = 1$  bit d'état requiert une table de 128 mégabytes sans contraction et 16 mégabytes avec contraction.

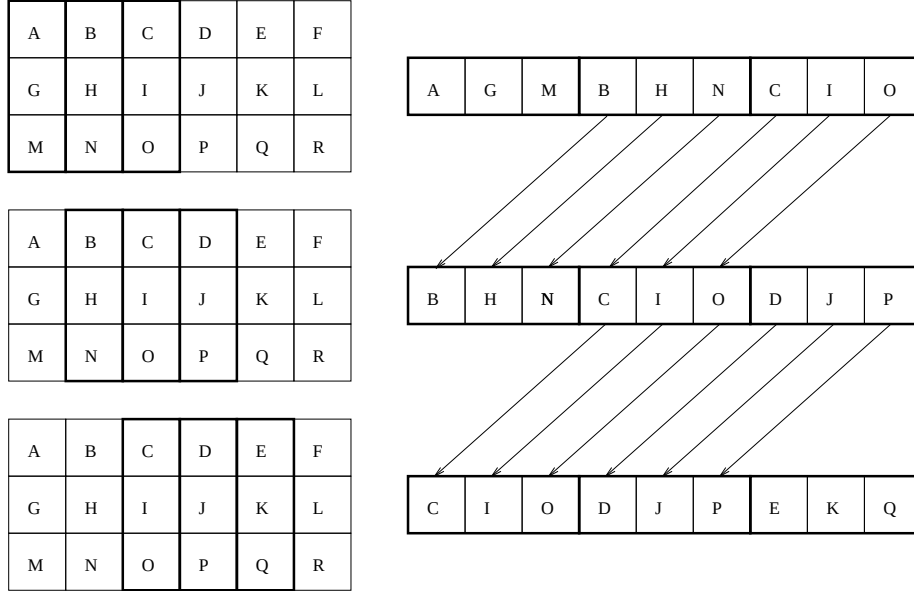
Nous pensons que les avantages sont plus grands que les inconvénients. Le nombre d'AC accessibles, même limité, reste énorme, et l'étude des AC concerne pour une bonne part les plus simples d'entre eux. De plus il est possible de créer des fonctions complexes en combinant<sup>31</sup> l'application de tables de fonction simple.

### 2.3.2 Opérations: plasticité des tables de transitions

Un autre avantage des tables est de permettre très simplement la génération ou la transformation de fonctions. Un paramètre élémentaire de génération de fonctions est par exemple donné par la densité de remplissage de la table<sup>32</sup>.

<sup>31</sup>Voir § 2.3.3, page 65.

<sup>32</sup>Il s'agit du paramètre  $\lambda$  Langton (1992), d'une richesse étonnante (cf. § 3.3, page 91) puisqu'il est directement associé à la classe de comportement dynamique de la fonction.



Cette figure montre comment pipeliner la fonction de mapping pour un voisinage de Moore, réduisant l'obtention d'un nouvel index à 1 shift et 3 copies, à la place des 9 copies nécessaires en l'absence de pipeline.

Figure 2.20: *Trois étapes successives de collecte de voisinage, illustrant le principe du pipeline de collecte mis en œuvre par la fonction d'application.*

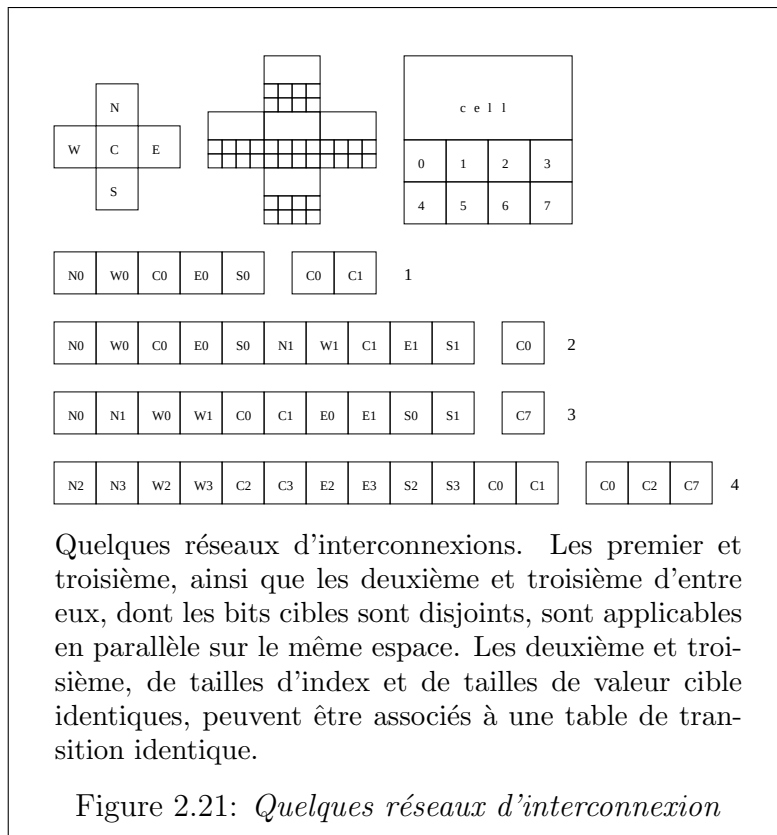
En fait, une fonction assimilée à un vecteur d'états devient un objet cible de multiples réductions et transformations au même titre qu'un espace de dimension 1. Des attributs comme la tranquillité<sup>33</sup> ou l'isotropie impliquent, cependant, une connaissance du mapping espace / index du voisinage. Le simple comptage des accès aux entrées de la table pour une transition (ou cumulé sur le temps) fournit une information statistique relativement complexe<sup>34</sup> sur le point d'espace précédent pour un coût CPU négligeable<sup>35</sup>. On peut encore citer les opérateurs de bruitage (ou mutation) sur table et de cross-over sur deux tables, qui permettent la réalisation d'algorithmes génétiques pour l'exploration de l'espace des fonctions, ou enfin l'utilisation possible de réseaux

<sup>33</sup>Un voisinage dont toutes les cellules sont à zéro (un état conventionnel nul), donne une cellule cible également à zéro.

<sup>34</sup>L'histogramme de l'occurrence de toutes les combinaisons d'espaces possibles sur un fragment d'espace de la taille du voisinage, qui peut servir à calculer l'entropie de l'espace.

<sup>35</sup>Au prix néanmoins d'une consommation mémoire  $M$  pénalisante pour les plus grandes tables  $M = \log_2(SN)T$ , en pratique  $2T$  ou  $4T$  bytes.

### 2.3. LES LOIS: LA FONCTION DE TRANSITION

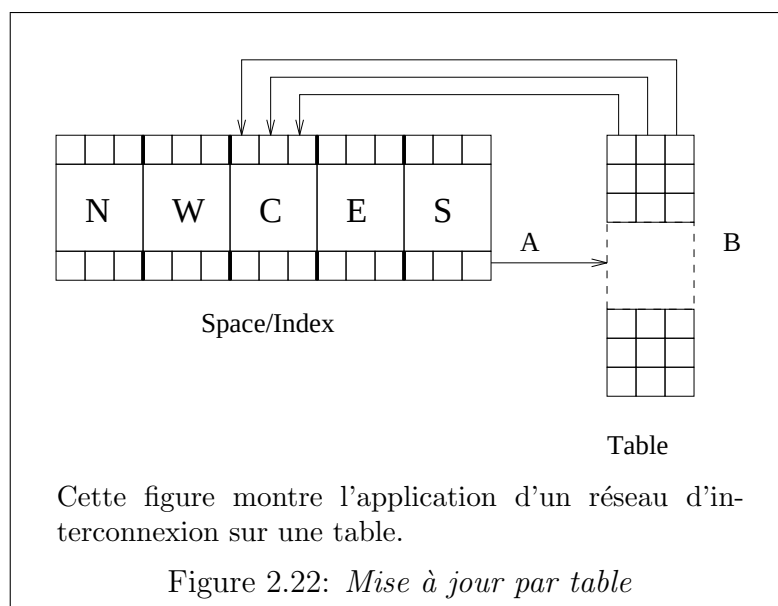


de neurones sur une table.

### 2.3.3 Composition: les chemins de données

La coordination de plusieurs tables nécessite la mise au point d'un système de liaison paramètres / valeurs pour le multiplexage spatial des tables et d'un séquenceur pour le multiplexage temporel.

La nécessité d'adresser rapidement chaque cellule de l'espace sur des machines adressables au niveau du byte, conduit au choix d'une organisation en plan de vecteur de bits (tableau de bytes) plutôt qu'en vecteur de plan de bits. Le réseau d'interconnexion est une liste de couples cellule / bit spécifiant les coordonnées spatiales de chacun des bits constituant l'index d'accès à la table, associé à une liste des bits cibles de la valeur de l'entrée dans la table.

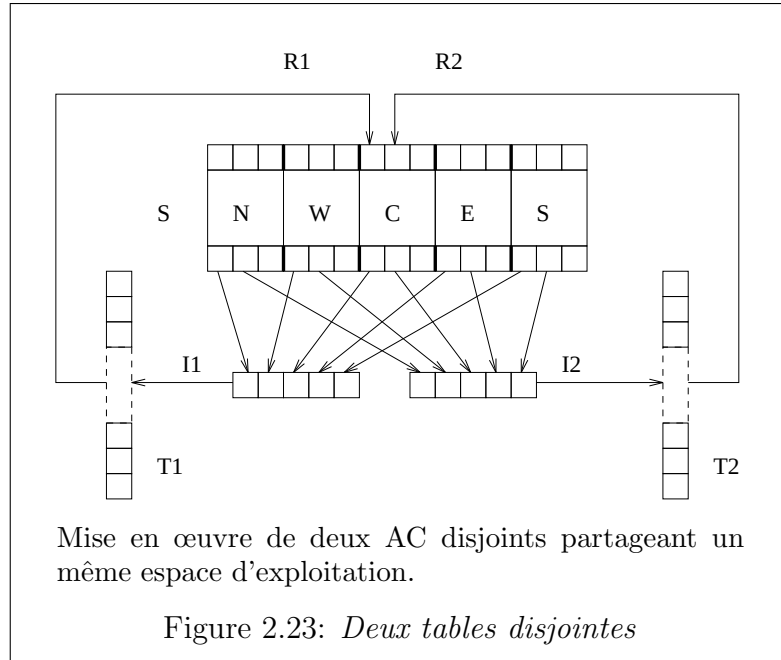


### 2.3.4 Voisinages spéciaux: nécessaire compromis ou généralisation?

On remarque sur la figure 2.24 la présence d'un élément (X) de voisinage non issu de l'espace. La fonction d'application peut fournir une valeur fonction du temps ou de l'espace utilisable comme élément de l'index. Ce type d'extension à la notion de voisinage peut sembler contraire à la définition que nous avons donné d'un AC. Même si les AC qui nous intéressent sont bâtis par combinatoire d'AC simples, il existe toujours un couple *espace initial*, *fonction de transition* permettant de calculer et de diffuser une valeur globale quelconque de l'espace ou du temps<sup>36</sup>. Les éléments de voisinages spéciaux sont des accélérateurs d'accès à ces valeurs. Certains d'entre eux sont simplement des économiseurs d'espace. Par exemple, au lieu d'utiliser un plan mémoire pour stocker un pattern régulier permettant à l'AC de connaître la parité des coordonnées d'une cellule<sup>37</sup>, la fonction d'application maintient un bit de parité par dimension d'espace. D'autres permettent de réduire la taille d'une table en fournissant un compteur ou un timer syn-

<sup>36</sup>Ou si l'on préfère il est possible de simuler une architecture de von Neuman dans un espace cellulaire.

<sup>37</sup>Ce qui revient à organiser un multiplexage\* spatial des règles, qui pourrait également être réalisé en accédant à la table de transition à travers une indirection supplémentaire via un index de type spatial d'une cellule. \*Paramétrage des règles selon leurs positions dans l'espace.



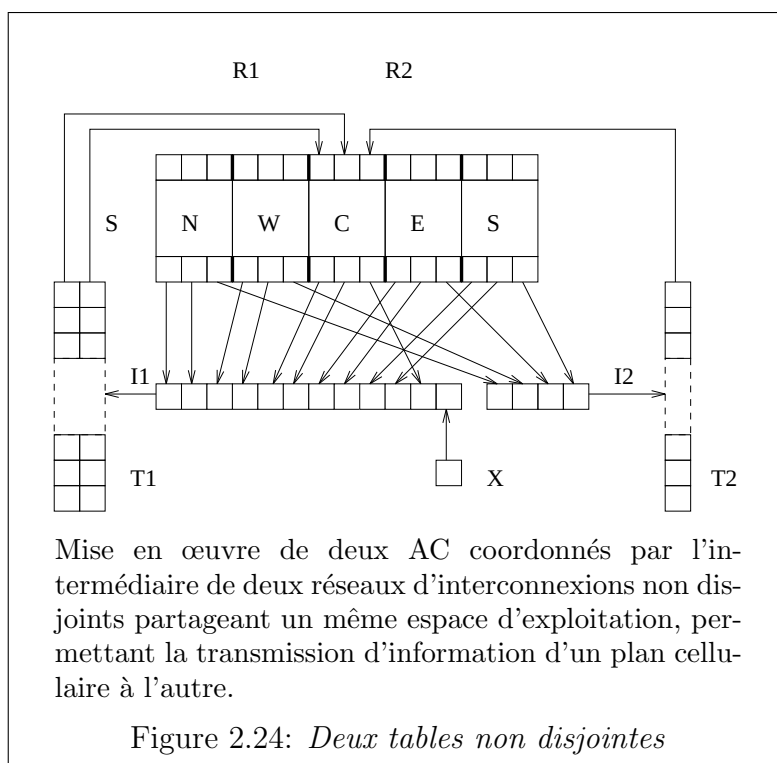
chrone sur l'espace<sup>38</sup>. Enfin certaines réductions de l'espace, dont le calcul nécessiterait un protocole cellulaire complexe, sont évidemment utiles, tel le OR de l'ensemble des valeurs d'un plan de bit qui permet à chaque cellule de savoir que ce plan est à zéro pour toutes les cellules de l'espace.

La fonction d'application peut également calculer une valeur locale, permettant ainsi une réalisation par tables de fonctions autrement inaccessibles. Si la fonction de transition opère une réduction simple sur l'index, celle-ci peut être confiée à la fonction d'application. Si l'opérateur de réduction est commutatif, associatif, et possède un opérateur inverse, la réduction peut être pipelinée. Par exemple, un voisinage de  $r$  cellules à  $k$  bits d'état donnerait une table de taille  $2^{rk}$ . La distribution de l'opérateur d'addition sur les  $r$  cellules sources permet de réduire la taille de la table à  $r2^k$ . Dans ce cas pour  $k = 8$ , la taille de la table passe de  $2^{72}$  bytes à  $2^{12}$  bytes, et le pipeline de liaison remplace une opération de décalage<sup>39</sup> par une soustraction, et trois concaténations (un shift et un or) par trois additions.

La distribution de l'opérateur d'addition sur les  $r - 1$  cellules sources (privées de la cellule cible dont la préservation est en général souhaitable pour obtenir une gamme d'AC accessibles plus vaste) associée à la concaténation de la cellule cible, donnerait une taille  $M = 2^{\log_2((r-1)2^k)+k} = 2^{\log_2(r-1)+2k} =$

<sup>38</sup>Dans ce cas, la table contient plusieurs fonctions de transition distinctes qui alternent cycliquement, et il est sans doute plus avantageux d'utiliser un séquenceur externe.

<sup>39</sup>A condition de disposer d'une arithmétique sur mots de 72 bits.

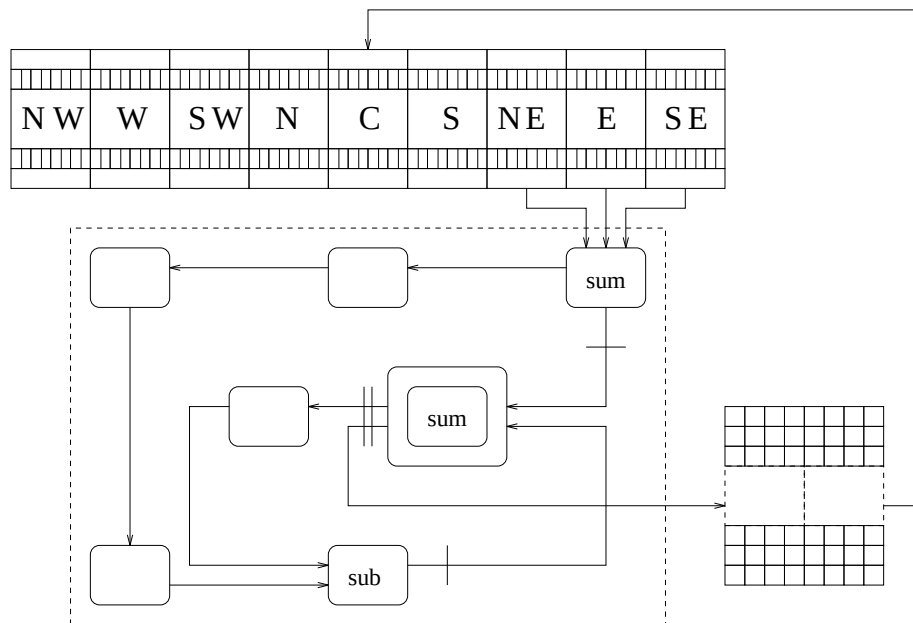


$(r - 1)2^k$ . Cette architecture opère l'addition pipelinée des cellules du voisinage de Moore moins la cellule cible, et la concaténation de cette somme avec la valeur de la cellule cible. Dans ce cas, pour  $k = 8$ , la taille de la table passe à  $2^{19}$  bytes.

D'autres règles locales, telles les minimums et maximums des valeurs cellulaires, pourraient être calculées par la fonction d'application. L'association d'AC dont les cellules possèdent une variété de densité large pour une variété de comportements réduite, à des AC de nature opposée, est un de nos objectifs.

En plus d'éléments de voisinages spéciaux, la fonction d'application peut fournir toutes sortes de voisinages non standards, destinés à une utilisation dans le cadre du séquenceur, ou à un usage généralisé du mécanisme des tables de transitions. Nous avons déjà évoqué (cf. § 2.2.3, page 58) un voisinage à deux bits associé à la table *Xor* pour la réalisation d'une transformation sur le temps. Le séquençement d'une série de voisinages à deux cellules, balayant un espace régulier standard en pivot sur la cellule cible, permet d'accumuler une opération quelconque sans faire appel à un élément de voisinage spécial (non spatial) pour cette réduction<sup>40</sup>. Une table à deux arguments,

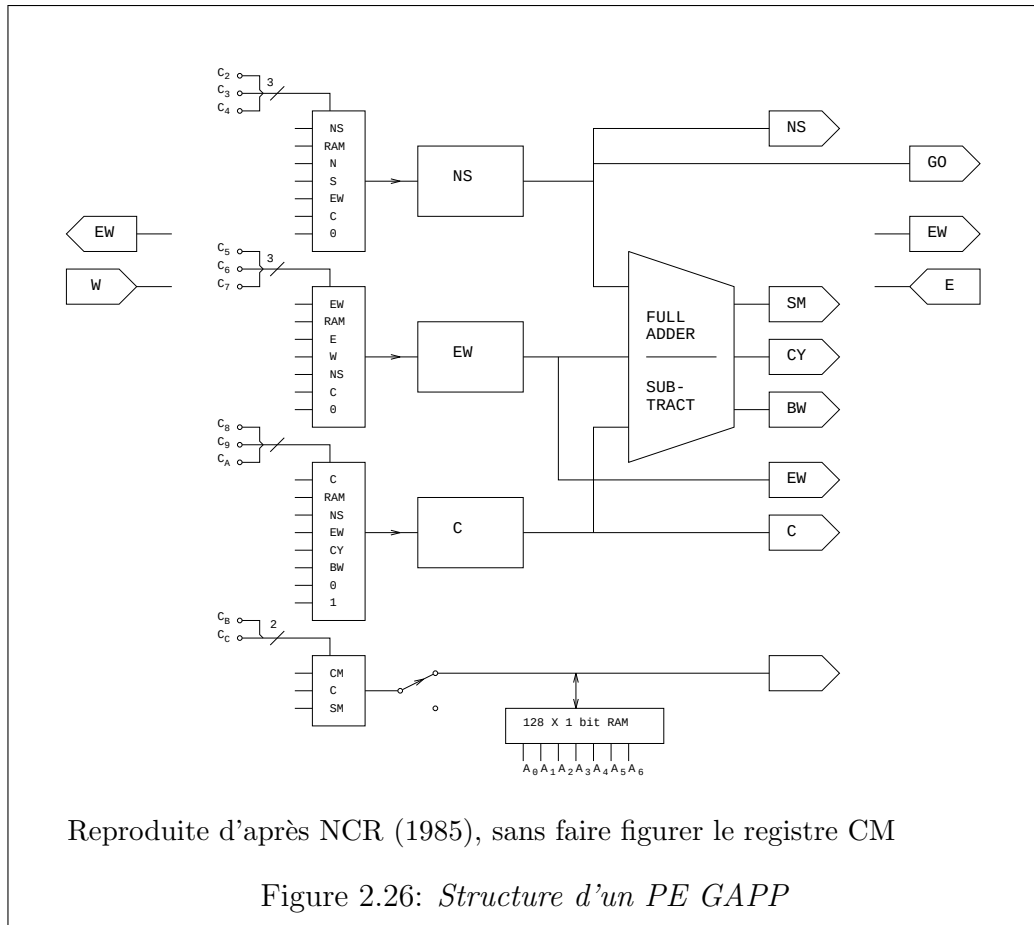
<sup>40</sup>Le nombre d'opérations nécessaires à la fabrication d'un index est réduit mais le ba-



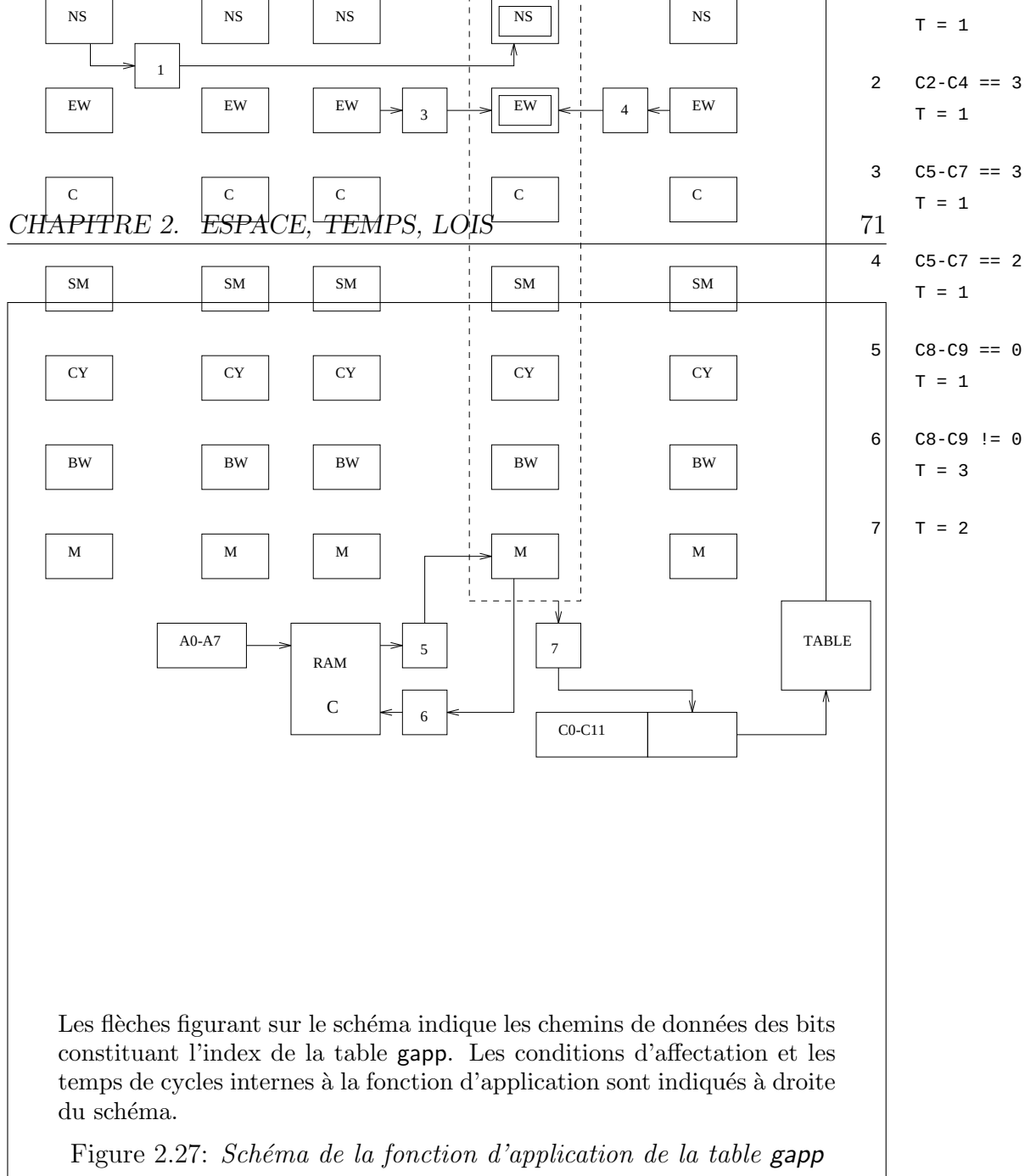
utilisable comme accumulateur, a deux entrées de tailles différentes et une sortie de taille égale à l'entrée accumulatrice. Cette table peut être utilisée par une fonction d'application pour le calcul d'une valeur locale en place de la concaténation pour la fabrication de l'index.

Dans le cas d'une généralisation de l'usage de tables à une combinatoire (multiplexage spatial et temporel) d'usage de tables non disjointes sur un même espace, le gain escompté n'est pas toujours un gain de mémoire, mais également du pouvoir d'expression des structures de données associées à cette combinatoire. Par exemple, il n'est pas nécessaire de considérer le OR de tous les bits d'un plan cellulaire comme un élément de voisinage spécial dont le calcul est laissé au soin de la fonction d'application. Il est possible d'utiliser une variable spéciale, un accumulateur (une variable scalaire, con-

layage de la mémoire est multiplié et la présence possible d'un pipeline pour les voisinages disposant d'un plus grand nombre de cellules réduit cet avantage.



trairement aux variables cellulaires), comme un élément de cellule cible, et non plus seulement d'index. Le binding de la table de la fonction binaire OR, ou de toute autre fonction binaire, aux arguments CENTRE et ACU et au résultat ACU, et le report au séquenceur de la tâche de l'application de cette fonction sur l'espace (ou plus probablement à la capacité de la fonction d'application d'appliquer plusieurs AC sur un même pipeline) permet d'utiliser la machinerie cellulaire à son propre service. Bien sûr, dans ce cas, seul le résultat de la réduction appliquée sur le plan précédent est accessible (à moins de faire deux passes), et ce type d'approche n'est pas possible sur une architecture SIMD réelle.



### 2.3.5 Utilisation: construction, composition, mélange et séparation

Si l'on veut considérer l'application de tables sur des espaces, lesquels ne sont pas toujours confondus avec l'espace cellulaire à proprement parler, comme la structure de contrôle élémentaire<sup>41</sup> de l'univers cellulaire, celle-ci doit être munie d'une structure de données supportant la construction, le mélange et

<sup>41</sup>En tout cas dans un environnement SISD. Nous choisissons délibérément d'ignorer les problèmes que poserait le portage du système sur une architecture SIMD. Les AC étant quant à eux évidemment indépendants de l'architecture. Nous pensons que l'usage d'une architecture SIMD simulée reposant sur une organisation pipelinée de la transformation de plans mémoires par tables permet une utilisation efficace des ressources d'une architecture von Neumann, applicable à une vaste classe de problèmes généraux.

```

typedef struct {
    Bit_Field input;
    Bit_Field output;
    Bit_Field instruction;
    Bit_Field iparts;
    int* alu;
    int size;
    unsigned char* pe_tab;
    void (**ctl_tab)();
    char** ctl_tab_names;
} Gapp;

Gapp *Gapp_new(Gapp *zis);
void Gapp_mk_pe_table(Gapp *zis);
void Gapp_mk_ctl_table(Gapp *zis);

```

La simplicité de l'implémentation apparaît clairement. Les champs *Bit\_Field* contiennent les parties déclaratives de l'architecture. Les deux méthodes associées à cet objet sont la fabrication de la table de transition et la fabrication de la table de sélection d'une fonction d'application qui réalise l'association instruction / fonction d'application.

Figure 2.28: *La structure de données de description de l'architecture d'un PE GAPP*

la composition.

- La construction des tables permet l'association d'une fonction à ses paramètres statiques. Les paramètres statiques d'une fonction sont les paramètres de la fonction qui ne sont pas associés à un des éléments du voisinage cellulaire, mais permettent de généraliser le comportement de la fonction. Il s'agit par exemple de la valeur de timers locaux (propres à chaque cellule) ou de valeurs de seuil déclenchant des portions de code de la fonction.
- La composition des tables permet la construction d'une liste de fonctions travaillant en parallèle sur un même espace. On peut obtenir le même effet en utilisant un séquenceur temporel, mais au détriment de l'usage du pipeline<sup>42</sup>. De plus, la disponibilité d'un objet spécifique de composition de fonctions permet l'application d'opérateurs de mélange et de séparation sur les tables. On peut enfin imaginer une généralisation de cette liste à un arbre, permettant l'utilisation de résultats intermédiaires d'application de table comme élément de voisinage d'une autre table.

<sup>42</sup>Voir § 40, page 68.

```

inline void gapp_pe(int* a, int* i, int* o) {
    int nst[] = { i[GAPP_NS], i[GAPP_RAM], i[GAPP_NOS],
                  i[GAPP_NOS], i[GAPP_EW], i[GAPP_C], 0 };
    int ewt[] = { i[GAPP_EW], i[GAPP_RAM], i[GAPP_EOW],
                  i[GAPP_EOW], i[GAPP_NS], i[GAPP_C], 0 };
    int ct[] = { i[GAPP_C], i[GAPP_RAM], i[GAPP_NS], i[GAPP_EW],
                 i[GAPP_CY], i[GAPP_BW], 0, 1 };
    int ramt[] = { i[GAPP_RAM], 0, i[GAPP_C], i[GAPP_SM] };
    int tmp;

    o[GAPP_NNS] = nst[i[GAPP_NSI]];
    o[GAPP_NEW] = ewt[i[GAPP_EWI]];
    o[GAPP_NC] = ct[i[GAPP_CI]];
    o[GAPP_NRAM] = ramt[i[GAPP_RAMI]];

    tmp = a[(o[GAPP_NNS] << 2) | (o[GAPP_NEW] << 1) | o[GAPP_NC]];

    o[GAPP_NSM] = !! (tmp & 4);
    o[GAPP_NCY] = !! (tmp & 2);
    o[GAPP_NBW] = tmp & 1;
}

```

Libéré des contraintes héritées du mélange de la spécification et de l'exécution, la fonction implémentant le PE GAPP est un mapping presque immédiat de sa spécification architecturale<sup>a</sup>.

<sup>a</sup>L'ajout d'une spécification de même type pour la partie ALU, permettra la synthèse automatique de fonctions d'implémentation de PE à partir de spécification purement déclaratives.

Figure 2.29: *La fonction implémentant un PE GAPP*

- Le mélange de deux tables consiste en la combinaison de leurs paramètres formels en une seule liste de paramètres, résultant en une table dont la taille est égale au plus (si l'intersection des ensembles de paramètres est vide) au produit de la taille de chaque table<sup>43</sup>. Cette opération est applicable soit au niveau des fonctions elles mêmes, et nécessite une nouvelle invocation de chaque fonction pour la construction de la nouvelle table<sup>44</sup>, soit directement sur les tables elles mêmes. Le mélange de tables permet une optimisation de l'usage des ressources CPU au détriment des ressources mémoire. Cette optimisation peut sembler douteuse puisqu'elle échange une augmentation linéaire<sup>45</sup> de

<sup>43</sup>Il s'agit de la taille de la table en nombre d'entrées, qui doit être multipliée par le cardinal de l'union des paramètres, exprimé en bits, pour obtenir la taille en bits.

<sup>44</sup>Cela peut s'avérer relativement coûteux pour les grandes tables, si la fonction est fournie par un interprète.

<sup>45</sup>Le facteur de gain dépend de façon difficile à prévoir du coût relatif de la construction séparée ou unifiée de l'index et de la dispersion des index produit par la simulation.

```

inline int Gapp_next_state(Gapp* zis, int input) {
    Bit_Field_explode(&zis->input, input);
    gapp_pe(zis->alu, zis->input.explode, zis->output.explode);
    return Bit_Field_implode(&zis->output);
}

void Gapp_mk_pe_table(Gapp* zis) {
    int i;

    for (i = 0; i < zis->size; ++i) {
        zis->pe_tab[i] = Gapp_next_state(zis, i);
    }
}

```

La fabrication de la table ne pose évidemment aucun problème, il s'agit d'une simple boucle. La figure fait apparaître clairement comment le passage par table libère le style de l'implémentation, en effet les fonctions *implode* et *explode* qui assurent les conversions entre une représentation bit-packed et bit-expanded<sup>a</sup> peuvent être utilisées sans préoccupations inutiles de performances ou d'occupation mémoire<sup>b</sup>.

<sup>a</sup>Dans laquelle un bit est stocké dans un mot de la taille naturelle de l'architecture, ici un entier de 32 bits est utilisé pour stocker 1 bit.

<sup>b</sup>La présence d'une déclaration *inline*, indique toutefois qu'il reste possible d'utiliser des optimisations qui ne compromettent pas la généralité de l'implémentation. En phase de développement, la rapidité de construction de la table est importante.

Figure 2.30: *La fabrication de la table de transition de l'AC PE GAPP*

la vitesse de calcul contre une augmentation exponentielle de la taille de la mémoire utilisée. De plus la construction de grandes tables redondantes est en elle-même un processus coûteux, voire très coûteux si l'on en vient à faire appel à la mémoire paginée. Cependant une mémoire qui se compte en dizaine de mégabytes sur une station de travail, rend intéressant ce genre d'optimisation. D'autant plus qu'une fois achevée la construction de la table, son usage peut révéler une densité d'accès concentrée sur des fractions de la table<sup>46</sup> allant jusqu'à justifier l'usage de tables d'une taille supérieure à la mémoire réelle disponible. Encore que dans ce cas, le cache devrait plutôt être fourni par l'ap-

<sup>46</sup>Cela dépend des espaces initiaux et des AC, bien sûr. Tous ceux qui ont observé life partant d'un espace initial aléatoire de densité  $\frac{1}{2}$ , comprennent qu'après les premiers points de temps la densité d'impact des index se concentre en fonction de la densité générale de l'espace, et de la densité des patterns produits.

```
typedef struct {
    Bit_Field_Spec* input;
    Bit_Field_Spec* output;
    Bit_Field_Spec* instruction;
    Bit_Field_Spec* iparts;
    int* alu;
} Gapp_Pe_Spec;

extern Gapp_Pe_Spec gapp_pe_spec;
```

Un PE est composé d'un champ de bits pour les entrées, les sorties, l'instruction, et d'une table de vérité pour l'alu<sup>a</sup>. Une description purement déclarative d'une architecture de PE, nécessiterait la mise en champ de bits de celle-ci.

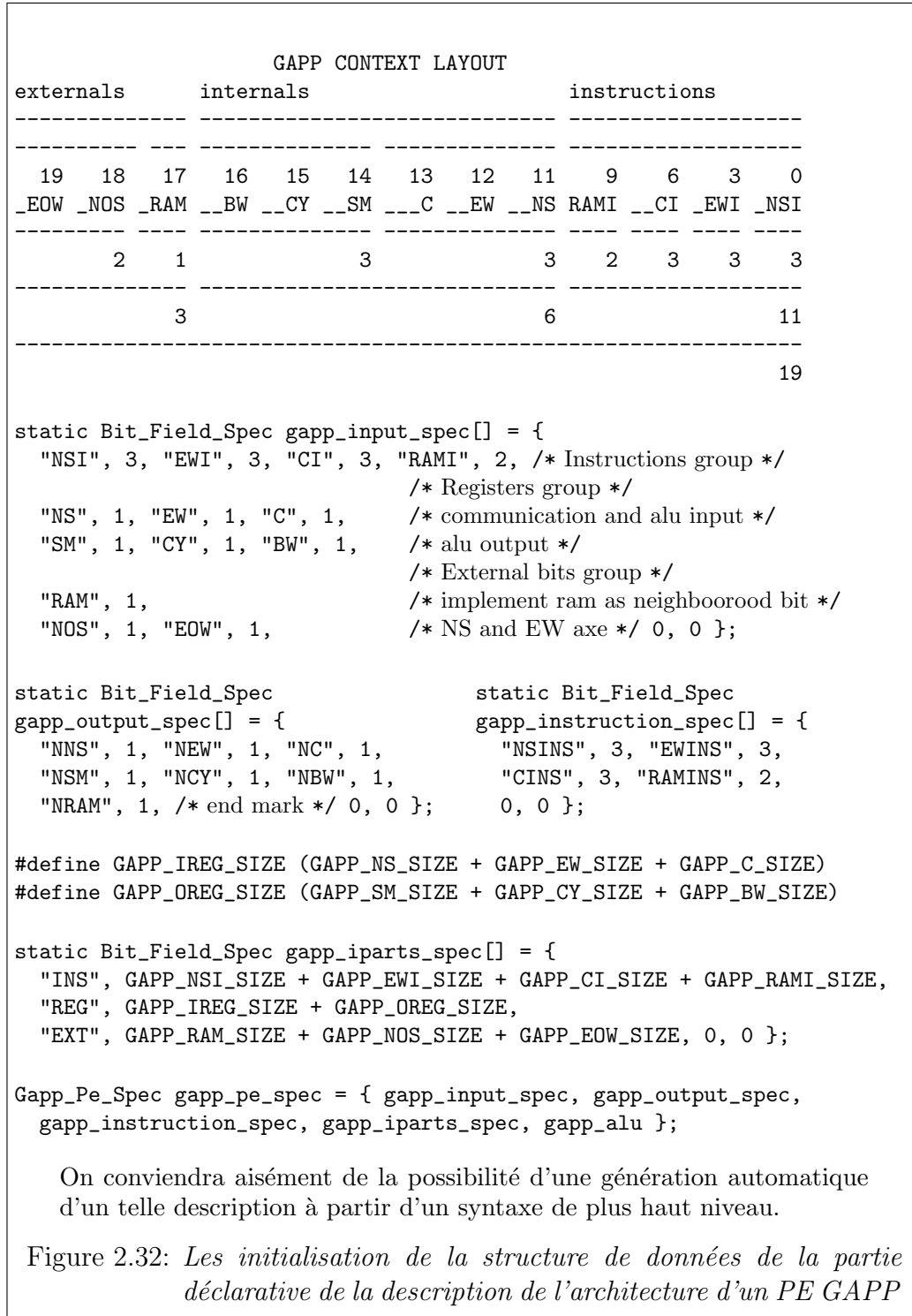
<sup>a</sup>Cette table est ici donnée in extenso sans description associée (cf. figure 2.33, page 77).

Figure 2.31: *La structure de données de la partie déclarative de la description de l'architecture d'un PE GAPP*

plicatif, sous forme d'une construction virtuelle de la table<sup>47</sup>, que par le système d'exploitation. La dualité opposant l'usage de nombreuses petites tables appliquées en parallèle sur un même espace, à l'usage d'une seule grande ou très grande table, permet de choisir entre une mise en œuvre adaptée au développement incrémental interactif (faible délai de mise en marche d'une architecture, associé à un débit de mise à jour cellulaire non optimum), et une mise en œuvre adaptée à l'exploration de la dynamique à long terme d'AC sur de grands espaces (temps de démarrage important, associé à un débit de mise à jour cellulaire optimum pour une architecture matérielle donnée).

- La séparation des tables est l'opération inverse du mélange. Cette opération (pour laquelle nous n'avons pas actuellement cherché d'algorithme) devrait permettre de déterminer la qualité de primitive d'une fonction.

<sup>47</sup>Pour ne pas être trop coûteuse, celle-ci doit reposer sur les services du système d'exploitation qui permettent une récupération (trap) des défauts de pages.



|   |         |          |  |                           |
|---|---------|----------|--|---------------------------|
|   | NS EW C | SM CY BW |  | static int gapp_alu[] = { |
| 0 | 0 0 0   | 0  0 0 0 |  | 0, /* 0 */                |
| 2 | 0 1 0   | 5  1 0 1 |  | 5, /* 1 */                |
| 4 | 1 0 0   | 4  1 0 0 |  | 5, /* 2 */                |
| 6 | 1 1 0   | 2  0 1 0 |  | 3, /* 3 */                |
| 1 | 0 0 1   | 5  1 0 1 |  | 4, /* 4 */                |
| 3 | 0 1 1   | 3  0 1 1 |  | 2, /* 5 */                |
| 5 | 1 0 1   | 2  0 1 0 |  | 2, /* 6 */                |
| 7 | 1 1 1   | 7  1 1 1 |  | 7 /* 7 */ };              |

Une architecture mixte SIMD / cellulaire considère l'ALU comme une table de transition d'un AC, et permet le séquençement des chemins de données à travers ces tables.

Figure 2.33: *Initialisation de l'ALU d'un PE GAPP*

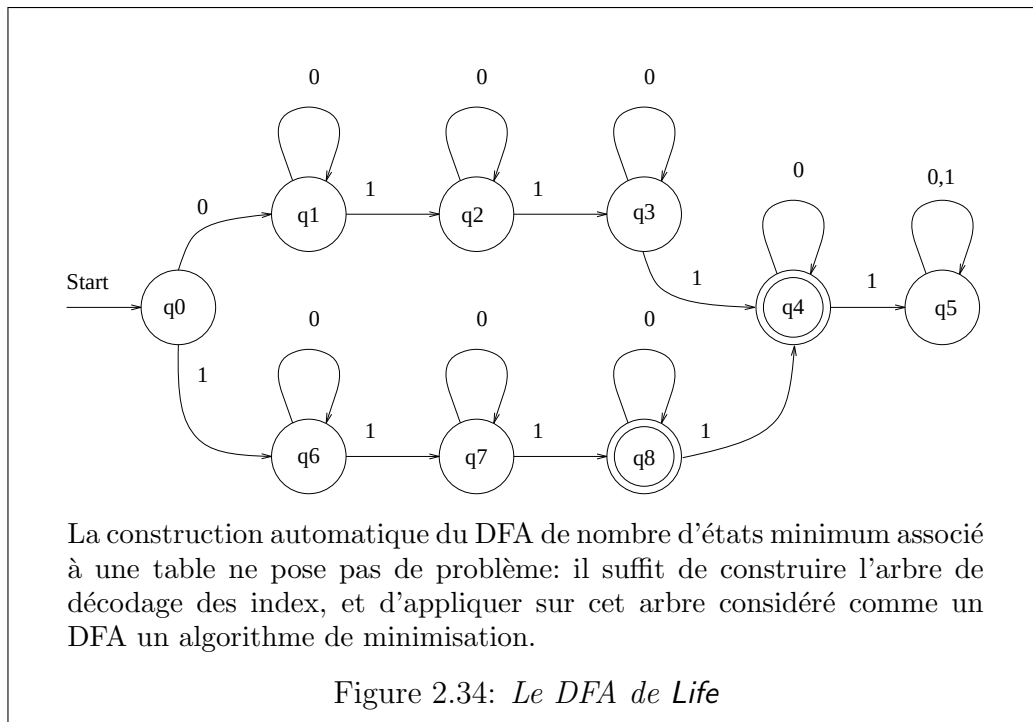
### 2.3.6 Des machines virtuelles: compilation d'architectures SIMD pour une architecture cellulaire presque pure

Nous avons déjà évoqué l'équivalence entre une fonction de transition et une machine SIMD. La cellule d'espace est la mémoire locale à un PE, la table de transition le PE, le réseau d'interconnexion les liens entre les PE, et la fonction d'application l'émulateur (la machine) qui déplace les bits de la mémoire aux PE. Le choix d'une fonction d'application détermine la taille de la table de transition, et donc le nombre de machines virtuelles possibles associées à cette fonction. Les contraintes qui pèsent sur la conception d'une architecture matérielle réelle ne sont pas les mêmes que celles qui pèsent sur une architecture matérielle virtuelle. Les machines virtuelles peuvent être très spécifiques, et pas nécessairement universelles. Les machines SIMD réelles doivent être universelles. Néanmoins, le but de l'objet loi de transition, est bien de permettre la spécification d'une architecture SIMD quelconque, et sa mise en œuvre la plus efficace possible.

Que doit être une fonction de transition pour simuler un plan de  $N$  GAPP?

La cellule est composée des 3 registres (C, NS et EW), des 3 bits de sortie de l'alu (SM, CY et BW), des 2 bits de voisinage (N ou S, et E ou W), et de 1 bit de ram (M). La fonction d'application reçoit en plus des arguments usuels, les 11 bits de contrôle, les 7 bits d'adresse et un espace supplémentaire de  $N \times 128$  bits. Voir la figure 2.27.

Le travail de décodage des bits de contrôle est minimal et peut donc être



sorti de la boucle d'application.

Le bit de ram de la cellule est chargé depuis la mémoire.

Condition  $Ctl_8 Ctl_9 \equiv 0$ .

```
*cellule &= CLEAR_RAM;
*cellule |= !(ram[adresse_hi] & adresse_pos) << RAM;
```

Le registre NS est chargé depuis la cellule au nord.

Condition  $Ctl_4 Ctl_3 Ctl_2 \equiv 2$ .

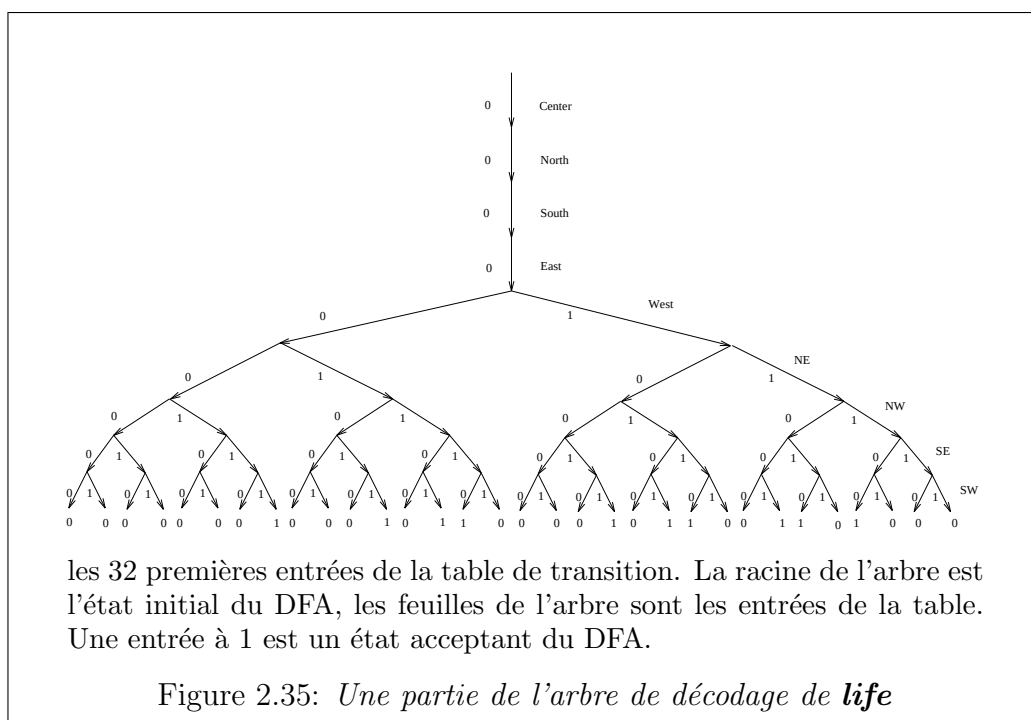
```
*cellule &= CLEAR_NS;
*cellule |= *north & NS;
```

L'action existe aussi pour les cas  $NS \leftarrow S$ ,  $EW \leftarrow E$  et  $EW \leftarrow W$ .

L'index est composé avec les 11 bits de contrôle et les 7 bits d'états de la cellule<sup>48</sup>. Le nouvel état de la cellule est calculé.

```
*cellule = transition[*cellule | controle];
```

<sup>48</sup>La table de transition a donc une taille de 1024 kilobytes, chaque byte contenant 7 bits utiles.



Le bit de ram est écrit dans la mémoire.  
Condition  $\text{Ctl}_8\text{Ctl}_9 \neq 0$ .

```
ram[adresse_hi] &= clear_ram;
ram[adresse_hi] |= !(*cellule & GET_RAM) << adresse_pos;
```

Les 3 conditions possibles pour l'accès à la ram (pas d'accès ou read ou write) et les 5 conditions possibles pour l'accès aux voisins (pas d'accès ou  $\text{NS} \leftarrow (\text{N} \vee \text{S})$  ou  $\text{EW} \leftarrow (\text{E} \vee \text{W})$ ) donnent 15 fonctions de boucle d'application, qui peuvent précalculer certaines valeurs associées aux conditions (offset en bytes et en bits de l'adresse par exemple), et dans le cas d'une instruction sans accès mémoire ni voisins, peut se ramener à `*cellule = transition[*cellule]` (après un précalcul `transition += contrôle`).

On peut aussi ajouter le traitement du *global output*, dont la condition de réalisation est sous contrôle du séquenceur.

```
global |= *cellule & NS;
```

Nous avons tout de même besoin d'écrire la fonction qui remplit la table de transition, mais le passage par table simplifie considérablement la tâche, comme le montre clairement les figures 2.28, 2.29 et 2.30<sup>49</sup>.

<sup>49</sup>L'implémentation de la construction de la table.

### 2.3.7 Et leur compilateur: la décompilation des lois pour des architectures SIMD

Nous avons montré comment un AC réalisé par table peut simuler une architecture SIMD. Comment transformer une fonction de transition en une séquence d'instructions pour une machine SIMD? Soit une table de transition de  $N$  bits de voisinage et  $R$  bits de résultats. Construisons  $R$  tables de taille  $2^N$  bits. Nous cherchons un générateur d'instructions pour une machine SIMD pour chacune des  $R$  fonctions booléennes associées à la fonction de transition. Soit  $T$  une table de  $N$  bits de voisinage, avec  $R = 1$ . La table  $T$  définit un langage sur les  $2^N$  index de la table. Le nombre de mots du langage étant fini, il est toujours possible de construire un DFA pour reconnaître ce langage. Les expressions régulières constituent donc un mode d'expression possible d'une table de transition. Par exemple pour Life, si le premier bit de l'index est le bit cible:

$(00^*10^*10^*10^*)|(10^*10^*10^*10^*)|(10^*10^*10^*)$

| index  | DFA transition                         | state set |
|--------|----------------------------------------|-----------|
| Center | 016                                    | 16        |
| North  | 112, 667                               | 1267      |
| South  | 112, 223, 667, 778                     | 123678    |
| East   | 112, 223, 334, 667, 778, 884           | 1234678   |
| West   | 112, 223, 334, 445, 667, 778, 884      | 12345678  |
| NE     | 112, 223, 334, 445, 555, 667, 778, 884 | 12345678  |
| NW     | 112, 223, 334, 445, 555, 667, 778, 884 | 12345678  |
| SE     | 112, 223, 334, 445, 555, 667, 778, 884 | 12345678  |
| SW     | 112, 223, 334, 445, 555, 667, 778, 884 | 12345678  |

Une table représentant l'encodage du DFA de Life, utilisant les noms des états de la figure 2.34

Tableau 2.9: *Calcul d'une génération de Life*

Chaque PE de la machine SIMD est dédié à une cellule. La mémoire locale de chaque PE maintient l'état de la cellule, plus l'état courant du DFA. Pour chacun des bits de voisinage de l'automate, chaque PE reçoit le sous-ensemble du DFA construit à partir de l'ensemble des états du DFA précédemment accédés. Le DFA est transmis sous la forme de triplets (état courant, état si 1, état si 0). Pour finir, chaque PE reçoit la liste des états acceptants.

| index  | DFA transition                       | state set |
|--------|--------------------------------------|-----------|
| Center | 016                                  | 16        |
| North  | 1-2, 6-7                             | 1267      |
| South  | 1-2, 2-3, 6-7, 7-8                   | 123678    |
| East   | 1-2, 2-3, 3-4, 6-7, 7-8, 8-4         | 1234678   |
| West   | 1-2, 2-3, 3-4, 4-5, 6-7, 7-8, 8-4    | 12345678  |
| NE     | 1-2, 2-3, 3-4, 4-5, —, 6-7, 7-8, 8-4 | 12345678  |
| NW     | 1-2, 2-3, 3-4, 4-5, —, 6-7, 7-8, 8-4 | 12345678  |
| SE     | 1-2, 2-3, 3-4, 4-5, —, 6-7, 7-8, 8-4 | 12345678  |
| SW     | 1-2, 2-3, 3-4, 4-5, —, 6-7, 7-8, 8-4 | 12345678  |

Il est possible d'obtenir un encodage plus économique<sup>a</sup> du DFA de Life, plus proche d'un encodage utilisant la nature arithmétique de la règle. L'optimisation présentée repose quant à elle sur un mécanisme général.

<sup>a</sup>En nombre d'instructions.

Tableau 2.10: *Une optimisation de la table 2.9*

## 2.4 Les séquenceurs

Nous pensions initialement que la phase de spécification de l'architecture d'une expérience cellulaire pouvait être séparée de la phase de séquençement d'exécution. Nous n'avons pas de solution générale satisfaisante au problème de la séparation de ces opérations, de la spécification<sup>50</sup> et de l'implémentation des séquenceurs. Les solutions particulières que nous avons envisagées, testées ou implémentées reposent toutes sur une approche *classique* centrée sur des méthodologies ou des outils d'implémentation d'interfaces. Cependant, nos expériences de traduction pratiques d'architectures SIMD vers cellulaire, et cellulaire vers SIMD, nous amènent à proposer une nouvelle approche des fonctions du séquenceur, basée sur la définition d'une architecture mixte: une architecture SIMD incluant des primitives de traduction et de manipulation d'architectures cellulaires pures. Le problème du séquenceur cellulaire devient alors celui, plus général, de la spécification et du pilotage d'une machine SIMD virtuelle.

Nous avons cherché les solutions possibles, expérimenté plusieurs modes de séquençement et implémenté une interface *langage de commande* basée sur Tcl et une extension graphique interactive basée sur Tk.

La solution la plus simple consiste à bâtir une commande spécialisée par classe de fonction de la librairie, puis à utiliser le shell Unix comme séquenceur et le système de fichiers comme système de nommage des objets intermédiaires. Cette solution, la solution Unix classique, reste irremplaçable. Nous l'avons largement utilisée. Le scénario de base est le suivant:

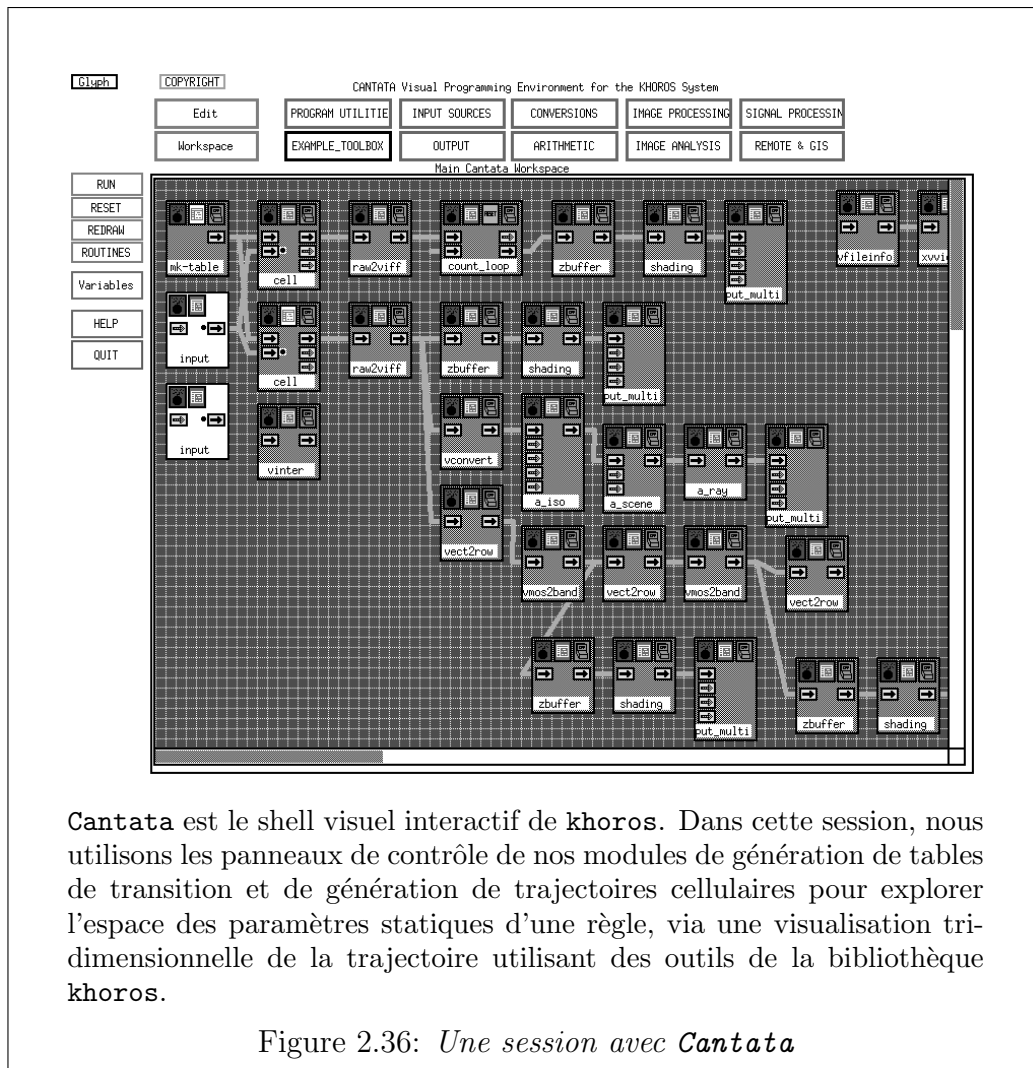
```
mk-rule-foo > foo.rule; mk-space-bar > bar.space  
run-ca foo.rule bar.space | xdumper  
run-ca -n 8192 foo.rule bar.space | do-stats > data  
plot data
```

Une autre solution classique consiste à définir un langage (Almasi et al., 1987; Eckart, 1992) doté de structures de données et de contrôles adaptées aux objets manipulés. Les outils de réalisation existent<sup>51</sup>, mais les choix des objets et des opérations primitives étant précisément un des objets de notre recherche, ne peuvent venir avant l'expérimentation.

---

<sup>50</sup>Une fonction de séquençement exemplaire est la régulation d'un temps relatif pour une trajectoire. Certain AC présentent une évolution croissante tendant vers un point fixe ou un cycle limite court, passant par une phase de croissance très rapide suivie d'une phase de croissance très lente d'une variable macroscopique élémentaire comme la population d'un plan de bits. Le couplage de la variation de cette variable à la commande d'enregistrement du point de temps courant permet de réaliser des films à échantillonnage variable et à variation de densité spatiale constante.

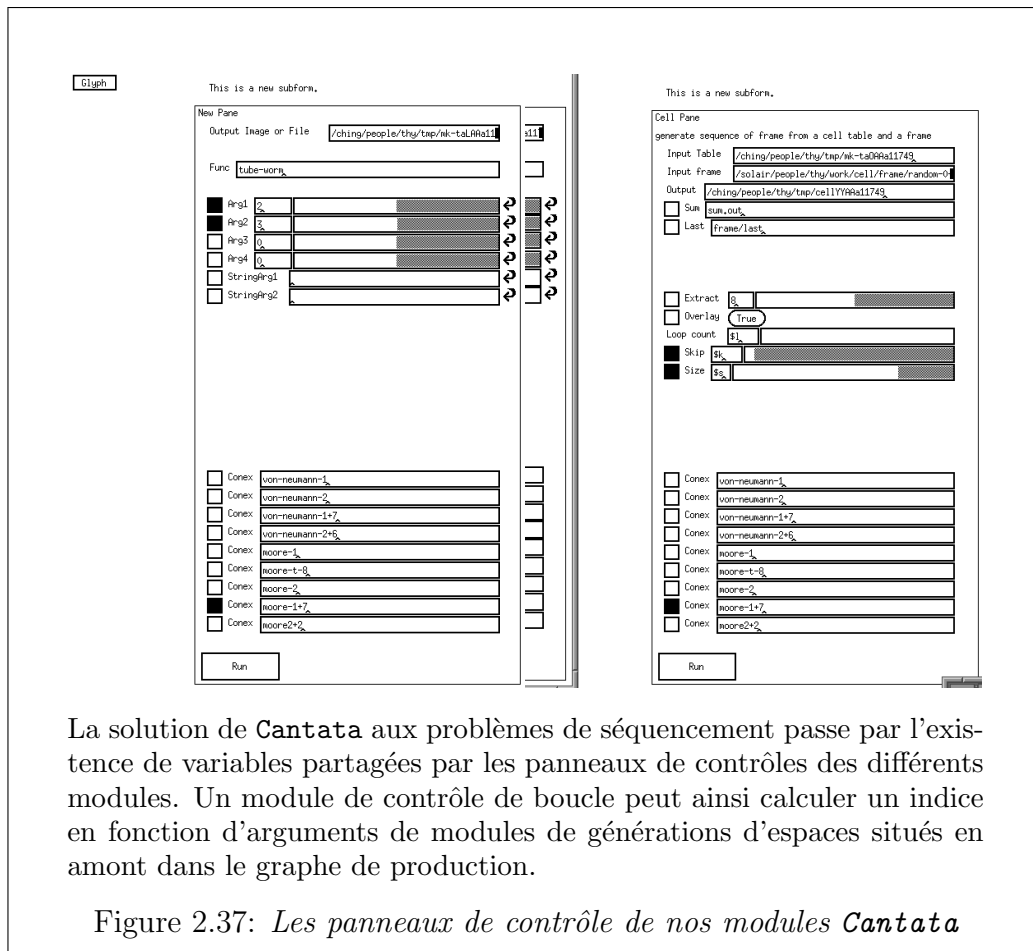
<sup>51</sup>lex, yacc, TXL (Cordy et al., 1991)



Une solution intermédiaire se trouve dans l'usage d'un interprète d'un langage extensible permettant une manipulation interactive des objets de la librairie. C'est la solution que nous avons retenue pour le workbench<sup>52</sup>, sur base Tcl.

Nous avons également imaginé l'usage d'interprètes spécialisés dans les traitements numériques, en particulier matriciels, comme support d'interface à la librairie. En effet, nous avons réalisé des manipulations de données à l'aide d'outils à la *matlab*: *Octave*<sup>[26]</sup>, *Rlab*<sup>[27]</sup>. Ceux-ci constituent un environnement bien adapté à la manipulation statistique des données associées à

<sup>52</sup>C'est aussi la solution retenue pour le contrôle de la machine CAM6(Toffoli et Margolus, 1987), sur base Forth.



La solution de **Cantata** aux problèmes de séquençement passe par l'existence de variables partagées par les panneaux de contrôles des différents modules. Un module de contrôle de boucle peut ainsi calculer un indice en fonction d'arguments de modules de générations d'espaces situés en amont dans le graphe de production.

Figure 2.37: Les panneaux de contrôle de nos modules *Cantata*

une trajectoire, et sont par ailleurs des langages suffisamment généraux pour la réalisation de fonctions de séquençement simples. L'idée est d'ajouter les types de données *espace* et *règle*, et une opération d'application, ainsi que des opérations de conversion matrices / espaces et matrices / règles à ces environnements.

La dernière solution classique, est l'usage des outils de visualisation scientifique, en particulier de leurs shells graphiques interactifs permettant la construction de graphes de traitement. Nous avons utilisé **Khoros**<sup>[12]</sup>, dont l'outil de programmation visuelle est **cantata**<sup>[13]</sup>. A fin de prototypage nous avons développé des panneaux de contrôles **Khoros**, associés à l'interface *commande Unix* de notre librairie<sup>53</sup>. Le scénario de base est alors une session interactive **Cantata**, au cours de laquelle les sorties des modules génération d'espaces de

<sup>53</sup>La solution la plus complète consisterait à développer un toolkit **Khoros** basé sur notre librairie.

```

# func
proc func {func} {simple.func set [$func]}
# time
proc first {} {simple.time first}
proc last {} {simple.time last}
proc next {} {simple.time next}
proc prev {} {simple.time prev}
proc point {} {simple.time point}
proc goto {point} {simple.time point $point}
proc mark {} {simple.time mark}
proc swap {} {simple.time swap}
proc step {} {simple.time step}
proc stamp {} {simple.time stamp}
# space
proc zero {} {simple.time.now zero}
proc one {{plane 0}} \
  {simple.time.now fill $plane 1 1}
proc solid {{size 16} {plane 0}} \
  {simple.time.now fill $plane $size 1}
proc rand {{rand 2} {plane 0}} \
  {simple.time.now fill $plane 0 $rand}
proc brand {{bloc 0} {rand 2} {plane 0}} {
  simple.time.now fill $plane $bloc $rand}
proc extract {{plane 0} {flip 1}} \
  {simple.time.now extract $plane $flip}
proc save \
  {{file plane.pbm} {plane 0} {flip 0}} {
  simple.time.now write $plane $flip $file
  extract $plane}

# simple
proc apply {} {simple apply}
# dump
proc view {} {dump dump}
# gen
proc repeat {{cnt 1} body} \
  {loop i 0 $cnt {uplevel $body}}

# proc
proc store {} {apply; next; swap; step; stamp}
proc sync {} {extract; view}

proc tofirst {body} \
  {while {[simple.time firstp]} \
   {uplevel $body}}
proc tolast {body} \
  {while {[!simple.time lastp]} \
   {uplevel $body}}

proc run {{cnt 1}} \
  {loop i 0 $cnt {apply; swap; sync}}
proc record \
  {{view 1} {run 1}} \
  {tolast {store; repeat $view {run $run}}}}
proc forw {{skip 1}} {
  for {set i [simple.time point]; \
    set cnt [last; simple.time point]} \
    {$i <= $cnt} {incr i $skip} {
      simple.time point $i
      view
    }
  if {$i > $cnt} { last; view }
}
proc back {{skip 1}} {
  for {set i [simple.time point]} \
    {$i >= 0} {incr i -$skip} \
    {simple.time point $i; view}
  if {$i < 0} { first; view }
}

proc cycle {{skip 1}} {forw $skip; back $skip}
proc cont {} {last; swap; first; swap; sync}

proc seq {{view_timer 1} {store_timer 1}} {
  set store 1
  set view $view_timer
  while {[!simple.time lastp]} {
    apply
    if {$store && ![incr store -1]} {
      set store $store_timer
      next
    }
    swap; step; stamp
    if {$view && ![incr view -1]} {
      set view $view_timer
      extract
      view
    }
  }
  extract; view
}
view

```

Ces procédures travaillent sur des objets top-level destinés à l'interface humaine<sup>a</sup>

<sup>a</sup>Soit par un tty, soit par une interface Tk. Voir figure 1.1, page 7.

Figure 2.38: Définitions de procédures de séquencements

configurations et de génération de tables de transitions, adéquatement paramétrées via les panneaux de contrôle associés, sont connectés aux entrées correspondantes d'un module *moteur cellulaire*, lequel est lui-même connecté à un module de visualisation. L'intérêt principal de cette approche, outre l'aspect visuel, réside dans la possibilité, une fois l'adaptation aux normes d'interface réalisée, d'utiliser la vaste bibliothèque de modules disponibles dans ce genre d'environnement, et en particulier sous **Khoros**. Des modules de synthèse et manipulation ou d'analyse d'images de **Khoros** peuvent être utilisés en complément de nos modules.

Aucune des solutions précédentes ne nous a permis de nous affranchir des problèmes de séquençement.

La solution que nous proposons pour améliorer cette situation passe par la mise en pratique d'une association architecturale mixte SIMD / cellulaire.

Nous pensons que l'association d'AC localement universels (PE) à des AC non localement universels fournit un cadre conceptuel adapté au calcul cellulaire appliqué à l'étude des AC dans un contexte matériel séquentiel ou de parallélisme non massif.

En complément de la transformation d'un AC donné par sa table de transition en séquence d'instructions pour une machine SIMD, nous considérons la transformation d'un algorithme cellulaire en un ensemble d'AC donné par leur table, associé à une séquence d'instructions pour une machine SIMD, réalisant le séquençement des sous-AC de l'algorithme.

Le problème du séquençement devient alors un programme d'étude des architectures de calcul cellulaire appliqué à l'expérimentation et l'étude des AC<sup>54</sup>. Nous insistons sur le point suivant: une architecture cellulaire n'a pas besoin d'une machine parallèle. Hypothèse: *On peut implémenter de manière efficace une architecture cellulaire sur une machine séquentielle*. Une telle implémentation a pour vocation de supporter des implémentations cellulaires d'algorithmes cellulaires sur machine non cellulaire sans perte d'efficacité<sup>55</sup>. Les avantages sont multiples: cohérence et clarté de la description des algorithmes dans leur environnement architectural naturel. Simplicité<sup>56</sup>, facilité de debug, portabilité et efficacité de la programmation par look-up table, enfin parallélisation facile, ou plutôt, mapping sur une architecture parallèle arbitraire d'une description cellulaire d'un algorithme.

Les AC peuvent alors être vus comme les primitives d'une architecture

---

<sup>54</sup>Les architectures cellulaires sont évidemment applicables à d'autres domaines que les AC: traitement d'image, visualisation, certain calculs numériques, simulation d'architecture matérielle, en général tous les domaines pour lequel le SIMD est applicable.

<sup>55</sup>Les algorithmes du traitement d'image sont candidats à la validation de cette hypothèse.

<sup>56</sup>Au sens où la nature cellulaire de l'algorithme n'a pas à être traduite en boucles.

cellulaire, et ceux-ci deviennent ainsi les moyens d'étude de leur propre univers, car ils posent de problèmes de visualisation, de réduction numérique, de spécification d'architecture matérielle et de dynamique cellulaire, toutes choses pour lesquelles le calcul cellulaire est applicable.

## Chapitre 3

# Structure de l'espace des automates cellulaires

Intuitivement, la classification des AC est une classification qualitative de leurs comportements dynamiques. Cette classification repose sur des mesures statistiques des comportements<sup>1</sup>.

La première question est celle de l'existence d'un certain comportement, une réponse générale est donnée par l'existence d'AC universels. Un cadre général est donné par l'utilisation des méthodes de la mécanique statistique pour qualifier les comportements.

La deuxième question est celle du nombre d'AC d'un type donné et de leurs localisations dans l'espace des règles. Cette question introduit le problème de l'existence de paramètres permettant de parcourir l'espace des règles.

Le problème de la classification des AC se situe à l'intersection du problème *forward*<sup>2</sup> et inverse<sup>3</sup> (Gutowitz, 1990). Il implique l'existence d'un ensemble de mesures réalisables sur les règles et sur leurs dynamiques.

L'existence de paramètres de contrôle de la dynamique des AC permet de déterminer une structure pour l'espace des AC.

---

<sup>1</sup>Par exemple l'entropie  $S = -\sum_i p_i \log(p_i)$  ou l'information mutuelle (la corrélation de deux distributions)  $M = \sum_i \sum_j p_{ij} \log \frac{p_{ij}}{p_i p_j}$  (Li et al., 1990).

<sup>2</sup>Déterminer les propriétés à partir de la règle.

<sup>3</sup>Déterminer la règle à partir des propriétés.

### 3.1 Une version minimale du problème

Les AC élémentaires, tel que définis par Wolfram (1986) sont les AC de dimension  $d = 1$ , de rayon<sup>4</sup>  $r = 1$  et de nombre d'états  $k = 2$ . Une simplification ultime considère l'espace des AC de paramètres  $d = 0$ ,  $r = 0$ ,  $k = 2$ , Ce qui donne  $k^{dr+1} = 4$  règles (Zéro, Flip, Same, One), lesquelles donnent 8 dynamiques:

$Z(0) = 0^*$ ,  $Z(1) = 10^*$ ,  $O(0) = 01^*$ ,  $O(1) = 1^*$

$S(0) = 0^*$ ,  $S(1) = 1^*$ ,  $F(0) = (01)^*$ ,  $F(1) = (10)^*$

soit 4 points fixes sans transitoire (en 2 variétés), 2 points fixes avec transitoire de taille 1, 2 cycles de taille 2 sans transitoire.

Les AC élémentaires, dont le nombre est 256 peuvent être énumérés<sup>5</sup>, mais l'étude des dynamiques associées à une règle dépend du nombre  $N$  de cellules de l'espace de configuration (qui tend vers l'infini) et fait nécessairement appel à des méthodes statistiques. Pour  $d = 1$ ,  $k = 2$ ,  $r = 2$ , le nombre de règle est  $2^{32}$  et ne permet déjà plus une énumération.

### 3.2 La classification de Wolfram

La classification de Wolfram est une classification de référence pour de nombreux auteurs (Vichniac, 1990; Li et al., 1990; Wootters et Langton, 1990; McIntosh, 1990; Chate et Manneville, 1990; Gutowitz, 1990).

Dans les quatre classes de Wolfram, les classes I, II et III sont vues comme des attracteurs de systèmes dynamiques:

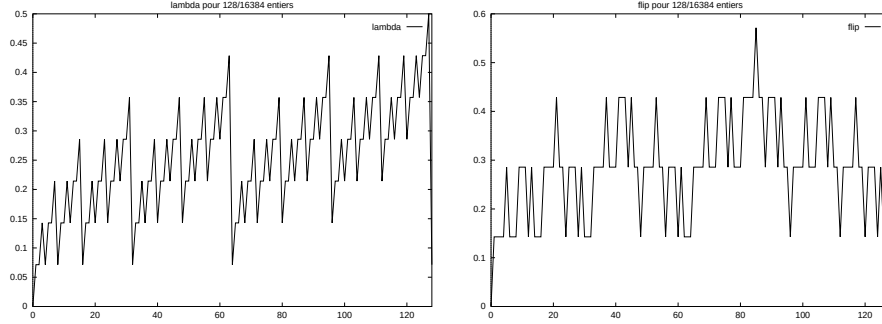
#### classe I

- point limite
- ou tendance à un état spatialement homogène
- ou les formes disparaissent avec le temps
- ou de petits changements sur la configuration initiale sont sans effet sur l'état final

---

<sup>4</sup>Le nombre de cellules voisines est  $dr + 1$ .

<sup>5</sup>Des données statistiques peuvent être obtenues pour chaque règle appliquée sur un espace de configuration.



Le paramètre appelé  $\lambda$  par Li et al. (1990) est défini par  $\lambda = \frac{k^{dr+1} - \#0}{k^{dr+1}}$  (cf. § 3.3, page 91). Pour les règles binaires,  $\lambda$  est une version normalisée de la fonction  $\#_1(n) = n - \sum_{i=1} \lfloor n/2^i \rfloor$  (Wolfram, 1983) donnant le nombre de bits à 1 dans la représentation binaire d'un nombre. Nous définissons Flip pour les règles binaires comme le nombre de *bit-flip* rencontrés en balayant les entrées de la table de transition (cf. § 3.5, page 96). Les calculs sont fait sur les 16384 premiers entiers considérés comme autant de règles d'un hypothétique voisinage à  $\log_2(14)$  voisins.  $\#0$  désigne le nombre de zéros de la table de transition.

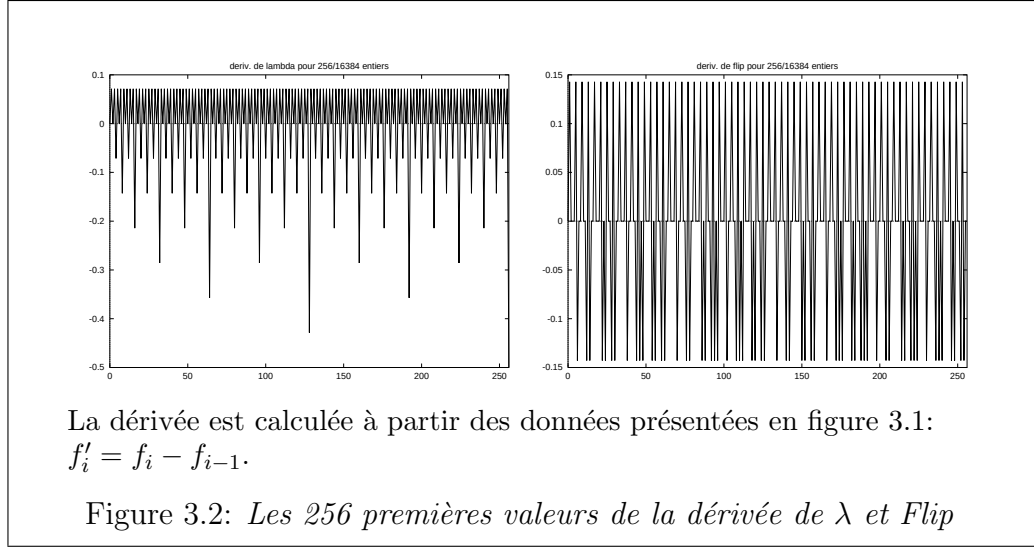
Figure 3.1: Les 128 première valeur de  $\lambda$  et Flip, pour un calcul sur les 16384 premiers entier

## classe II

- cycle limite
- ou montrant une séquence de structure simple, stable ou périodique
- ou les formes évoluent vers une taille fixe finie
- ou de petits changements sur la configuration initiale induisent des changements sur des régions de tailles finies

## classe III

- attracteur chaotique
- ou montrant un comportement chaotique apériodique
- ou les formes croissent indéfiniment à une vitesse fixe
- ou de petits changements sur la configuration initiale induisent des changements sur des régions de tailles toujours croissantes



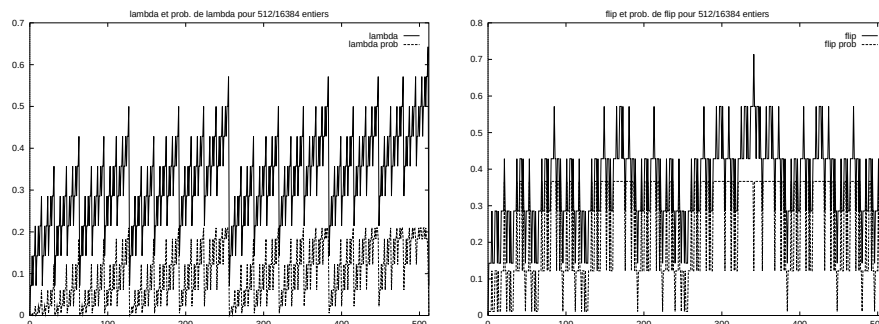
## classe IV

- comportement plus complexe que les précédents, conjecture de calcul universel
- ou montrant des structures localisées complexes, éventuellement propagantes
- ou les formes croissent et se contractent avec le temps
- ou de petits changements sur la configuration initiale induisent des changements irréguliers

La description des classes varie selon les auteurs. On peut introduire la notion de décalage spatial homogène (Li et al., 1990), souligner la difficulté que présente certains comportements intermédiaires (McIntosh, 1990), ou proposer des interprétations structurelles faisant, par exemple, des AC de la classe IV un *mélange* d'AC de la classe II et III (Kayama et al., 1993). Les 4 classes (fixe, périodique, chaotique, complexe) apparaissent actuellement comme un point de départ inévitable de toute tentative de classification. La question de la structure de l'espace, et donc des paramètres de contrôle devient alors la question première.

## 3.3 Le paramètre $\lambda$ de Langton

Le paramètre  $\lambda$  est défini comme le pourcentage d'entrées de la table de transition dont la valeur est différente de l'état zéro:  $\lambda = \frac{k^{dr+1} - \#0}{k^{dr+1}}$ .



La distribution des valeurs d'une fonction utilisée comme paramètre d'échantillonnage d'un espace de règles influence la nature de l'échantillonnage. Les expériences utilisant  $\lambda$  comme paramètre de contrôle (Li et al. (1990) par exemple), ne tiennent pas compte de la non linéarité de la distribution des probabilités d'apparition des valeurs de  $\lambda$  sur un espace de règles.

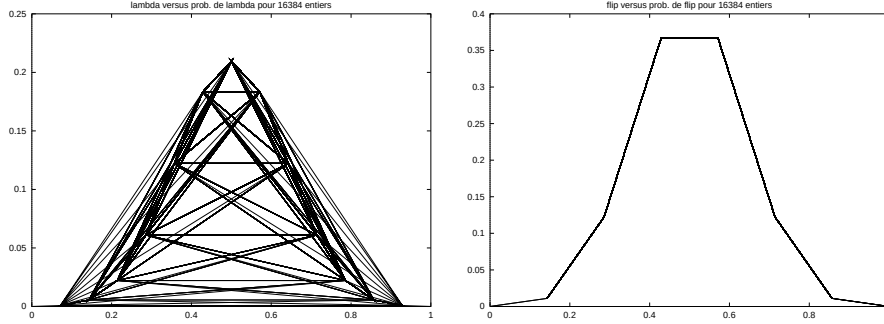
Figure 3.3: Les 512 premières valeurs des fonctions  $\lambda$  et Flip, et de leur probabilité d'apparition

A une petite variation de  $\lambda$  (Langton, 1992) entre deux règles correspond une petite distance de Hamming entre deux règles. La variation du paramètre  $\lambda$  permet d'échantillonner un espace de règles pour  $d$ ,  $k$  et  $r$  fixés. Chaque règle est appliquée sur un ensemble de configurations initiales, et des mesures statistiques sont faites à la volée. Les séries temporelles de mesures sont utilisées pour déterminer l'appartenance de la règle à une classe ou une autre. On peut alors essayer d'établir une corrélation entre  $\lambda$  et la classe d'appartenance de l'AC.

Cette procédure est a priori applicable pour d'autres paramètres, et vise à travers l'échantillonnage à révéler la structure générale que la variation du paramètre de contrôle imprime à l'espace des règles.

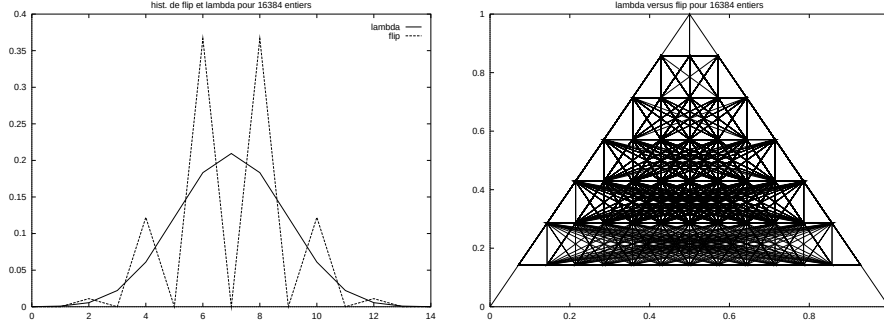
Une question associée à la structure de l'espace défini par un paramètre est la présence et le type d'une éventuelle transition de phase, qui associe à une valeur critique du paramètre de contrôle une transition plus ou moins brusque de la classe de comportement des règles.

Une autre question est la présence d'une tendance au continu associée à l'augmentation de la taille de l'espace des règles.



Nous n'avons pas actuellement d'explication complète de ces résultats, il est néanmoins clair que  $\lambda$  et Flip possèdent des propriétés suffisamment différentes pour justifier la poursuite d'expériences utilisant Flip

Figure 3.4: *Plot de  $\lambda$  et Flip vs. leur probabilité d'apparition*

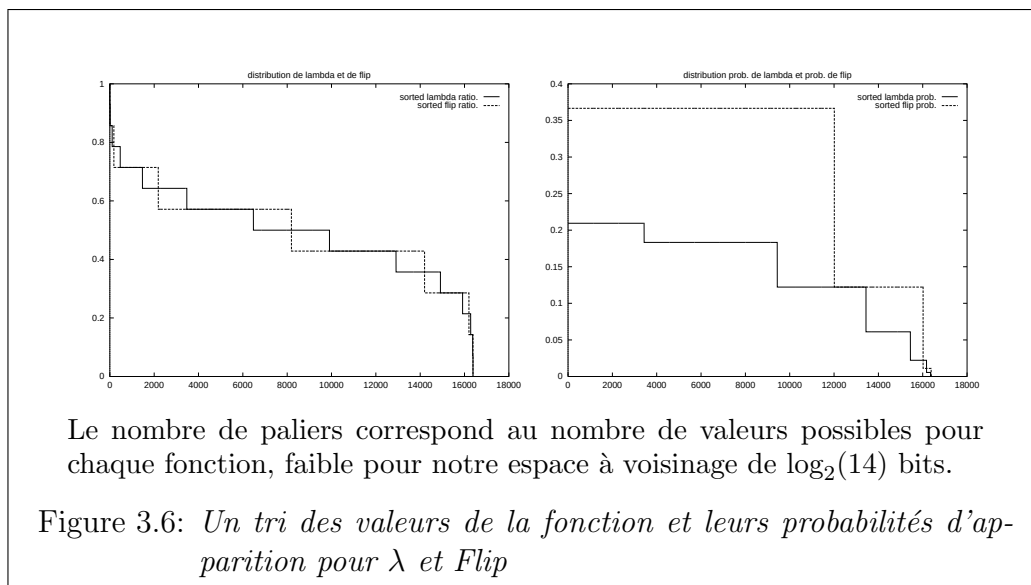


Les valeurs de l'histogramme apparaissent ici non normalisées, i.e. en nombre de bits à 1 pour  $\lambda$  et en nombre de bit-flips pour Flip.

Figure 3.5: *L'histogramme des distribution de  $\lambda$  et Flip, et  $\lambda$  vs. Flip.*

### 3.4 Expérimentations sur le paramétrage des classes de comportements

Li, Packard et Langton (1990) proposent l'existence de transitions de phase contrôlées par le paramètre  $\lambda$ . Il s'agit d'une transition du premier ordre (transition brutale) entre les règles de type II et III, ou d'une transition de deuxième ordre (transition graduelle) associant les règles de type IV à une zone d'intersection des types II et III. Les mesures statistiques utilisées sont l'entropie, l'information mutuelle et la vitesse de propagation des



différences sur l'espace cellulaire<sup>6</sup>. Les espaces de règles sont typiquement bornés par  $d \leq 2$ ,  $k < 10$ ,  $r < 12$ . Ce type d'expériences a d'abord été mené sur les AC élémentaires (Li et Packard, 1990). Wootters et Langton (1990) étendent la recherche à des espaces de règles plus vastes, dans le but de réduire la dispersion de la valeur critique. L'augmentation de  $r$  supprime la transition (la valeur critique tend vers zéro), l'augmentation de  $k$  tend à réduire la dispersion de la valeur critique.

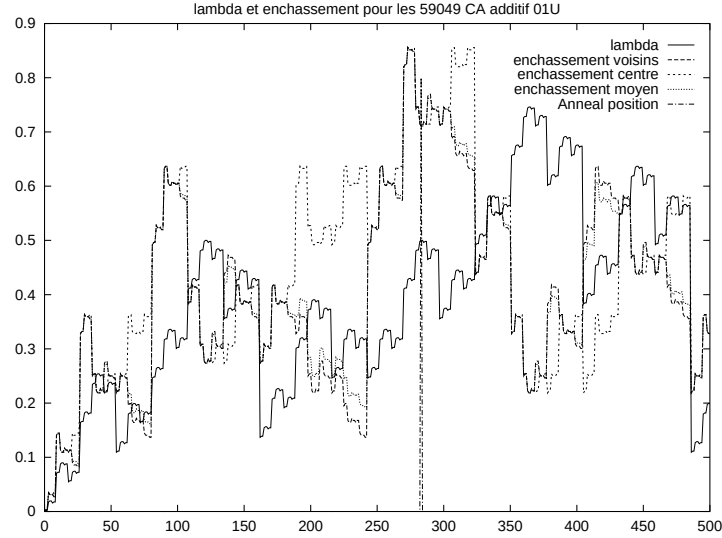
Toutefois, la critique d'une expérience de Packard (1988), formulée par Mitchell et al. (1993) fait apparaître les limites de l'hypothèse de Langton sur l'émergence du *calcul*<sup>7</sup> à la frontière du chaos (Langton, 1990, 1992). Packard utilise un algorithme génétique pour faire évoluer des règles effectuant un calcul particulier, et interprète les résultats comme une indication de l'existence d'une pression sélective sur l'évolution des règles vers les valeurs critiques de  $\lambda$  associées à une transition de phase et à la réalisation d'un calcul complexe.

Mitchell trouve des résultats différents, et souligne la nécessité d'une meilleure compréhension (définition) des concepts utilisés. Mitchell montre, par exemple, que l'évolution par algorithme génétique d'une règle calculant l'état majoritaire de la configuration initiale en tendant vers un point fixe homogène de l'état majoritaire, se trouve nécessairement loin du point critique (0.27

<sup>6</sup>Considérée comme une mesure de l'imprédictibilité de l'évolution des *formes* de l'espace cellulaire, elle devient une mesure de la qualité chaotique du comportement de l'AC.

<sup>7</sup>Il s'agit en fait de la transition vers des règles dont les mesures statistiques indiquent une appartenance à la classe IV, lesquelles sont supposées universelles.

### 3.4. PARAMÉTRAGE DES CLASSES DE COMPORTEMENTS



Nous avons calculé les valeurs de  $\lambda$  et  $EN^a$  pour un sous-espace échantillon de l'espace des règles binaires à voisinage de Moore. Le sous espace est constitué des règles additives, centre inclu, dont la table secondaire supporte les fonctions constantes *zéro* et *un* et la fonction *unchanged*. L'espace contient donc  $3^{10} = 59049$  règles. Les valeurs plotées pour  $EN$  sont:  $EN_{centre}$ ,  $EN_{rayon}$  et la valeur moyenne de  $EN$  sur tous les voisins. Le pic montre la position de *Anneal* dans le sous-espace de règles. Nous avons tenté de corréler les paramètres calculés sur l'espace des règles aux propriétés des trajectoires associées à ces règles pour des configurations initiales aléatoires de densité  $\frac{1}{2}$  sur des espaces de tailles entre  $64 \times 64$  et  $256 \times 256$ , en vue de déterminer les restrictions supplémentaires à appliquer à une recherche automatique.

<sup>a</sup>L'enchâssement, voir équation 3.1.

Figure 3.7: Valeurs de  $\lambda$  et  $EN$

d'après Wootter et Langton) avec  $\lambda = 0.5$ .

Li a par ailleurs suggéré (Li, 1989), en apparente contradiction avec l'idée d'une association entre calcul, complexité et phénomènes critiques (Li et al., 1990) une structure fractale (Mandelbrot, 1984) pour la distribution des règles complexes dans l'espace des règles.

### 3.4. PARAMÉTRAGE DES CLASSES DE COMPORTEMENTS

### 3.5 Proposition pour une recherche d'un paramétrage de l'enchâssement

Face aux problèmes soulevés par l'utilisation du paramètre  $\lambda$ , nous proposons quelques remarques pouvant servir de point de départ à de nouvelles expériences sur la structure de l'espace des AC.

La première remarque concerne l'existence d'autres paramètres de contrôles que  $\lambda$ . Li, Packard et Langton (Li et al., 1990) soulignent l'importance de la recherche d'autres paramètres de contrôle et suggèrent que la dispersion de la valeur critique pourrait être réduite par un paramètre supplémentaire (le *paramètre mystérieux* permettant de localiser des frontières plus précises entre les différents comportements). Il nous semble néanmoins peu probable qu'un seul paramètre supplémentaire suffise.

La deuxième remarque concerne la structure enchâssée des espaces de règles. Nous suggérons que l'étude du degré d'enchâssement d'une règle dans un espace permet la définition d'un nouveau paramètre de contrôle.

La troisième remarque concerne les techniques de réductions du nombre de règles légales à considérer pour un espace donné, c'est à dire l'ablation des règles anisotropiques ou enchâssées, et la possibilité d'utiliser un mapping intermédiaire.

#### Le paramètre Flip

Nous proposons l'utilisation du paramètre *Flip* défini comme le nombre de changements d'états rencontré en balayant successivement chaque entrée de la table de transition.

$$\text{flip} = \frac{1}{N} \sum_{i=0}^{N-1} \text{diff}(f_i - f_{(i+1) \bmod N})$$

avec  $N = k^{dr+1}$ , et

$$\text{diff}(x) = \begin{cases} 1 & \text{if } x \neq 0 \\ 0 & \text{if } x = 0 \end{cases}$$

Si l'on plonge<sup>8</sup> une règle  $R_N$  paramétrée par  $r = N$  dans une règle  $R_{N+1}$ , l'insensibilité de la règle aux deux voisins supplémentaires se traduit par une duplication des entrées de la règle  $R_N$ . Par exemple pour  $k = 2$ , si pour la colonne  $i$  l'égalité de chacune des  $N/2$  paires  $x_j = x_{j+2^i}$  est vérifiée, la

---

<sup>8</sup>Nous définissons le plongement de règles comme augmentation de la taille du voisinage sans influence sur les trajectoires.

colonne  $i$  est inutilisée, et soit la règle est anisotropique, soit la règle est isomorphe à une règle d'espace de taille inférieure et peut être isotropique.

La somme des degrés d'égalité de chaque colonne est un candidat possible au rôle de paramètre de contrôle.

Parce que l'on peut construire une règle pour un espace de taille donnée en plongeant une règle d'un espace de taille inférieure (par duplication des entrées), à  $\lambda$  et Flip constant, puis déplacer peu à peu la règle vers l'espace de taille supérieure en flippant des entrées (éventuellement en conservant  $\lambda$  constant), la mesure de Flip pourrait être associée au degré d'enchâssement d'une règle.

L'utilisation d'un mapping intermédiaire permet de réduire la taille de l'espace des règles, tout en maintenant une cohérence forte sur le sous-ensemble choisi. Par exemple la réduction du voisinage à une somme excluant la cellule centrale, conserve un sous ensemble isotropique de règle. Pour  $d = 2$ ,  $k = 2$ ,  $r = 1$  les  $2^{512}$  règles deviennent  $4^9$ , en utilisant les 4 opérations des règles minimales (Zéro, Same, Flip, One) comme valeur de la table intermédiaire.

## Le paramètre d'enchâssement

Nous définissons le paramètre d'enchâssement d'un voisin pour une règle comme l'influence de ce voisin sur la règle. Soit une règle  $R$  d'un AC de  $k = 2^n$  états,  $d$  dimensions et de rayon de voisinage  $r$ .  $N = k^{dr+1} = 2^b$  est le nombre d'entrées de la table. Pour un AC de  $k = 2$  états les  $b = dr + 1$  bits du voisinage donnent un vecteur  $E$  de  $b$  valeurs.

$$(3.1) \quad E_b = \frac{1}{N} \sum_{i=0}^{N/2-1} R_{e_{ib}} - R_{e_{ib}+2^b}$$

avec :  $e_{ib} = 2i + i \bmod 2^b$

Une valeur moyenne  $E = \frac{1}{b} \sum_{i=0}^b E_b$  peut être définie, pour servir de paramètre de contrôle à la façon du paramètre  $\lambda$  cependant d'autre réduction que la moyenne pourrait être choisie, afin d'exploiter l'information sur le degré de symétrie  $R$ .

$$\begin{aligned} E_{life} &= 0.617 \ 0.617 \ 0.617 \ 0.617 \ 0.890 \ 0.617 \ 0.617 \ 0.617 \ 0.617 \\ \hat{E}_{life} &= 0.647569 \\ \lambda_{life} &= 0.273438 \end{aligned}$$

Le problème de la structure de l'espace des AC, semble être un problème encore largement ouvert. La question des formes possibles de la dynamique des AC pour des espaces de règles de grande taille, et la question d'une combinatoire de règles permettant de contrôler l'émergence de ces formes ne possèdent pas actuellement de réponses.

## Chapitre 4

# Quelques remarques sur la règle Anneal

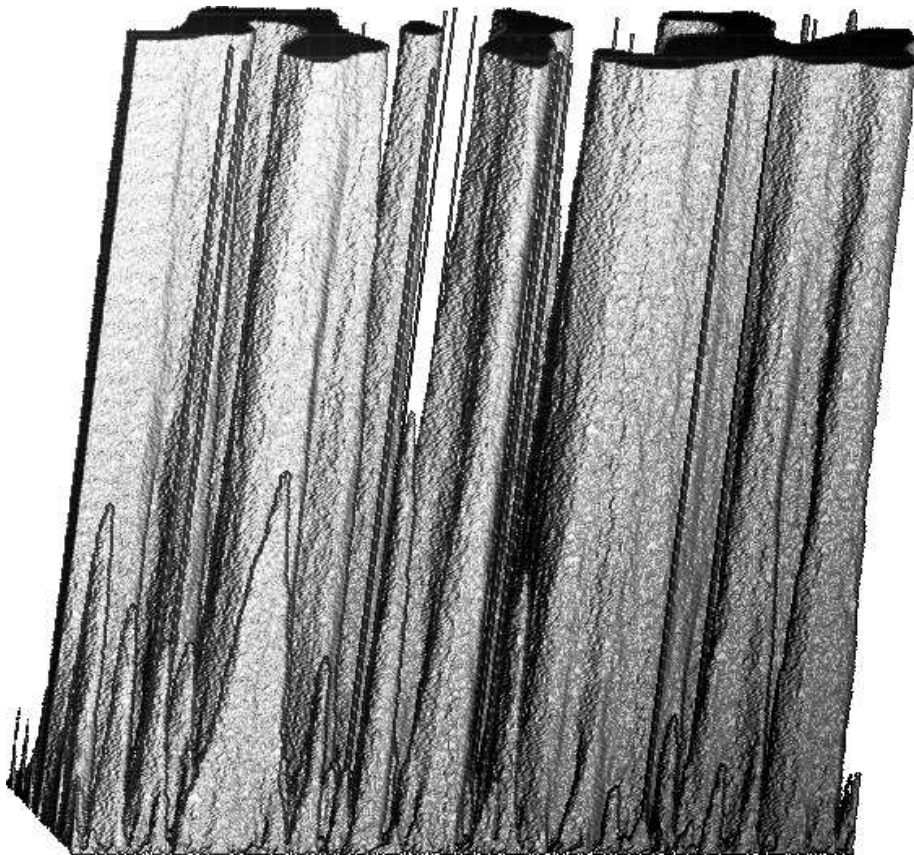
L'observation de la règle **Anneal** nous conduit à proposer l'existence d'une relative invariance spatiotemporelle des trajectoires dans l'espace des configurations d'espaces pour des configurations initiales de densité proche de  $\frac{1}{2}$ . Nous avons mesuré la densité et la taille des frontières des espaces cellulaires pour des tailles d'espaces de  $2^{20}$  cellules donnant des transitoires de plus de 100000 générations. Nous avons découvert qu'il existe des cycles limites dont la taille est de l'ordre de grandeur du rayon de l'espace cellulaire. Nous proposons plusieurs expériences permettant de compléter cette étude.

### 4.1 Trajectoires et espaces de trajectoires

Les objets de notre étude sont les automates cellulaires (AC). Les AC sont des systèmes dynamiques discrets. Le temps, l'espace et les cellules sont des entités discrètes. Une fonction de transition (règle) est appliquée de manière synchrone sur chacune des cellules de l'espace cellulaire. Cette fonction reçoit en argument une cellule et un ensemble de cellules voisines (constituant le voisinage pour une règle) au temps  $t_n$ . Elle calcule le nouvel état de la cellule (au temps  $t_{n+1}$ ).

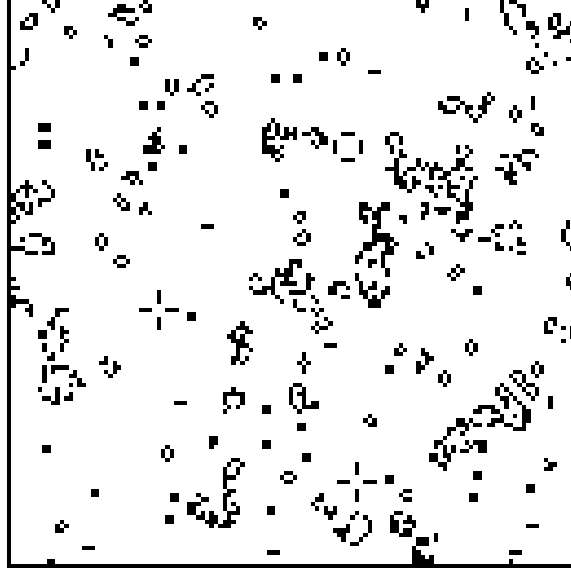
La séquence des  $E_i$  espaces générée par une règle  $R$  sur un espace initial  $E_0$  considéré comme une trajectoire dans l'espace des  $E_{s,d}$  (ou  $s$  est la taille d'un espace cellulaire fini, et  $d$  le nombre de dimension cartésienne de cet espace) états d'espaces constitue un des objets élémentaires de notre étude des AC. La règle de transition considérée comme une table en fournit un autre. La taille de cette table est  $k^{2dr+1}$  (pour une généralisation du voisinage de Moore),  $k$  étant le nombre d'états possibles pour une cellule (en

---



Cette image suggère l'intérêt d'une étude des textures produites par les plongement 3D d'AC 2D dont la dynamique, à l'image de Anneal, tend à produire des frontières.

Figure 4.1: *Une vue 3D de la dynamique de la règle Anneal*



Life sur un espace de taille  $128^2$ , au temps 128, pour un espace initial de densité aléatoire  $\frac{1}{2}$ .

Figure 4.2: *Exemple de signature d'espace cellulaire.*

pratique  $k = 2^p$  ou  $p$  est le nombre de plans de bits utile de l'espace),  $d$  le nombre de dimensions de l'espace et  $r$  le rayon du voisinage. Le passage en paramètre de  $E_0$  permet de considérer l'ensemble des trajectoires. Le passage en paramètre de  $R$  introduit l'espace des règles. Le nombre de règles est:

$$(4.1) \quad Rn_{kdr} = k^{k^{2dr+1}}.$$

Nous proposons l'étude de ces objets par le calcul des trajectoires, pour des familles échantillons sur les paramètres  $E_0$  et  $R$ .

## 4.2 Variété des objets cellulaires

Une première question porte sur la nature et la variété des  $E_i$  espaces. L'espace initial  $E_0$  peut être quelconque et sera en général<sup>1</sup> un Jardin d'Eden pour la règle  $R$ . Empiriquement, les  $E_i$  espaces possèdent une signature qui les rend aisément reconnaissable par l'observateur. La figure 4.2 par exemple sera associée à Life sans hésitation. Il convient cependant de remarquer qu'une telle appréciation ne renseigne que sur la vraisemblance de

<sup>1</sup>Statistiquement au moins.



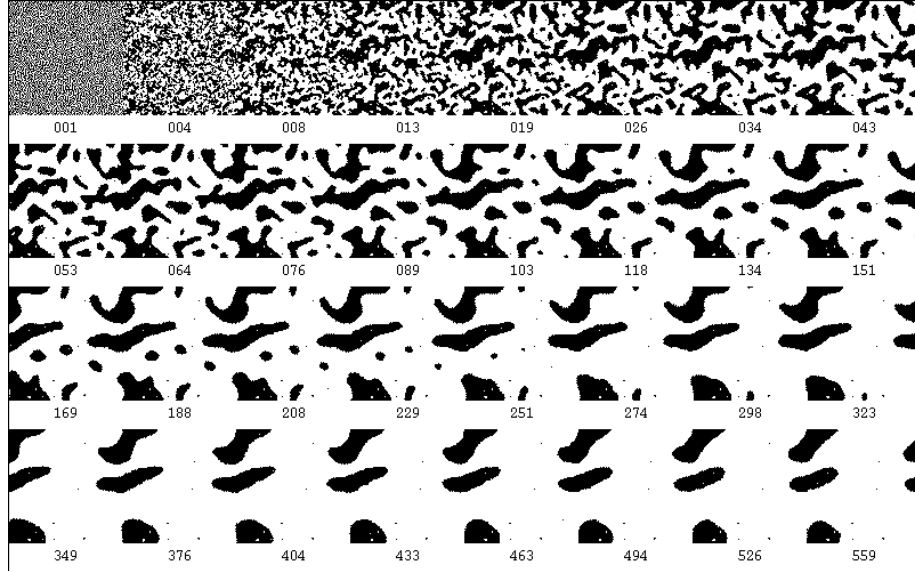
Anneal sur un espace de taille  $256^2$ , au temps 256, pour un espace initial de densité aléatoire  $\frac{1}{2}$ .

Figure 4.3: Une configuration typique de la règle *Anneal*

l'appartenance d'un espace à une trajectoire. Dans le cas le plus général la question de l'appartenance d'un espace à la trajectoire d'une règle (hors Jardin d'Eden) n'a pas de réponse en temps polynômial. En effet si la règle est universelle (ce qui est par exemple la cas de *Life*) la décision peut se ramener à l'énumération des trajectoires.

On reporte la question de la variété sur les trajectoires. Les séquences peuvent être considérés comme un espace de dimension supérieure et on recherche alors les formes associées à une règle. La localité du voisinage introduit une vitesse limite de propagation de l'information d'une cellule à une autre (une vitesse de la lumière cellulaire égale au rayon du voisinage). L'axe du temps est un axe d'observation privilégié, et les trajectoires peuvent être réduites par contraction de l'espace sur l'axe du temps. On utilise une fonction de réduction du nombre de bits d'un espace, qui joue le rôle d'une mesure effectuée sur un système physique, pour obtenir une séquence numérique temporelle. Le problème de la signature temporelle d'une règle peut alors être posé en termes statistiques. La mesure de la densité d'un espace (dans le cas le plus général, si  $k > 2$ , on peut considérer un histogramme de  $2^p$  état ou  $p$  densités) est la mesure la plus simple permettant d'obtenir une trace de la trajectoire d'une règle appliquée sur une espace initial. Le

#### 4.2. VARIÉTÉ DES OBJETS CELLULAIRES



559 générations de anneal sur un espace de taille  $256^2$ , pour un espace initial de densité aléatoire  $\frac{1}{2}$ .

Figure 4.4: Une séquence typique de la règle Anneal

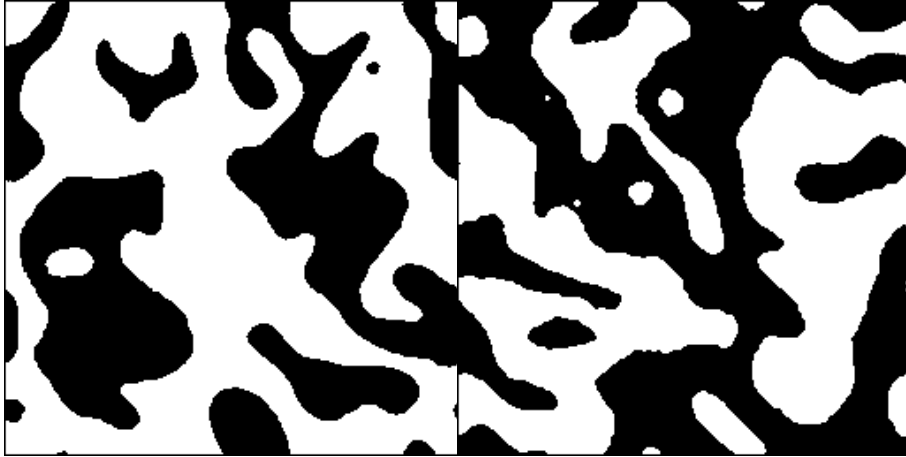
nombre des états d'espaces:

$$(4.2) \quad En_{ksd} = k^{s^d}$$

impose la réduction des espace initiaux à un sous-ensemble. Celui-ci peut être fourni par la fonction inverse d'une fonction de réduction de l'information d'un espace, utilisée comme fonction d'échantillonnage de l'espace des configurations initiales. L'étude de l'évolution de la densité par exemple, est fonction d'un paramètre de densité initiale, qui est l'inverse de la fonction de réduction utilisée, et qui permet d'effectuer un balayage statistique des états initiaux possibles associé à une distribution arbitraire. Enfin l'extension de la question de la variété à l'espace des règles (qui combine les difficultés combinatoires des équation 4.1 et 4.2) doit également s'appuyer sur l'utilisation de fonction de réduction de l'information associée à une règle, permettant le balayage des espaces de règles.

Nous pensons néanmoins qu'il serait utile de disposer d'un développement complet du calcul pour des espaces cellulaires de tailles réduites, et des espaces de règles contraintes<sup>2</sup>, en vue de l'obtention d'indices sur la nature

<sup>2</sup>Les règles additives, dont Life et Anneal sont des représentants exemplaires, semble un



anneal sur deux espaces initiaux de densité aléatoire  $\frac{1}{2}$ .

Figure 4.5: *Deux configurations de la règle anneal*

des symétrie, et des tendances structurelles de l'espace des paramètres permettant d'orienter le calcul sur les régions sensibles des espace de tailles combinatoires.

### 4.3 Quelques résultats expérimentaux à propos de Anneal

La règle Anneal est une règle additive<sup>3</sup> dont la table secondaire est:

0000101111

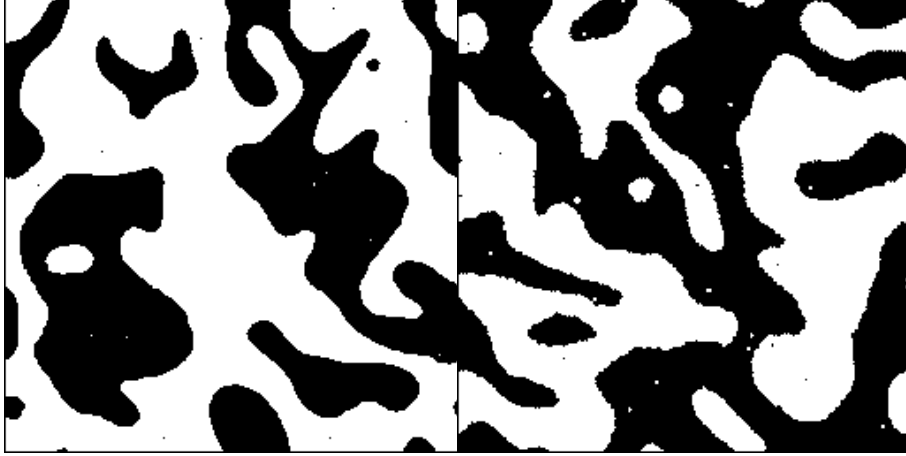
La figure 4.3 montre une configuration typique de cette règle pour un espace initial de densité  $\frac{1}{2}$ , la figure 4.4 une séquence typique de cette règle pour un espace initial de densité  $\frac{1}{2}$  et la figure 4.5 montre deux configurations de Anneal pour deux espaces initiaux de densité  $\frac{1}{2}$ .

Une propriété intéressante de cette règle est que des grandeurs caractéristiques des espaces générés, comme le nombres d'îles ou la tailles des frontières, sont indépendantes de l'échelle d'observation choisie, à condition de coupler le changement d'échelle sur l'espace à un changement d'échelle

---

bon point de départ.

<sup>3</sup>La règle est une fonction de la somme du voisinage. Voir § A, page 120.



Contrairement à la figure 4.3, les configurations locales stables ne sont plus visibles sans perte d'information.

Figure 4.6: *Les deux configurations de la figure 4.5 non retouchée*

sur le temps. La taille et l'âge des espaces de la figure 4.5 ne peuvent être déterminés simplement. En fait les espaces de la figure 4.5 ont été retouchés pour faire disparaître les configurations locales stables de quelques cellules, qui permettent de déterminer l'échelle d'espace. On admettra que cet effet tend à disparaître avec l'augmentation de tailles des espaces, et l'apparition d'un seuil au delà duquel le groupe isolé de quelques cellules n'est plus perceptible (par l'œil ou l'outil de mesure). L'espace à gauche de la figure 4.5 est de taille  $1024^2$ , son âge 4184 générations, l'espace droit est de taille  $360^2$ , son âge 600 générations.

La figure 4.6 montre les deux configurations de la figure 4.5 non retouchée. On peut remarquer que, déjà à cette échelle, les configurations locales stables ne sont plus visibles sans perte d'information, contrairement à l'échelle de la figure 4.3, où de telles configurations sont visibles.

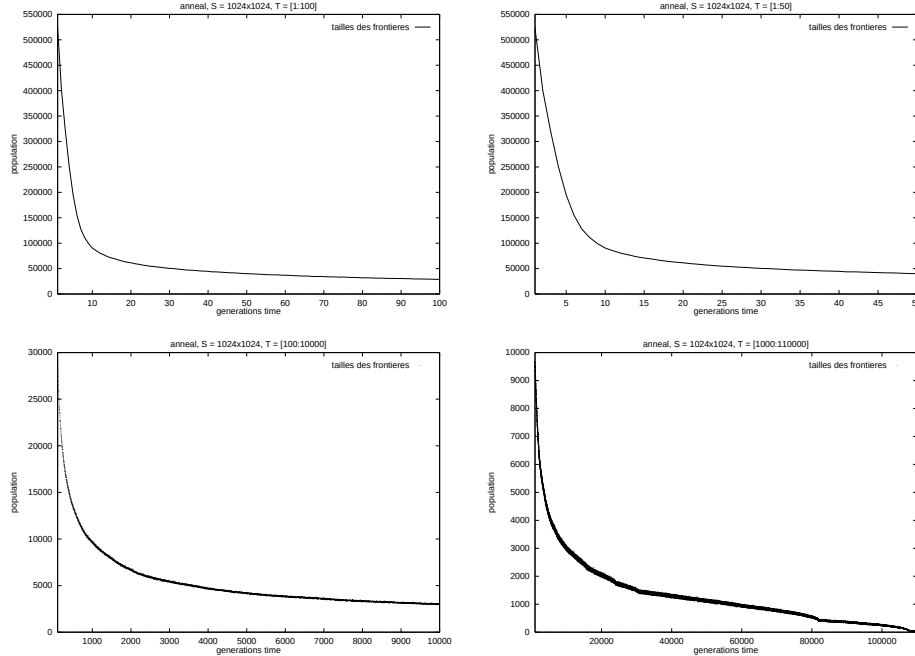
Toutes les trajectoires présentées ici font apparaître une diminution de la densité<sup>4</sup>. Cette densité est dans le cas de *Anneal* une quantité inversible<sup>5</sup>, La valeur du paramètre  $\lambda$ <sup>6</sup> Li et Packard (1990); Li et al. (1990) de *Anneal* est exactement  $\frac{1}{2}$ . Bien que nous ayons trouvé jusqu'à présent plus de tra-

<sup>4</sup>Appelée population sur les plots.

<sup>5</sup>Au sens d'une inversion photo-négative, d'un inverseur bit à bit appliqué sur un espace cellulaire avant l'opération de comptage.

<sup>6</sup>Le paramètre  $\lambda$  est défini comme le pourcentage d'entrées de la table de transition dont la valeur est différente de l'état zéro.  $\lambda = \frac{k^{dr+1} - \#0}{k^{dr+1}}$

### 4.3. RÉSULTAT EXPÉRIMENTAUX

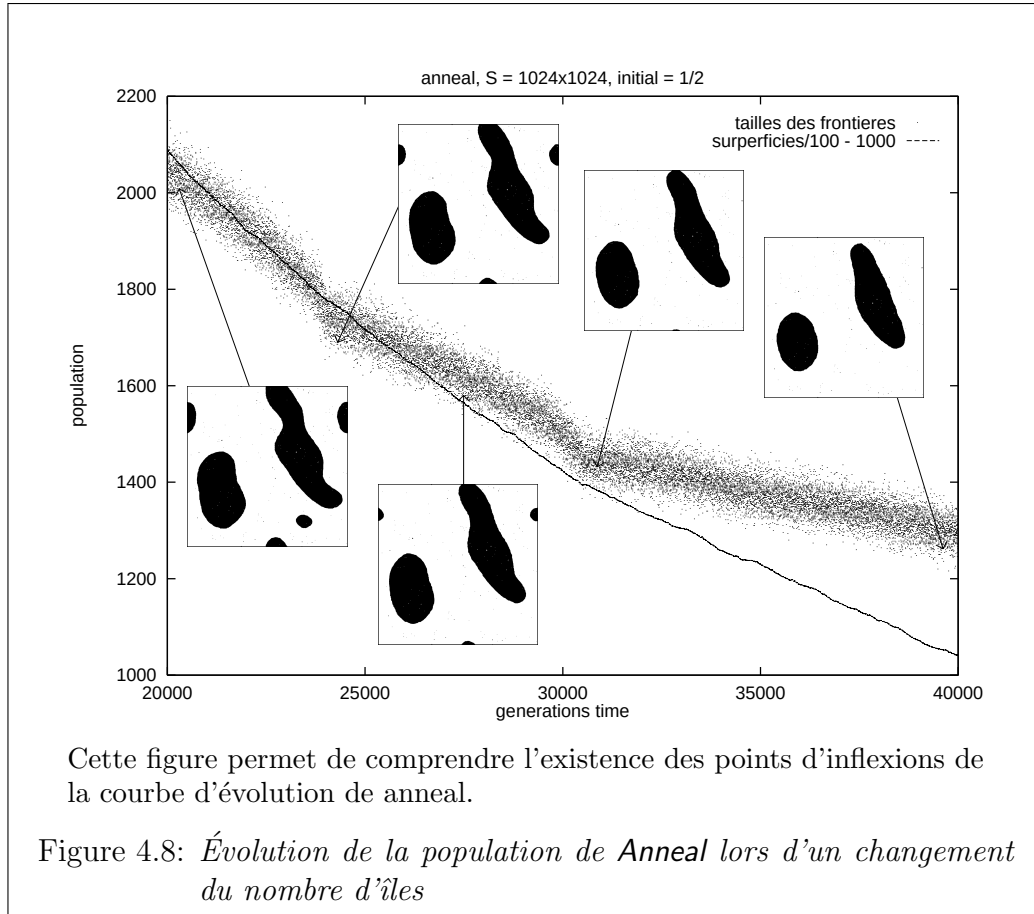


La taille des frontières est une approximation par la différence bit à bit entre l'espace de génération  $n$  et  $n - 1$ .

Figure 4.7: *Évolution de la taille des frontières de anneal*

jectoires montrant une diminution de la densité des bits à 1, nous pensons qu'un meilleur contrôle sur l'écart de la configuration initiale à la valeur  $\frac{1}{2} s^d$  permettrait de vérifier le ratio  $\frac{1}{2}$  pour les diminutions de densités.

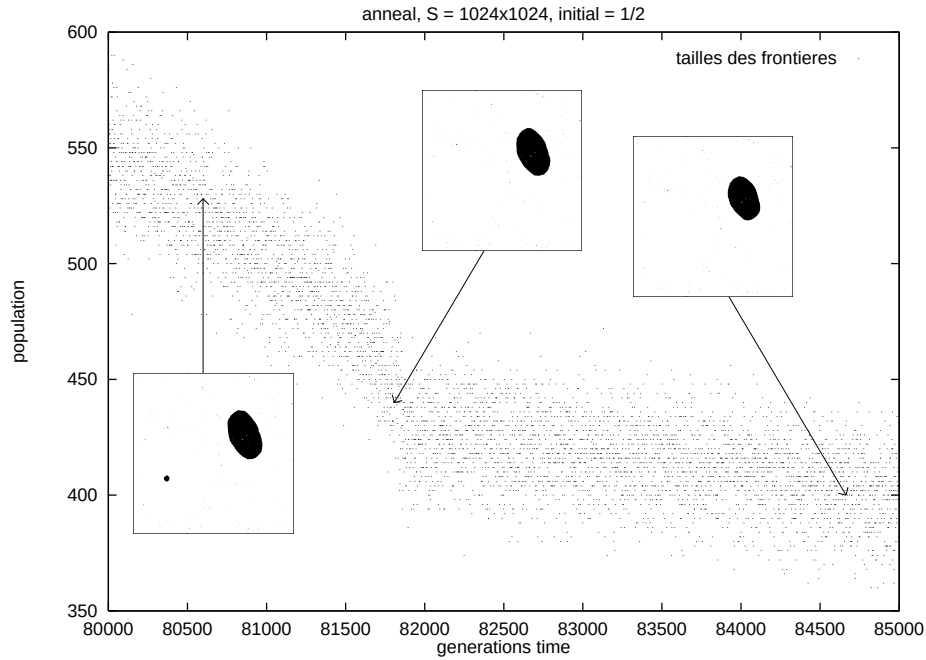
Le plot droit de la figure 4.11 montre les densités des trajectoires pour 4 espaces initiaux de tailles  $2^{128}$ ,  $2^{256}$ ,  $2^{512}$  et  $2^{1024}$ . La première atteint un point fixe en 3563 générations, les autres atteignent un cycle limite de taille 5, 768 et 24 après 6008, 29442 et 107869 générations. Les cycles limites des trajectoires 2 et 4 sont inférieurs d'un ordre de grandeur au rayon de l'espace, alors que celui de la trajectoire 3 est du même ordre de grandeur que le rayon de l'espace. De plus, il est visible sur le plot, que la densité finale du transitoire est très importante pour la trajectoire 3 contrairement aux autres, pour lesquels, tant la taille du cycle que la densité moyenne de celui-ci peuvent être considérées comme une valeur approchée d'un des deux points fixes majoritaires de **Anneal**. Les densités initiales exactes sont pour chaque trajectoire respectivement:  $8172/128^2 = 0.498779296875$ ,  $32673/256^2 = 0.4985504150390625$ ,  $130960/512^2 = 0.49957275390625$ ,  $523619/1024^2 =$



0.49936199188232421875, L'écart à  $\frac{1}{2}$  étant toujours négatif peut expliquer le destin vers zéro de toutes ces configurations. La question de la sensibilité de Anneal aux écarts de la densité  $\frac{1}{2}$ , et en particulier du rapport entre cette densité et le nombre de trajectoire n'atteignant pas un quasi point fixe, reste pour nous ouverte.

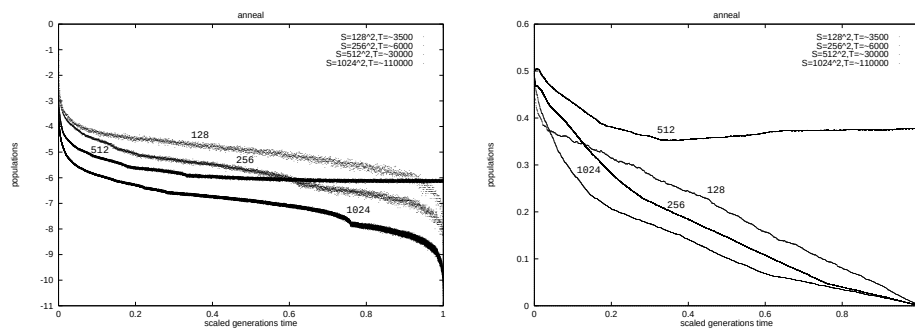
Une question suggérée par ces expériences est celle de la taille des transitoires de anneal au regard de la taille moyenne des transitoires de la famille de règle associée à un voisinage. Le problème de la fabrication d'une règle de taille de transitoire donné (éventuellement maximum), ne possède pas de réponse simple. Nous proposons de considérer la règle anneal comme prototype pour l'étude de cette question.

La figure 4.7 montre l'évolution de la taille des frontières d'une trajectoire de Anneal pour un espace de taille  $1024^2$  et de densité initiale  $\frac{1}{2}$ . La figure 4.8 montre sur les mêmes données que celle de la figure 4.7, une transition sur le nombre d'îles et les variations de la population associée.



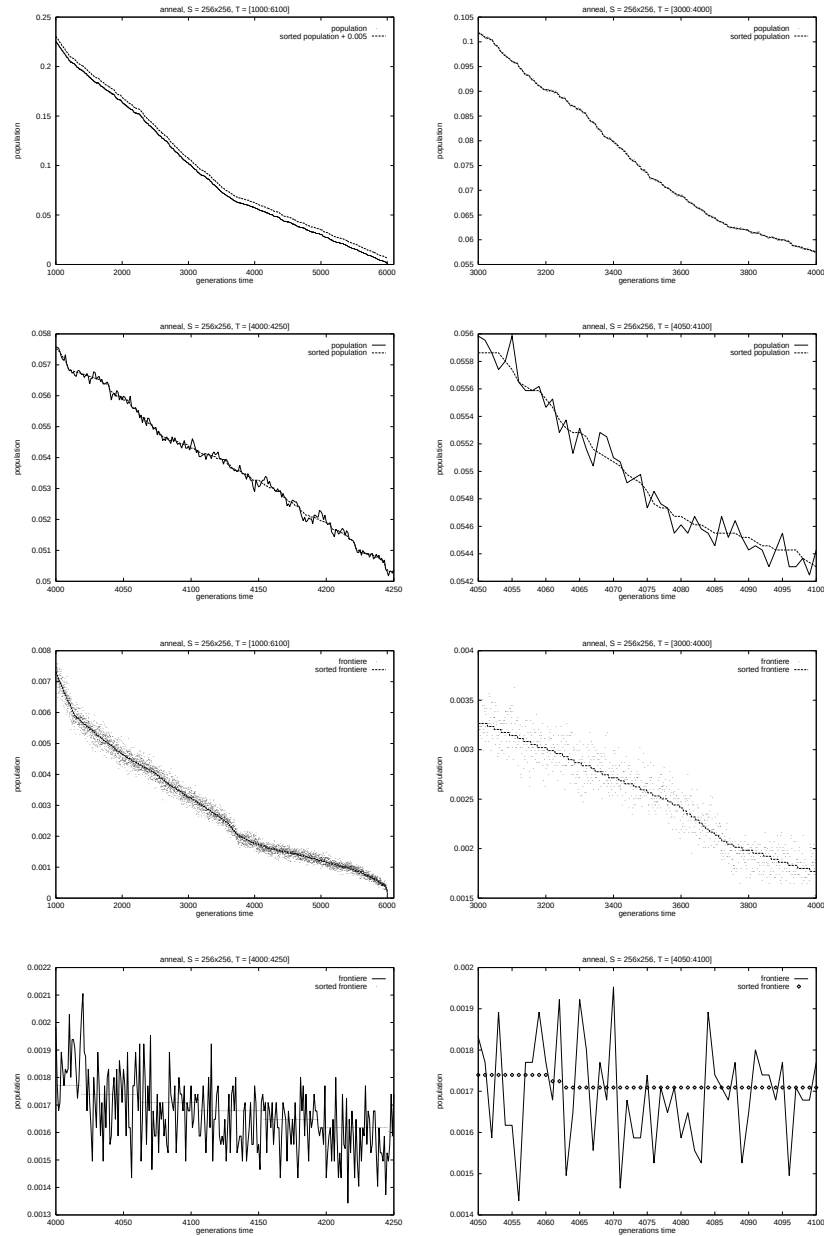
Une autre zone de transition sur les mêmes données que celle de la figure 4.8.

Figure 4.9: *Transition vers la dernière île*



Réductions intégrales normalisées des populations des trajectoires associées à 4 espaces initiaux de tailles respectives:  $2^{128}$ ,  $2^{256}$ ,  $2^{512}$  et  $2^{1024}$ . Le plot de gauche présente le logarithme de la taille des frontieres.

Figure 4.10: *Populations normalisées pour 4 espaces initiaux de tailles croissantes*



Les quatre plots du haut présentent quatre zooms successifs sur la population et la séquence triée des populations. Les quatre plots du bas zooment sur la taille des frontières.

Figure 4.11: *Une série de zooms sur la population et la taille des frontières*

# Chapitre 5

## Conclusion

Nous avons exposé les raisons pour lesquelles nous défendons une approche bottom-up générale en vue de la réalisation d'un environnement de développement cellulaire à vocation universelle. L'ampleur de la tâche, qui reste largement inachevée, mais aussi l'expérience acquise, nous conduit à préciser quelques directions privilégiées de développement ultérieur.

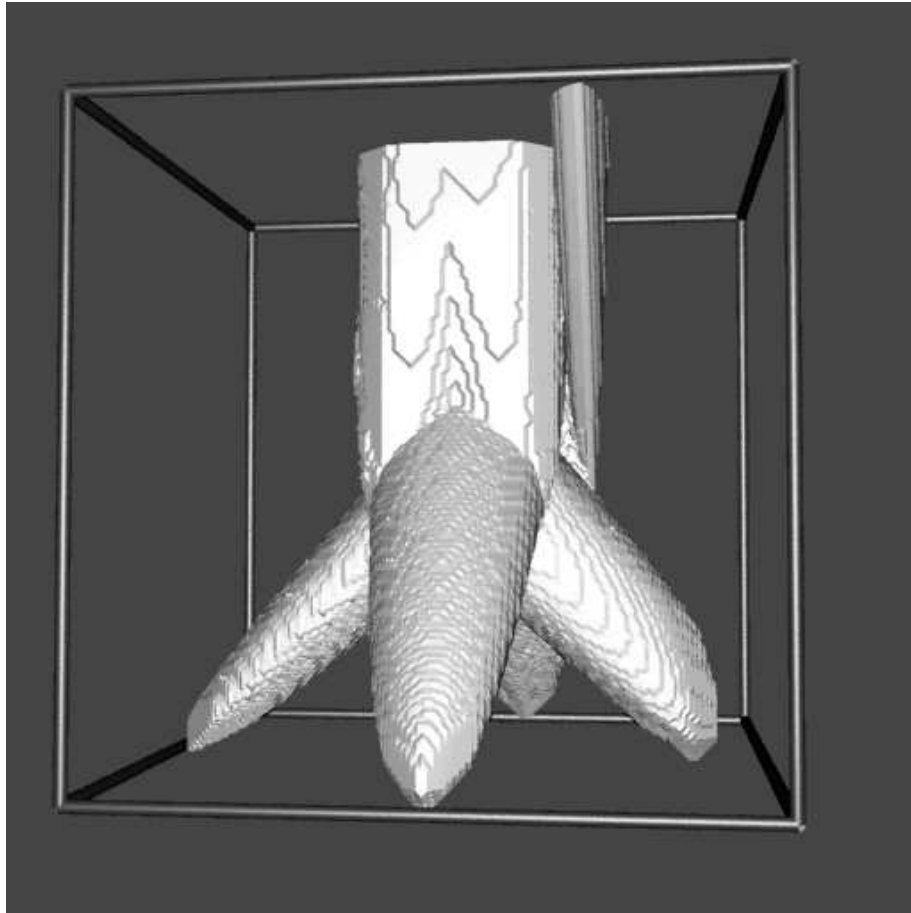
Les principaux résultats portent sur l'obtention d'un moteur cellulaire rapide et sur la validation de plusieurs outils bien adaptés à l'analyse et la présentation visuelle des dynamiques générées par le Workbench.

La réalisation des mécanismes de spécification des architectures cellulaires pose un problème de bootstrap. Ce problème se pose également pour la réalisation d'une bibliothèque d'objets cellulaires élémentaires. Ainsi, nous pensons qu'il convient de consolider le noyau de fonctionnalités acquises, par une intégration plus étroite de quelques moteurs cellulaires simples et rapides aux environnements externes, tout en poursuivant la recherche des méthodes utilisables pour la définition d'architectures plus générales. Néanmoins, la réalisation d'outils puissants (rapides et flexibles) de manipulation de tables de transitions, n'éloignerait pas la tentation d'expérimenter des AC moins radicalement bottom-up.

Nous présentons donc en conclusion un AC dont la réalisation effective exclut l'usage d'une combinatoire de tables de transitions. Cet AC utilise un voisinage de von Neumann, l'horloge interne est à deux pas, et la structure cellulaire est à six champs. Chaque champ est un entier, dont la taille est, dans notre réalisation, de 32 bits. La fonction de transition est exprimée<sup>1</sup> en terme de modification simultanée des cinq cellules du voisinage, avec consultation de l'état local uniquement. La règle de transition réalise une *physique* énergétique qualitative. Les quatre premiers champs de la variable d'état

---

<sup>1</sup>Contrairement à toutes les autres règles spécifiées dans la présente étude.



Un cube d'espace-temps d'arête 128, rendu par ray-tracing d'iso-surface. De tels traitements sont, avec les outils que nous avons utilisés à la limite des capacités des ressources dont nous disposions.

Figure 5.1: *La dynamique de l'AC  $xu$*

cellulaire associent des poids aux quatre directions de l'espace (nord, sud, est et ouest). Les deux derniers champs associent des potentiels cinétiques aux axes nord/sud et est/ouest. Les poids sont des données initiales. La somme des poids sur l'ensemble des cellules de l'espace reste invariante par conservation de l'énergie. Les potentiels associés aux axes NS et EO sont fonction du cumul des poids des deux directions de chaque axe. Le rapport de cette quantité à la somme des poids des quatre directions détermine le déplacement additif, champ par champ, de la cellule vers une de ses quatre voisines par fusion. Ce mouvement-fusion a lieu au temps 2 de chaque cycle.

L'opération de fusion porte également sur les potentiels, dont les reliquats sont conservés.

Par exemple pour l'axe EO:

```

/* Code du choix de direction du mouvement */
dx = icell->poids[EST] - icell->poids[OUEST];
icell->potentiel[EO] += dx;

if (abs(icell->potentiel[EO]) >= sum) {
    if (dx > 0) {
        icell->potentiel[EO] -= sum;
        x = modx(x + 1);
    } else {
        icell->potentiel[EO] += sum;
        x = modx(x - 1);
    }
}

```

Au temps 1 de chaque cycle, la somme est comparée à la constante universelle  $K$  de notre physique. Si la somme est supérieure à  $K$ , la cellule explose. Cette explosion distribue la cellule dans les quatre voisins par l'opération de fusion. La division de la cellule en quatre parties est une division entière: le reliquat de cette division permet d'ajouter la rupture de symétrie à la conservation de la quantité allouée au temps zéro. Le choix de la distribution du reliquat de cette division constitue avec le choix de l'espace initial et de la valeur de  $K$  les seuls paramètres contrôlables. Un reliquat subit une rotation et une translation relative à sa position cellulaire. Les exemples de dynamique exposés le sont pour  $K = 4$ , rotation = clockwise next, translation = clockwise next.

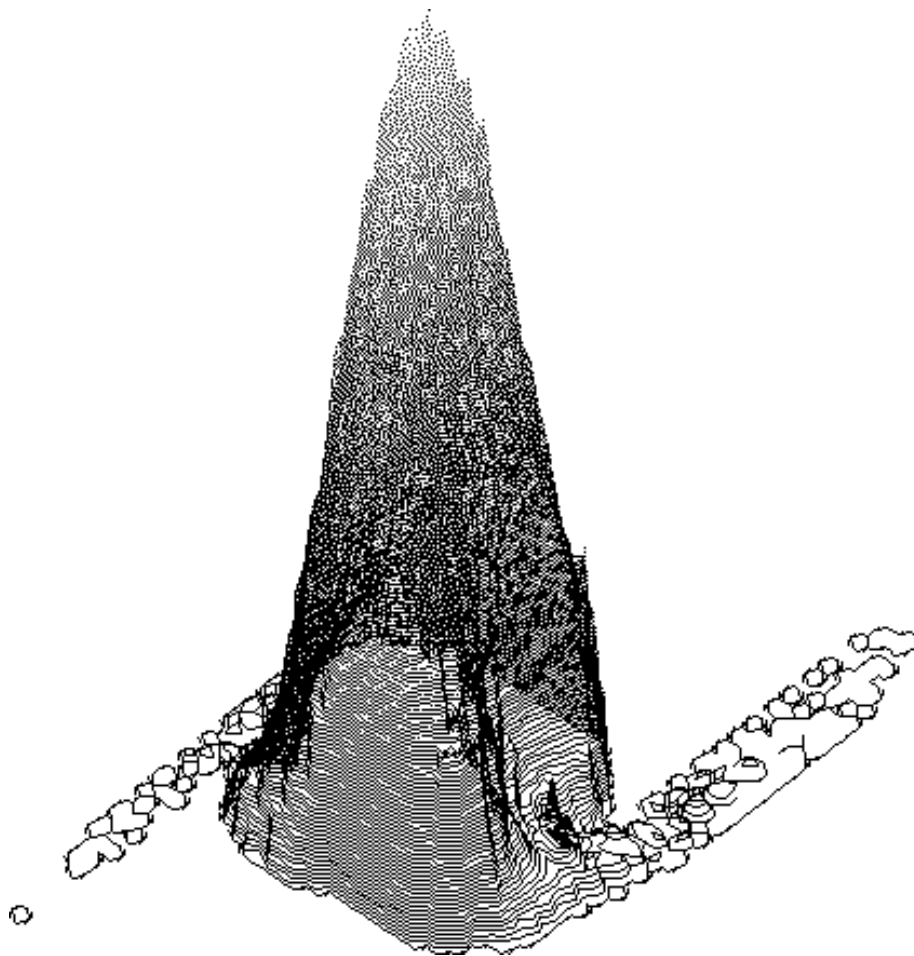
la figure 5.1 est une photo de l'évolution (de bas en haut) de l'espace initial suivant:

| X   | Y   | N      | S      | E      | O      |
|-----|-----|--------|--------|--------|--------|
| -60 | 0   | 0      | 100    | 100000 | 0      |
| 60  | 8   | 0      | 100    | 100    | 100000 |
| 0   | -60 | 0      | 100000 | 0      | 0      |
| 0   | 48  | 100000 | 0      | 200    | 400    |

de taille 128 par 128, sur 128 générations. L'origine du repère est au centre.

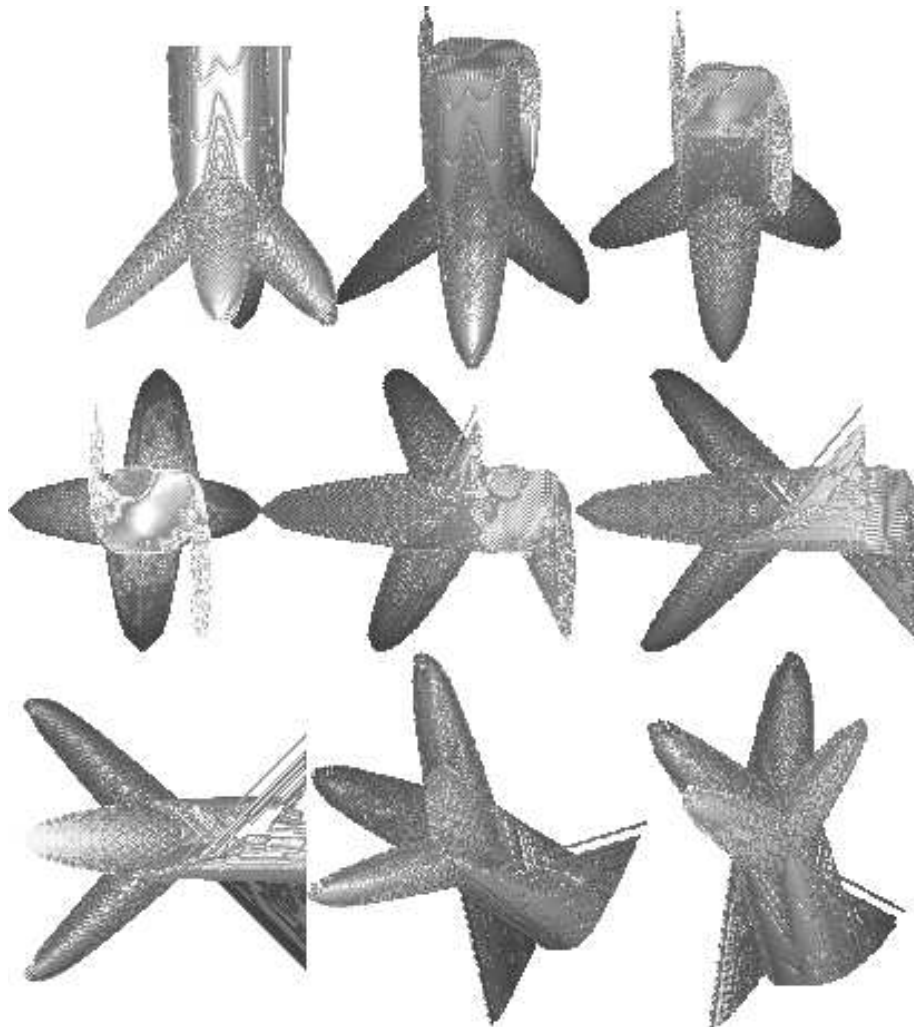
Les réalisations et propositions que nous avons exposées constituent des éléments d'une étude pratique des espaces du calcul cellulaire reposant sur un usage généralisé de tables de transitions qui permet la construction de moteurs rapides pour le calcul cellulaire sur machines séquentielles. Nous avons montré comment cette approche permet des transformations cellulaire-SIMD. Nous avons également proposé l'usage d'un nouveau paramètre de contrôle sur les espaces de règles et des techniques de visualisation spéciales pour les espaces-temps cellulaire. Nous comptons utiliser ces éléments pour base de nouveaux développements intégrant la dimension esthétique des visualisations comme méthode d'évaluation heuristique des propriétés morphogénétiques des régions d'espaces du calcul cellulaire visitées.

---



La photo 5.1 permet de se faire une idée de la dynamique de croissance de la règle, mais elle ne permet pas de distinguer le pattern de distribution énergétique de chaque tranche de temps.

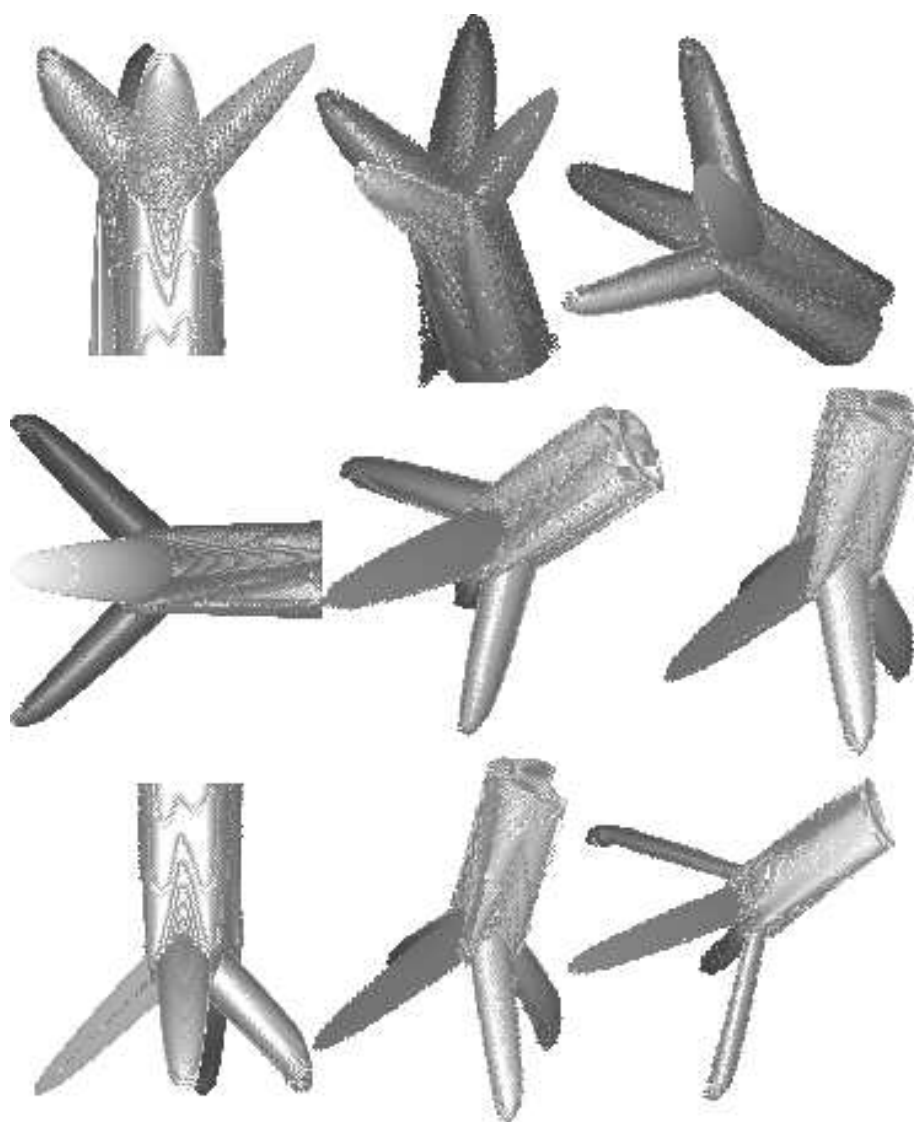
Figure 5.2: *Un rendu d'une tranche des valeurs du dernier plan de la figure 5.1*



La présentation d'une photo d'une projection 2D d'un plongement 3D de la dynamique d'un règle sur un espace cellulaire, ne remplace pas l'expérience d'une présentation incarnée dans le temps<sup>a</sup>, qui plus est, ces photos appellent par elle-mêmes le besoin d'un film de la variation des paramètres de projections.

<sup>a</sup>A priori un film classique, respectant le temps propre de la dynamique, mais sans exclure la possibilité que des distortions de l'axe de projection temporelle soient porteuse d'informations pertinentes.

Figure 5.3: *D'autres points de vue*



Réalisé en variant linéairement la valeur de seuil d'extraction d'iso-surface

Figure 5.4: *Et un épluchage*

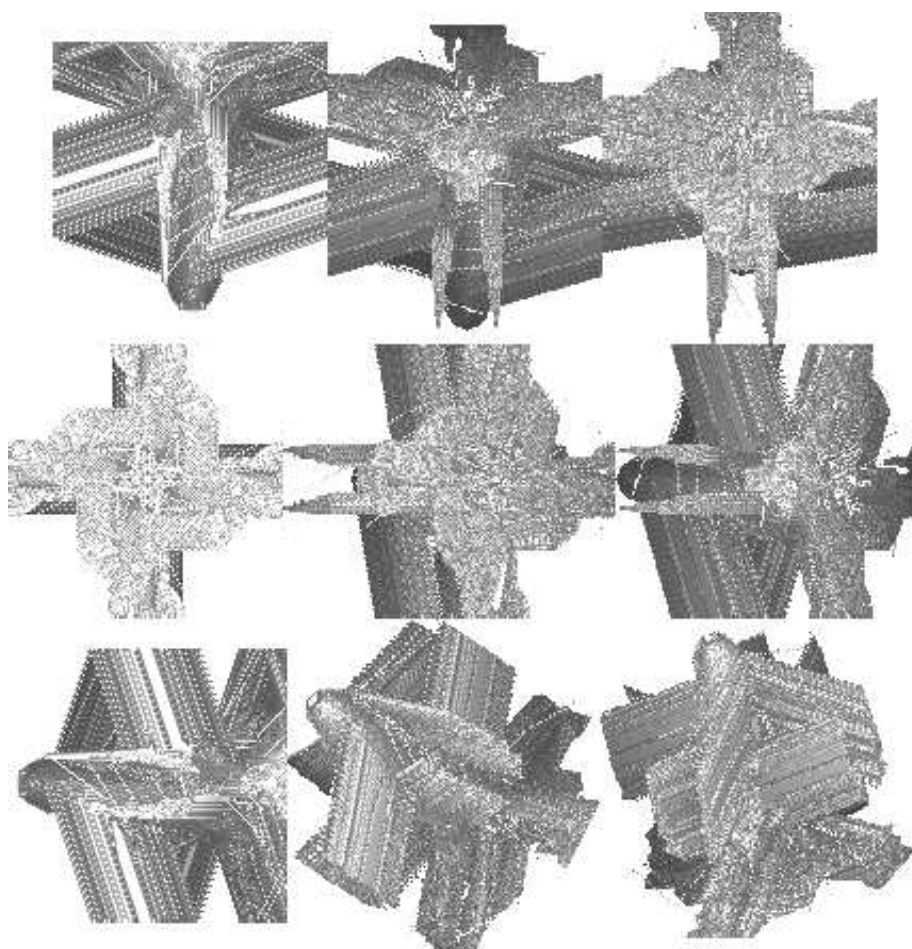


Figure 5.5: *Une autre configuration initiale*

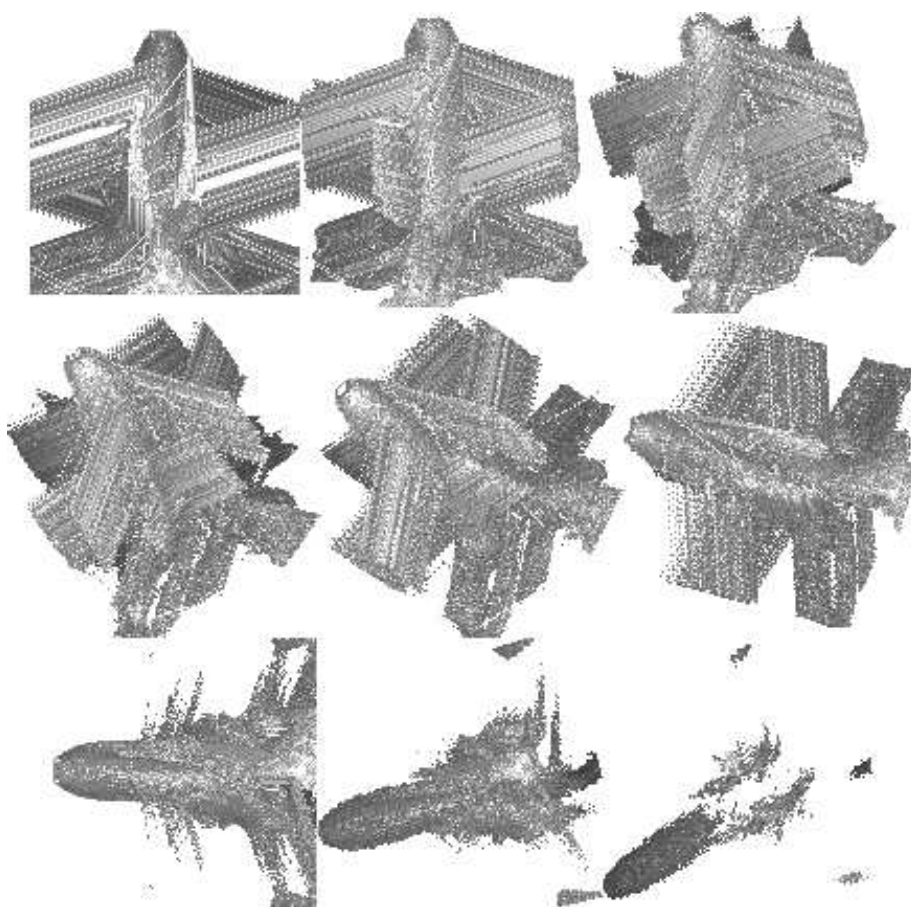


Figure 5.6: *Et un épluchage exponentiel*

# Annexe A

## Les fonctions de transitions utilisées

La règle Xor9:

Center = Center  $\oplus$  north  $\oplus$  south  $\oplus$  east  $\oplus$  west  $\oplus$

Neast  $\oplus$  Nwest  $\oplus$  Swest  $\oplus$  Seast

```
/* Construct Mean */  
tmp = (SUM + C) / 9;  
/* Destruct it */  
tmp = tmp & mask == match? tmp: tmp + delta;
```

Nous avons expérimenté plusieurs paramètres statiques<sup>a</sup> pour cette règle qui nous a servi initialement à tester le moteur à voisinage spécial décrit en 2.25. Les paramètres utilisés dans les illustrations de la 2.1.5 sont: mask = 3; match = 0; delta = -1. Ces paramètres sont parmi les premiers que nous ayons essayés, et donne un des comportements les plus riches<sup>b</sup> que nous ayons trouvés pour cette famille<sup>c</sup> de règles.

---

<sup>a</sup>Voir § 2.3.5, page 72.

<sup>b</sup>Intuitivement, en remarquant cependant que ce paramétrage est inspiré par la recherche d'une opposition construction / destruction, égalisation / différenciation, etc. typique d'un règle telle Life, et évocatrice de phénomènes de frontières.

<sup>c</sup>La famille est ici comprise comme l'espace de règles engendré par un système de paramètres statique pour un algorithme de génération de table de transition.

Figure A.1: *Le cœur de la règle Qmean*

```

int* sum_tab_parse(int ac, char** av, int n) {
    static int tab[10];
    int i;

    if (ac != 1) { return 0; }
    if (strlen(av[0]) != n) { return 0; }
    for (i = 0; i < n; ++i) {
        switch (av[0][i]) {
            case '0': tab[i] = 0; break;
            case '1': tab[i] = 1; break;
            case 'U': tab[i] = 2; break; /* Unchanged */
            case 'F': tab[i] = 3; break; /* Flip */
            default: return 0;
        }
    }
    return tab;
}

int* sum9_tab_parse(int ac, char** av) {
    return sum_tab_parse(ac, av, 10);
}

Moore_sum9_tab(Moore* i, Local* o, int* tab) {
    static int T[] = { 0, 1, -1, -1 };
    Local sum;

    T[2] = i->c;
    T[3] = !i->c;
    Moore_sum9(i, &sum);
    *o = T[tab[sum]];
}

```

Le source de la fonction C implémentant la fonction de transition de la règle Sum9, qui reçoit en argument statique une table secondaire décrivant une règle additive. Voir §C, page 128.

Figure A.2: La règle *Anneal* est implémentée via la règle *Sum9*

```

Func cmd sum9_tab \
    [ConexSpec $moore_1 $local_1 moore_1] \
    Moore_sum9_tab sum9_tab_parse

sum9_tab bind 0000101111

```

Anneal est décrite sous les noms: M46789 dans Vichniac (1986), Lava Lamp (une version 3D) dans Pickover (1992), Anneal dans Toffoli et Margolus (1987). Il s'agit d'une modification de la règle dite Vote Majoritaire Toffoli et Margolus (1987), qui *recuit* la dynamique du vote majoritaire par échange des bits 5 et 6.

Figure A.3: L'interface de binding Tcl pour Anneal

```

tube_worm() /* moore-1+2 */
{
    /* set plane 0, 1 et timers only to random */

    /* try looking only frame n/(4 + timer) */

    /* try with timer = 5 */
    static int alarm_tab0[] = { 0, 1, 1, 1, 1, 1, 1, 1, 1, };

    /* try with timer = 2, 3, 4 or 5 */
    static int alarm_tab1[] = { 0, 0, 1, 1, 1, 1, 1, 1, 1, };

    /* try with timer = 3, 4 or 5 */
    static int alarm_tab2[] = { 0, 0, 0, 1, 1, 1, 1, 1, 1, };

    /* try with timer = 1, 2 (size >= 256), 3, 4 or 5 */
    static int alarm_tab3[] = { 0, 0, 1, 0, 1, 1, 1, 1, 1, };

    static int* alarm_tab[] = { alarm_tab0, alarm_tab1, alarm_tab2, alarm_tab3, };
    static int timer_tab[] = { 0, 0, 1, 2, 3, 4, 5, 6, 7, };

    int iworm, oworm, ialarm, oalarm, itimer, otimer, alarm, timer, ret;

    alarm = num_arg_tab[0];
    timer = num_arg_tab[1];

    if (!alarm && ! timer) {
        alarm = 2;
        timer = 3;
    }

    iworm = C;
    ialarm = L & 1;
    itimer = (L & 7 << 1) >> 1;

    oworm = !itimer;
    oalarm = alarm_tab[alarm][_8sum()];
    otimer = iworm & ialarm ? timer : timer_tab[itimer];

    ret = oworm | oalarm << 1 | otimer << 2;

    return ret;
}

```

Le source de la fonction C implémentant la fonction de transition de la règle Tube-Worm, d'après Toffoli et Margolus (1987).

Figure A.4: *La règle Tube-Worm*

# Annexe B

## Prédateur/proie

Existe-t-il une méthode de transformation systématique<sup>1</sup> d'une description différentielle continue d'un système en équation cellulaire? Confronté au problème de la découverte d'une règle de transition codant un comportement<sup>2</sup>, la démarche usuelle repose plus sur la recherche intuitive d'une description phénoménologique locale, que sur l'utilisation combinatoire d'un répertoire de primitives doté d'un comportement prévisible<sup>3</sup>. Partant d'un comportement global donné, cette démarche intuitive postule l'existence d'un comportement local associé à un observable global. Une règle locale est obtenue par la simplification symbolique d'une description analytique. Comme exemple de l'utilisation de cette méthode empirique, prenons un problème classique de dynamique des populations. Nous avons choisi un système déjà fort complexe, permettant une formulation cellulaire: un système proie/prédateur à deux espèces. Le squelette de la fonction de transition s'obtient aisément par la description littérale des comportements individuels supposés d'une population naturelle:

- Les individus naissent, durent et meurent.
- Les proies meurent en présence des prédateurs.

---

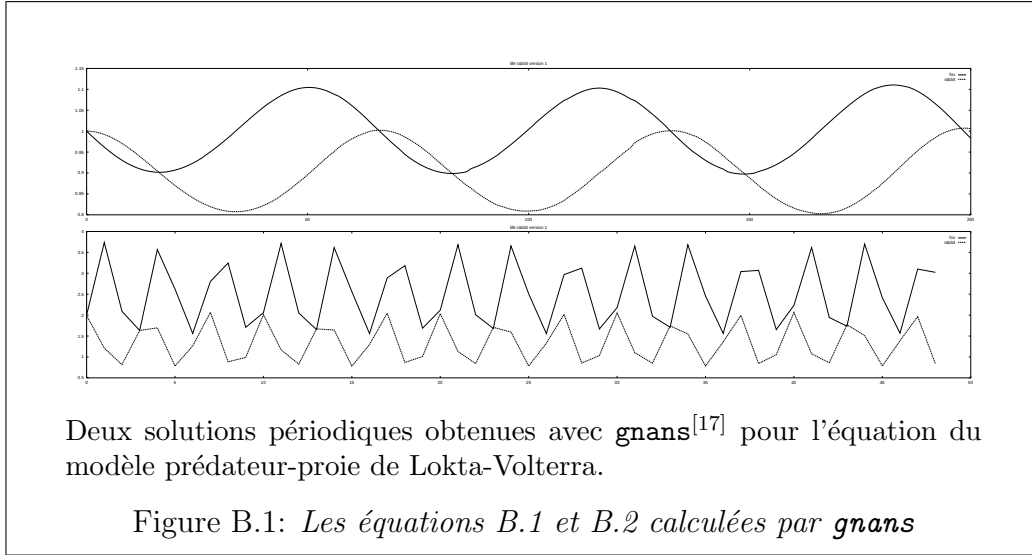
<sup>1</sup>Pouvant à terme devenir un algorithme ou au moins une heuristique.

<sup>2</sup>Le problème de la réduction d'un comportement à une dynamique, ou plus généralement la détermination des observables de l'espace-temps, nous amène à une utilisation plus métaphorique de termes désignant les données initiales ou résultantes d'une expérience cellulaire.

<sup>3</sup>Voir Richards et al. (1990) pour une recherche de règles à partir d'espaces de configurations, utilisant un algorithme génétique pour la recherche dans l'espace des règles, mais concernant des AC probabilistes.

---

Un bit d'existence et  $n$  bits de longévité sont alloués dans l'espace cellulaires à chaque espèce<sup>4</sup>. La naissance d'un individu dépend de la densité de la population locale. Les paramètres facteurs de croissance et de décroissance de chaque population ne sont donc pas fixés de manière absolue, mais résultent de la conjonction du compteur de longévité et des seuils locaux déclenchant une naissance. A l'observation, les variations de population résultantes présentent l'aspect cyclique corrélé attendu, mais le répertoire des comportements accessibles via les paramètres de la fonction de transition ou par des modifications du squelette de l'automate<sup>5</sup> est difficilement comparable à l'expression différentielle classique de ce problème.



La figure B.1 est obtenue avec **gnans**<sup>[17]</sup> pour l'équation du modèle prédateur-proie de Lokta-Volterra. D'après Murray (1990):

$$(B.1) \quad \frac{dN}{dt} = N(a - bP),$$

$$(B.2) \quad \frac{dP}{dt} = P(cN - d),$$

Avec  $a$ ,  $b$ ,  $c$  et  $d$  constantes positives. Les équations B.1 et B.2 sont spécifiées tel que à **gnans**:

<sup>4</sup>Nous avons utilisé des compteurs sur 3 bits.

<sup>5</sup>Par exemple en imposant la présence de proies dans le voisinage d'une cellule, en plus de la présence de prédateurs, comme une condition nécessaire à la naissance d'un prédateur.

```
CONTINUOUS TIME SYSTEM lokta;
```

```
AT TIME t:
d(n) = n * (a - b * p);
d(p) = p * (c * n - dd);
```

Les valeurs des variables et des paramètres sont, pour la solution 1<sup>6</sup>:

```
STATE n = 2;
STATE p = 1.99;
TIME t;

PARAMETER a = 4;
PARAMETER b = 3;
PARAMETER c = 2;
PARAMETER dd = 5;
```

Et pour la solution 2<sup>7</sup>:

```
STATE n = 1;
STATE p = 1;
TIME t;

PARAMETER a = .8;
PARAMETER b = .1;
PARAMETER c = .1;
PARAMETER dd = .1;
```

La figure B.2 montre l'évolution des populations proie-prédateurs obtenue avec la règle Fox-Rabbit version 1, pour 100 générations sur un espace taille  $128 \times 128$  (haut) et  $512 \times 512$  (bas). Dans les deux cas, l'espace initial est de densité aléatoire  $\frac{1}{2}$  sur le plan *fox* et sur le plan *rabbit*.

Fox-Rabbit version 1:

```
fox_sum = W0 + N0 + S0 + E0;
rabbit_sum = W1 + N1 + S1 + E1;
ofox = ifox;
orabbit = irabbit;

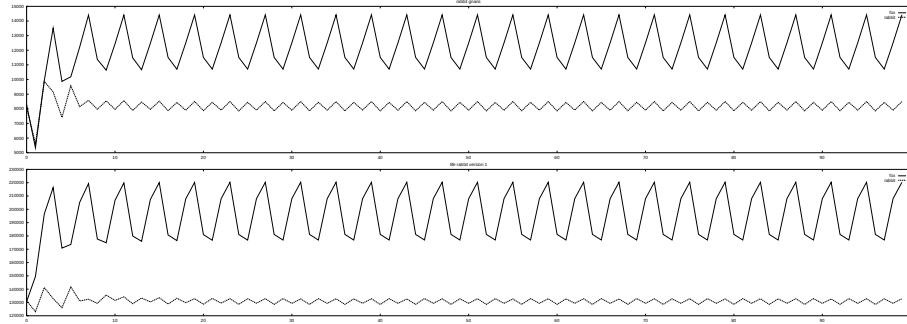
if (ifox && irabbit) {
    orabbit = 0;
}
if (ifox) {
    ofox_life = ifox_life - 1;
```

---

<sup>6</sup>Et les conditions opérationnelles: timestep = 1, stoptime = 48, algo = RKF45.

<sup>7</sup>Timestep = 1, stoptime = 200, algo = Ode.

---



Évolution des populations proie-prédateurs obtenue avec la règle Fox-Rabbit version 1

Figure B.2: *Une tentative transposition cellulaire*

```

        ofox = ofox_life > 0;
    }
    if (irabbit && !ifox) {
        orabbit_life = irabbit_life - 1;
        orabbit = orabbit_life > 0;
    }
    if (!ifox && fox_sum >= 2 && rabbit_sum >= 1) {
        ofox = 1;
        ofox_life = 3;
    }
    if (!irabbit && rabbit_sum >= 2) {
        orabbit = 1;
        orabbit_life = 3;
    }
}

```

La figure B.3 montre l'évolution des populations proie-prédateurs obtenue avec la règle Fox-Rabbit version 2 (haut) et 3 (bas). Pour 200 et 400 générations sur un espace taille  $128 \times 128$ , même espace initial que pour la figure B.2.

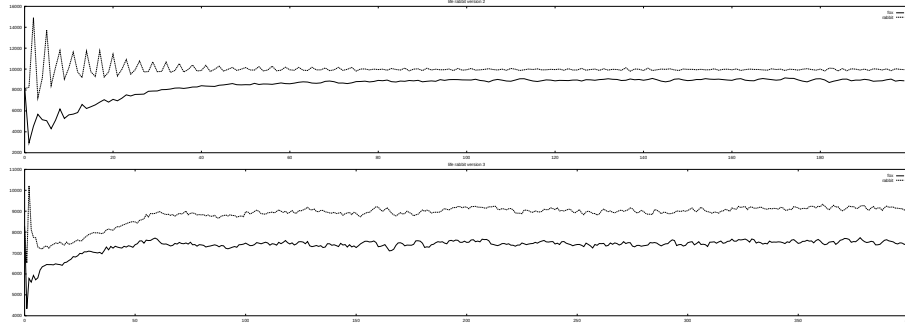
Fox-Rabbit version 2<sup>8</sup>:

```

    if (ifox && irabbit) {
        ofox_life = ifox_life - 1;
        ofox = ofox_life > 0;
        orabbit = 0;
    }
    if (ifox && !irabbit) {

```

<sup>8</sup>La dernière condition de création d'un rabbit est suspecte!



Nous considérons la variété des comportements dynamique obtenue pour les perturbations apportés à la règle initiale<sup>a</sup> comme un indicateur de l'intérêt d'une recherche empirique d'une typologie métaphorique des dynamiques cellulaires, i.e. d'une tentative de classification des scénarios utilisés pour établir des expériences cellulaires.

<sup>a</sup>Compte tenu de la taille du voisinage (2 plans von Neumann de 1 bit chacun et 6 bits locaux).

Figure B.3: *Fox-Rabbit Versions 2 et 3*

```

    ofox_life = ifox_life - 3;
    ofox = ofox_life > 0;
}
if (irabbit && !ifox) {
    orabbit_life = irabbit_life - 1;
    orabbit = orabbit_life > 0;
}
if (!ifox && fox_sum == 2 && rabbit_sum >= 1) {
    ofox = 1;
    ofox_life = 7;
}
if (!irabbit && (rabbit_sum >= 2 || rabbit_sum <= 3)) {
    orabbit = 1;
    orabbit_life = 2;
}

```

Fox-Rabbit version 3:

```

if (ifox && irabbit) {
    if (fox_sum > rabbit_sum) {
        ofox_life = ifox_life - 1;
        ofox = ofox_life > 0;
        orabbit = 0;
    } else if (fox_sum < rabbit_sum) {

```

```
        orabbit_life = irabbit_life - 1;
        orabbit = orabbit_life > 0;
        ofox = 0;
    }
}
if (ifox && !irabbit) {
    ofox_life = ifox_life - 2;
    ofox = ofox_life > 0;
}
if (irabbit && !ifox) {
    orabbit_life = irabbit_life - 2;
    orabbit = orabbit_life > 0;
}
if (!ifox && fox_sum == 2) {
    ofox = 1;
    ofox_life = 7;
}
if (!irabbit && rabbit_sum >= 2 && fox_sum >= 1) {
    orabbit = 1;
    orabbit_life = 3;
}
```

---

# Annexe C

## Exemple de code Tcl top-level

Tcl<sup>[18]</sup> est un langage de commande et une librairie utilisable comme interface de programmes interactifs.

```
# moore and vn are standard Moore and von Neumann neighborhood
set moore [ConexType moore 9]
set vn [ConexType vn 5]
# local mean only in center cell
set local [ConexType local 1]

# one and four state Moore
set moore_1 [ConexList [ConexVarList [ConexVar $moore 1]]]
set moore_2 [ConexList [ConexVarList [ConexVar $moore 2]]]
# von Neumann neighborhood allows 8 states per cells
set vn_3 [ConexList [ConexVarList [ConexVar $vn 3]]]
# local bits are cheap
set local_4 [ConexList [ConexVarList [ConexVar $local 4]]]

# these means one rule per planes
set moore_1_moore_1 \
    [ConexList [ConexVarList [ConexVar $moore 1] [ConexVar $moore 1]]]

# the thirds arg is the name of an application function
set Moore_1 [ConexSpec $moore_1 $local_1 moore_1]
set Moore_2 [ConexSpec $moore_2 $local_2 moore_2]
set VN_1 [ConexSpec $vn_1 $local_1 vn_1]
```

Les fonction Tcl sont associé à des fonctions C créant les objets permettant la composition des spécification de connexions de voisinage.

Figure C.1: *Création des structures de données pour le voisinage*

La figure C.3 présente du code Tcl de création et manipulation de table de transition. On peut y voir la liaison des règles `lnk`, `maj` et `anneal`. Ce sont des règles additives. Leurs lookup-tables peuvent être spécifiées par une table secondaire dont le nombre d'entrées correspond au nombre de voisins.

---

Les entrées de cette table secondaire codent la nouvelle valeur de la cellule, cette valeur est soit un des états de la règle (ici les règles sont binaires), soit un code désignant une fonction de génération de la nouvelle valeur. Dans le cas présent seul le code “U” pour *Unchanged* est utilisé. Pour des règles binaires nous proposons d’étudier, et avons expérimenté la seule autre fonction possible: “F”, pour *Flip* bit. L’intérêt de cette approche est de permettre un échantillonnage de l’espace des règles pour les voisinages classiques<sup>1</sup> dont la taille permet une exploration systématique. Le nombre de règles d’un des AC binaire pour un voisinage de Moore reste prohibitif, même en considérant une réduction par symétrie (Wolfram, 1983) et notre plongement de règle; le nombre de règles additives spécifié par une table secondaire<sup>2</sup> binaire est  $2^9$  ou  $2^{10}$  (Selon que l’on considère le centre ou pas dans les termes de la somme); le nombre de règles spécifié par un table secondaire utilisant les deux transformations binaires possible (U et F), en plus des deux constantes d’états, est  $2^{20}$ . L’idée est d’utiliser cette technique en augmentant le nombre d’états et en modulant le choix des fonctions de transformation figurant dans la table secondaire pour obtenir un espace de règles dont la taille est adaptée aux ressources disponibles fonction d’une recherche de valeurs limites pour les mesures effectuées.

```
source conex.tcl
source func.tcl

set cnt 256

Space2D cmd space $casdir/data/frame/random-0.pgm
Space2D cmd space1 $casdir/data/frame/neighborhood-size.pgm
Time2D cmd time2d [space] $cnt
Simple2D cmd simple [time2d] [sum9_tab]
Dump2D cmd dump [time2d] 1
```

Les structures de données créées correspondent aux objets décrits en 2, l’objet de type “Simple” est un séquenceur.

Figure C.2: Définitions d’objets top-level

<sup>1</sup>Moore et von Neumann.

<sup>2</sup>Dont les entrées correspondent à une fonction du voisinage, la somme dans le cas des règles additives

```

# VN shift
set vnfunclist \
  [list north south east west]

# VN op
foreach func {and or xor} {
  foreach narg {4 5} {
    lappend vnfunclist $func$narg
  }
}

foreach func $vnfunclist {
  Func cmd VN_$func $VN_1 VN_$func
  VN_$func rbind
  lappend func_list VN_$func
}

# Moore op
foreach func {and or xor} {
  foreach narg {4 5 8 9} {
    lappend moorefunclist $func$narg
  }
}

foreach func $moorefunclist {
  Func cmd Moore_$func $Moore_1 \
    Moore_$func
  Moore_$func rbind
  lappend func_list Moore_$func
}

# SumTab
Func cmd sum4_tab $VN_1 VN_sum4_tab \
  sum4_tab_parse
Func cmd sum5_tab $VN_1 VN_sum5_tab \
  sum5_tab_parse
Func cmd sum8_tab \
  [ConexSpec $moore_1 $local_1 moore_1] \
  Moore_sum8_tab sum8_tab_parse
Func cmd sum9_tab \
  [ConexSpec $moore_1 $local_1 moore_1] \
  Moore_sum9_tab sum9_tab_parse

linsert func_list 0 sum4_tab sum5_tab \
  sum8_tab sum9_tab

set casdir $env(CASDIR)

# Cellsim tbl
Func cmd tunnel \
  [ConexSpec $vn_3 $local_3 vn_3] \
  -f tunnel.pgm
Func cmd brain \
  [ConexSpec $moore_2 $local_2 moore_2] \
  -f $casdir/data/cellsim/Rules/brain.pgm

linsert func_list 0 tunnel brain

set life      "00U100000"
set ink8      "UUU1UUUUU"
set maj9      "0000011111"
set anneal9   "0000101111"

sum8_tab rbind $life
sum9_tab rbind $anneal9

```

Les règles additives *lnk*, *maj* et *anneal* sont décrites dans Toffoli et Margolus (1987).

Figure C.3: *Définitions des fonctions*

# Annexe D

## Outils logiciels

1. Cellsim - CA Simulator  
Cellsim copyright 1989, 1990 by Chris Langton and Dave Hiebeler (c-gl@lanl.gov, hiebeler@heretic.lanl.gov)  
Version 2.5 of Cellsim is a SunView-based cellular automata simulator allowing interactive specification, editing, running, and analysis of 1- and 2-D CA's. It will run on Sun-3's, -4's, and Sparc stations, color or B&W.
  2. Scamper - a simulator of Cellular automata.  
Copyright 1991 by Ian Palmer of Imperial College of Science, Technology and Medicine, University of London.
  3. CA LAB  
Copyright 1989 Rudy Rucker and Autodesk. Inc.
  4. Xlife - Conway's Game of Life for X, version 3.0  
XLife Copyright 1989 Jon Bennett  
jb7m+@andrew.cmu.edu, jcrb@cs.cmu.edu
  5. HODGE-C - A C implementation of Gerhard & Schuster's hodge-podge machine  
Copyright (C) 1993 Joerg Heitkoetter
  6. Cellang  
Copyright (C) 1992 J Dana Eckart  
cellc - Cellang 2.0 (cellular automata) compiler  
cellview - cellular automata viewer  
pe-scam - Cellang 2.0 (cellular automata) machine driver
-

7. pbmplus - Extended Portable Bitmap Toolkit  
Copyright (C) 1989, 1991 by Jef Poskanzer.
  8. groff - document formatting system  
J. Clark. Copyright (C) 1989, 1990, 1991, 1992 Free Software Foundation, Inc.  
From the README:  
Included in this release are implementations of troff, pic, eqn, tbl, refer, the -man macros and the -ms macros, and drivers for PostScript, TeX dvi format, and typewriter-like devices. Also included is a modified version of the Berkeley -me macros, an enhanced version of the X11 xditview previewer, and an implementation of the -mm macros contributed by Joergen Haegg.
  9. ghostscript - PostScript previewer  
Copyright (C) 1989, 1992 Aladdin Enterprises. Distributed by Free Software Foundation, Inc.
  10. xv - interactive image display for the X Window System  
Copyright 1989, 1990, 1991, 1992 by John Bradley and the University of Pennsylvania
  11. gnuplot - an interactive plotting program  
Copyright(C) 1986, 1987, 1990, 1991, 1992 Thomas Williams, Colin Kelley
  12. Khoros - an integrated software development environment for information processing and visualization.  
Khoros Consortium. Khoros is a Registered Trademark of The University of New Mexico  
From the README:  
Khoros components include a visual programming language, code generators for extending the visual language and adding new application packages to the system, an interactive user interface editor, interactive image display programs, surface visualization, an extensive library of image processing, numerical analysis and signal processing routines, and 2D/3D plotting packages. X applications built using the Khoros User Interface Libraries have built in journal/playback and groupware capabilities.
  13. cantata - visual programming environment for accessing the Khoros library of routines
-

14. qrt - Ray Tracer  
Copyright 1988 and 1989 Steve Koren
  15. vart - 3D object toolbox for Khoros  
Copyright 1992, Peter Averkamp. All rights reserved.
  16. pgmtexture - calculate textural features on a portable graymap  
J. D. McCauley. Copyright (C) 1991 by Texas Agricultural Experiment Station
  17. gnans - A program for Stochastic and Deterministic Dynamical Systems  
B. Martensson. Copyright (C) 1989, 1991 Free Software Foundation, Inc.
  18. Tcl - tool command language facilities  
J. Ousterhout. Copyright 1987-1991 Regents of the University of California
  19. Tk - an X11 toolkit that provides the Motif look and feel and is implemented using the Tcl command language.  
J. Ousterhout. Copyright 1987-1991 Regents of the University of California
  20. XDataSlice  
National Center for Supercomputing Applications, September 1989  
NCSA X DataSlice Version 1.0 source code and documentation are in the public domain.  
From documentation:  
NCSA X DataSlice is a color imaging and data analysis tool based on the X Window System. It was developed for analysis of three-dimensional 32-bit floating-point scientific data stored in NCSA HDF file format.
  21. IsoVis  
National Center for Supercomputing Applications, July 1990  
NCSA Isosurface Visualizer Version 1.0 source code and documentation are in the public domain.  
From documentation:  
Isosurface Visualizer, a 3D visualization tool, allows users to view data in a simple manner. Input data consists of a single 3D scalar array of floating-point values. The indices of this array form the x, y, z coordinates, and the variable at each node determines where the isosurface
-

passes through the data. You can think of an isosurface as a three-dimensional contour plot, although this is not what it actually is.

22. HDF

National Center for Supercomputing Applications, November 8, 1993  
NCSA HDF Version 3.3 source code and documentation are in the public domain.

From documentation:

The Hierarchical Data Format (HDF) was designed to be an easy, straight-forward, and self-describing means of sharing scientific data among people, projects, and types of computers. An extensible header and carefully crafted internal layers provide a system that can grow as scientific data-handling needs evolve.

23. VIS-5D VERSION 3.3

Space Science and Engineering Center, University of Wisconsin - Madison, 1225 West Dayton Street, Madison, WI 53706

From documentation:

VIS-5D is a software system for visualizing data made by numerical weather models and similar sources. VIS-5D works on data in the form of a five-dimensional rectangle. That is, the data are real numbers at each point of a "grid" or "lattice" which spans three space dimensions, one time dimension and a dimension for enumerating multiple physical variables.

24. viewit v3.13

NMR Imaging and Spectroscopy, April 20, 1993

C.D.Gregory Biomedical Magnetic Resonance Laboratory College of Medicine University of Illinois at Urbana-Champaign Urbana Illinois

From the README:

Perhaps, the most useful description of viewit is a "scientific calculator" for multidimensional arrays with extra "buttons" for signal processing, scientific visualization, image processing and multivariate analysis.

25. LIC - Line Integral Convolution

Brian Cabral and Casey Leedom. Copyright (c) 1993 The Regents of the University of California. All rights reserved.

26. Octave - a high-level language for numerical computations.

Copyright (C) 1992, 1993, 1994 John W. Eaton

27. RLaB - matrix oriented, interactive programming environment.

Copyright (C) 1992 Ian R. Searle

---

# Liste des figures

|      |                                                                                                             |    |
|------|-------------------------------------------------------------------------------------------------------------|----|
| 1.1  | Une interface humaine expérimentale pour le Workbench . . . .                                               | 7  |
| 1.2  | Un glider superluminique . . . . .                                                                          | 10 |
| 1.3  | Tranches d'espace-temps et pyramides relativistes . . . . .                                                 | 11 |
| 1.4  | Une séquence de tranches d'espace-temps . . . . .                                                           | 12 |
| 1.5  | Projection relativiste fixe . . . . .                                                                       | 13 |
| 1.6  | Projection relativiste mobile . . . . .                                                                     | 14 |
| 1.7  | Projection relativiste sur les 3 axes d'un cube d'espace-temps .                                            | 15 |
| 1.8  | Transparence globale et par plan de bits . . . . .                                                          | 16 |
| 1.9  | Une séquence de tranches d'espace-temps de la règle <b>Anneal</b> . .                                       | 17 |
| 1.10 | Faces du cube d'espace-temps de <b>Qmean</b> . . . . .                                                      | 18 |
| 1.11 | Une transparence éclatée par plan de bits . . . . .                                                         | 19 |
| 1.12 | Une transparence utilisant tout l'espace de valeur d'une cellule                                            | 20 |
| 1.13 | Une transparence par plan de bits pour <b>Anneal</b> . . . . .                                              | 21 |
| 1.14 | Une transparence par plan de bits pour <b>Life</b> . . . . .                                                | 22 |
| 1.15 | Une détection de contour appliquée sur une projection relativiste                                           | 23 |
| 1.16 | Une superposition de réduction intégrale . . . . .                                                          | 31 |
| 1.17 | Une autre colormap pour la figure 1.16 . . . . .                                                            | 32 |
| 2.1  | Variété des configurations initiales . . . . .                                                              | 37 |
| 2.2  | Transformation d'espace . . . . .                                                                           | 38 |
| 2.3  | Visualisations par bitmap: d'un espace, . . . . .                                                           | 39 |
| 2.4  | et de plusieurs séquences d'espaces. . . . .                                                                | 41 |
| 2.5  | Visualisations d'espaces par un pixmap . . . . .                                                            | 42 |
| 2.6  | ou par plusieurs bitmaps . . . . .                                                                          | 43 |
| 2.7  | Visualisation par lancer de rayons (ray tracing) . . . . .                                                  | 44 |
| 2.8  | Visualisation par films de vues ray-tracées . . . . .                                                       | 45 |
| 2.9  | Visualisation par surface d'élévation d'une réduction intégrale<br>du temps . . . . .                       | 50 |
| 2.10 | Visualisation par pixmaps des données de la figure 2.9 . . . . .                                            | 51 |
| 2.11 | Visualisation par bitmaps de séquences de réductions dif-<br>férentielles de la règle <b>Life</b> . . . . . | 52 |

---

|      |                                                                                                                                   |    |
|------|-----------------------------------------------------------------------------------------------------------------------------------|----|
| 2.12 | Évolution de populations de séquences de réductions différentielles . . . . .                                                     | 53 |
| 2.13 | Une réduction intégrale du temps et une réduction différentielle d'espace . . . . .                                               | 54 |
| 2.14 | Une visualisation 3D du temps, et d'un temps de réduction différentielle d'espace . . . . .                                       | 55 |
| 2.15 | Visualisation par bitmaps de séquences de réductions différentielles de la règle <b>Anneal</b> . . . . .                          | 57 |
| 2.16 | Évolution de la population de la règle <b>Anneal</b> . . . . .                                                                    | 58 |
| 2.17 | Évolution des deux plans de population de la règle <b>Heart</b> . . . . .                                                         | 59 |
| 2.18 | Un espace produit par une table importé . . . . .                                                                                 | 62 |
| 2.19 | Deux visions du voisinage . . . . .                                                                                               | 63 |
| 2.20 | Le pipeline d'application . . . . .                                                                                               | 64 |
| 2.21 | Quelques réseaux d'interconnexion . . . . .                                                                                       | 65 |
| 2.22 | Mise à jour par table . . . . .                                                                                                   | 66 |
| 2.23 | Deux tables disjointes . . . . .                                                                                                  | 67 |
| 2.24 | Deux tables non disjointes . . . . .                                                                                              | 68 |
| 2.25 | Fonction d'application pour un voisinage spécial . . . . .                                                                        | 69 |
| 2.26 | Structure d'un PE GAPP . . . . .                                                                                                  | 70 |
| 2.27 | Schéma de la fonction d'application de la table <b>gapp</b> . . . . .                                                             | 71 |
| 2.28 | La structure de données de description de l'architecture d'un PE GAPP . . . . .                                                   | 72 |
| 2.29 | La fonction implémentant un PE GAPP . . . . .                                                                                     | 73 |
| 2.30 | La fabrication de la table de transition de l'AC <i>PE GAPP</i> . . . . .                                                         | 74 |
| 2.31 | La structure de données de la partie déclarative de la description de l'architecture d'un PE GAPP . . . . .                       | 75 |
| 2.32 | Les initialisation de la structure de données de la partie déclarative de la description de l'architecture d'un PE GAPP . . . . . | 76 |
| 2.33 | Initialisation de l'ALU d'un PE GAPP . . . . .                                                                                    | 77 |
| 2.34 | Le DFA de <b>Life</b> . . . . .                                                                                                   | 78 |
| 2.35 | Une partie de l'arbre de décodage de <b>life</b> . . . . .                                                                        | 79 |
| 2.36 | Une session avec <b>Cantata</b> . . . . .                                                                                         | 83 |
| 2.37 | Les panneaux de contrôle de nos modules <b>Cantata</b> . . . . .                                                                  | 84 |
| 2.38 | Définitions de procédures de séquencements . . . . .                                                                              | 85 |
| 3.1  | Les 128 première valeur de $\lambda$ et Flip, pour un calcul sur les 16384 premiers entier . . . . .                              | 90 |
| 3.2  | Les 256 premières valeurs de la dérivée de $\lambda$ et Flip . . . . .                                                            | 91 |
| 3.3  | Les 512 premières valeurs des fonctions $\lambda$ et Flip, et de leur probabilité d'apparition . . . . .                          | 92 |
| 3.4  | Plot de $\lambda$ et Flip vs. leur probabilité d'apparition . . . . .                                                             | 93 |

|      |                                                                                                       |     |
|------|-------------------------------------------------------------------------------------------------------|-----|
| 3.5  | L'histogramme des distribution de $\lambda$ et Flip, et $\lambda$ vs. Flip. . . . .                   | 93  |
| 3.6  | Un tri des valeurs de la fonction et leurs probabilités d'apparition pour $\lambda$ et Flip . . . . . | 94  |
| 3.7  | Valeurs de $\lambda$ et EN . . . . .                                                                  | 95  |
| 4.1  | Une vue 3D de la dynamique de la règle <b>Anneal</b> . . . . .                                        | 100 |
| 4.2  | Une signature cellulaire . . . . .                                                                    | 101 |
| 4.3  | Une configuration typique de la règle <b>Anneal</b> . . . . .                                         | 102 |
| 4.4  | Une séquence typique de la règle <b>Anneal</b> . . . . .                                              | 103 |
| 4.5  | Deux configurations de la règle <b>anneal</b> . . . . .                                               | 104 |
| 4.6  | Les deux configurations de la figure 4.5 non retouchée . . . . .                                      | 105 |
| 4.7  | Évolution de la taille des frontières de <b>anneal</b> . . . . .                                      | 106 |
| 4.8  | Évolution de la population de <b>Anneal</b> lors d'un changement du nombre d'îles . . . . .           | 107 |
| 4.9  | Transition vers la dernière île . . . . .                                                             | 108 |
| 4.10 | Populations normalisées pour 4 espaces initiaux de tailles croissantes . . . . .                      | 108 |
| 4.11 | Une série de zooms sur la population et la taille des frontières . . . . .                            | 109 |
| 5.1  | La dynamique de l'AC xu . . . . .                                                                     | 111 |
| 5.2  | Un rendu d'une tranche des valeurs du dernier plan de la figure 5.1 . . . . .                         | 114 |
| 5.3  | D'autres points de vue . . . . .                                                                      | 115 |
| 5.4  | Et un épluchage . . . . .                                                                             | 116 |
| 5.5  | Une autre configuration initiale . . . . .                                                            | 117 |
| 5.6  | Et un épluchage exponentiel . . . . .                                                                 | 118 |
| A.1  | Le cœur de la règle <b>Qmean</b> . . . . .                                                            | 119 |
| A.2  | La règle <b>Anneal</b> est implémentée via la règle <b>Sum9</b> . . . . .                             | 120 |
| A.3  | L'interface de binding Tc1 pour <b>Anneal</b> . . . . .                                               | 120 |
| A.4  | La règle <b>Tube-Worm</b> . . . . .                                                                   | 121 |
| B.1  | <b>gnans</b> et Lokta-Volterra . . . . .                                                              | 123 |
| B.2  | <b>Fox-Rabbit</b> Version 1 . . . . .                                                                 | 125 |
| B.3  | <b>Fox-Rabbit</b> Versions 2 et 3 . . . . .                                                           | 126 |
| C.1  | Création des structures de données pour le voisinage . . . . .                                        | 128 |
| C.2  | Définitions d'objets top-level . . . . .                                                              | 129 |
| C.3  | Définitions des fonctions . . . . .                                                                   | 130 |

---

# Liste des tableaux

|      |                                                                                                        |    |
|------|--------------------------------------------------------------------------------------------------------|----|
| 2.1  | Nombres d'états pour quelques cubes d'interconnexions, . . . .                                         | 34 |
| 2.2  | quelques croix d'interconnexions, . . . . .                                                            | 35 |
| 2.3  | et quelques sphères d'interconnexions. . . . .                                                         | 36 |
| 2.4  | Taille d'espaces fonction de la dimension et de l'arête . . . . .                                      | 46 |
| 2.5  | Temps de calcul sur ces espaces de 1000 générations à 100 KCPS, 47                                     |    |
| 2.6  | et de 1000 générations à 10 MCPS. . . . .                                                              | 47 |
| 2.7  | Réductions statistiques sur un espace obtenu par réduction<br>intégrale du temps, . . . . .            | 56 |
| 2.8  | et par réduction intégrale d'une séquence temporelle de réduction<br>différentielle d'espaces. . . . . | 56 |
| 2.9  | Compilation d'un table pour le GAPP, . . . . .                                                         | 80 |
| 2.10 | et une optimisation immédiate. . . . .                                                                 | 81 |

# Bibliographie

1. Almási, J., Kovács, I., Medvey, M., Simon, E., et Székely, Z. (1987). “On The Implementation of The Cellas 2.1 System”. in Legendi et al. (1987), pages 83–89.
  2. Anderson, P. W. (1987). “Spin Glass Hamiltonians: A Bridge Between Biology, Statistical Mechanics and Computer Science”. in Pines (1988), pages 17–20.
  3. Atlan, H. (1979). *Entre le cristal et la fumée: Essai sur l'organisation du vivant*. Éditions du Seuil, Paris.
  4. Bak, P. (1993). “Self-Organized Criticality and Gaia”. in Stein et Varela (1993), pages 255–268.
  5. Bergé, P., Pommeau, Y., et Dubois-Gange, M. (1984). *Des rythmes au chaos*. Édition Odile Jacob, Paris.
  6. Bergson, H. (1989). *L'évolution créatrice*. Quadrige/Presse Universitaire de France, Paris, quatrième édition. Première édition: 1941.
  7. Berlekamp, E. R., Conway, J. H., et Guy, R. K. (1982). *Winning Ways for your Mathematical Plays*, tome 2. Academic Press, New York.
  8. Bolotski, M., DeHon, A., et Knight Jr., T. F. (1994). “Unifying FPGAs and SIMD Arrays”. Transit Note 95, MIT Artificial Intelligence Laboratory. Original Issue: September 1993.  
`ftp://transit.ai.mit.edu/transit-notes/tn95.ps.Z`.
  9. Braunl, T. (1989). *Structured SIMD Programming in Parallaxis*. Springer-Verlag.
  10. Cabral, B. et Leedom, L. C. (1993). “Imaging Vector Fields Using Line Integral Convolution”. in *SIGGRAPH '93*.
-

11. Casti, J. L. (1989). "Pattern and the Emergence of Living Forms: Cellular Automata and Discrete Dynamics". in *Alternate Realities: Mathematical Models of Nature and Man*, chapitre 2, pages 45–111. John Wiley & Sons, New York.
  12. Chaté, H. et Manneville, P. (1990). "Criticality in Cellular Automata". *Physica D*, 45, 122–135.
  13. Cheng, D. Y. (1993). "A Survey of Parallel Programming Languages and Tools". Report RND-93-005, National Aeronautics and Space Administration, Ames Research Center, Moffett Field, CA.
  14. Cherif, A. A. (1993). "Collective Behaviour for a Micro-colony of Robots". in Deneubourg et al. (1993), pages 1–8.
  15. Choquet, G. (1992). "Le continu, le discret, et ... tout le reste". in Salanskis et Sinaceur (1992), pages 267–278.
  16. Clouqueur, A. et d'Humièrre, D. (1987). "RAP1, A Cellular Automata Machine for Fluid Dynamics". *Complex Systems*, 1, 585–597.
  17. Cordy, J. R., Halpern-Hamu, C. D., et Promislow, E. M. (1991). "TXL: A Rapid Prototyping System for Programming Language Dialects". *Computer Languages*, 16(1), 97–107.
  18. Cypher, R. et Sanz, J. L. C. (1994). *The SIMD Model of Parallel Computation*. Springer-Verlag, New York.
  19. Davies, P. (1989). *The Cosmic Blueprint*. Unwin, London. First published by William Heineman 1987.
  20. de Boer, R. J., van der Laan, J. D., et Hogeweg, P. (1993). "Randomness and Pattern Scale in the Immune Network: A Cellular Automata Approach". in Stein et Varela (1993), pages 231–252.
  21. de Gennes, P.-G. (1989). "La matière ultradivisée". in Pour La Science (1989), chapitre 12, pages 193–203.
  22. de Latil, P. (1991). "Systèmes physiques et systèmes biologiques". in F. Tinsland, éditeur, *Systèmes Naturels Systèmes Artificiels*, collection milieux, pages 33–45. Champ Vallon, Seyssel.
  23. Demongeot, J., Golès, E., et Tchuente, M., éditeurs (1985). *Dynamical Systems and Cellular Automata*. Academic Press, London. This volume constitutes the proceedings of the "Journées de la Société Mathématique de France" on
-

- “Dynamical Behaviour of Cellular Automata: Theory and Applications,” held September 13–17, 1983, at Lumigny, France, in the Centre International de Rencontres Mathématiques (CIRM).
24. Deneubourg, J. L., Goss, S., Nicolis, G., Bersini, H., et Dagonnier, R., éditeurs (1993). *ECAL 93*. Université Libre de Bruxelles.
  25. Despain, A., Max, C. E., et Hasslacher, B. (1990). “Prospect for a Lattice-Gas Computer”. in Doolen et al. (1990), pages 211–218.
  26. Dewdney, A. K. (1988). “The Hodgepodge Machine Makes Waves”. *Scientific American*, 225(8), 104–107.
  27. Doolen, G. D., Frish, U., Hasslacher, B., Orszag, S., et Wolfram, S., éditeurs (1990). *Lattice Gas Methods for Partial Differential Equations*, tome IV de *Santa Fe Institute Studies in the Sciences of Complexity, Proc. Vol.* Addison–Wesley Publishing Company, Redwood City, CA. A volume of lattice gas reprints and articles, including selected papers from the workshop on large nonlinear systems, held august, 1987 in Los Alamos, New Mexico.
  28. Durand, B. (1994). *Automates cellulaires: Réversibilité et complexité*. Thèse de Doctorat, École normale supérieure de Lyon, Université Claude Bernard-Lyon 1.
  29. Eckart, J. D. (1992). *A Cellular Automata Simulation System: Version 2.0*. Radford University.
  30. Eckart, J. D. (1992). *Cellang 2.0: Language Reference Manual*. Radford University.
  31. Fogelman-Soulié, F., Havelange, V., et Milgram, M., éditeurs (1991). *Les théories de la complexité: autour de l'œuvre d'Henri Atlan*. La Couleur des idées. Éditions du Seuil, Paris. Colloque de Cerisy juin 1984.
  32. Fox, G. C. (1994). “Parallel Computers and Complex Systems”. *Complexity International*, 1. ISSN 1320-0682.  
<http://life.anu.edu.au/ci/vol1/Geoffrey.Fox/paper.html> .
  33. Fredkin, E. (1990). “Digital Mechanics: An Informational Process Based on Reversible Universal Cellular Automata”. *Physica D*, 45, 254–270.
  34. Giloi, W. K. (1987). “Interconnection Networks for Massively Parallel Computer Systems”. in Treleaven (1987), chapitre 10, pages 312–384.
  35. Gombrich (1971). *L'art et l'illusion*. NRF.
-

36. Gosper, W. (1984). "Exploiting Regularities in Large Cellular Spaces". *Physica D*, 10, 75–80.
  37. Gouyet, J. F. (1992). *Physique et structures fractales*. Masson, Paris.
  38. Green, D. G. (1994). "Emergent Behaviour in Biological Systems". *Complexity International*, 1. ISSN 1320-0682.  
<http://life.anu.edu.au/ci/vol1/David.Green/paper.html> .
  39. Greussay, P. (1985). "Programmation des Méga-Processeurs: Du GAPP à la Connection Machine". Notes de cours de D.E.A d'intelligence artificielle, Université Paris-8, Département d'Informatique.
  40. Greussay, P. (1988). "L'Ordinateur Cellulaire". *La Recherche*, 204, 1320–1330.
  41. Gutowitz, H. A. (1990). "Introduction (To Cellular Automata)". *Physica D*, 45, vii–xiv.
  42. Gutowitz, H. A. (1990). "A Hierarchical Classification of Cellular Automata". *Physica D*, 45, 136–156.
  43. Hameroff, S., Rasmussen, S., et Mansson, B. (1989). "Molecular Automata in Microtubules: Basic Computational Logic of the Living State?". in Langton (1989), pages 521–555.
  44. Hantcherlian, G. et Hervio, D. (1972). "Automates Cellulaires". Rap. tech., Université de Grenoble.
  45. Harthong, J. (1992). "Le continu ou le discret, un problème indécidable". in Salanskis et Sinaceur (1992), pages 350–365.
  46. Hasselbring, W. (1992). "Programming Cellular Automata for Image Processing". Presented at the BCS-PPSG Meeting on Cellular Automata, Imperial College, London, UK.
  47. Hasslacher, B. (1993). "Parallel Billiards and Monster Systems". in Metropolis et Rota (1993), pages 53–65. Originally appeared in the Winter 1992 issue of *Dædalus*, volume 121, number 1, from the proceedings of the American Academy of Arts and Sciences.
  48. Heckbert, P. (1990). "Digital Line Drawing". in A. S. Glassner, éditeur, *Graphics Gems*, page 99. Academic Press, Inc., San Diego, CA.
-

- 
49. Hibbard, W. L., Paul, B. E., Santek, D. A., Dyer, C. R., Battaiola, A. L., et Voidrot-Martinez, M.-F. (1994). "Interactive Visualisation of Earth and Space Science Computations". *IEEE Trans. Computers*, 27(7), 65–72.
  50. Hiebeler, D. (1990). "A brief review of CA packages". *Physica D*, 45, 463.
  51. Hiebeler, D. (1990). "Cellular Automata and Discrete Physics". in L. Lam, éditeur, *Introduction to Nonlinear Physics*, chapitre 14. Springer-Verlag, Berlin. To appear. <http://www.santafe.edu/~hiebele>.
  52. Hiebeler, D. E. (1993). "Implication of Creation". *Idealistic Studies*, 23(1). Special issue on philosophy and computers.
  53. Hillis, W. D. (1986). *The Connection Machine*. Distinguished Dissertations. The M.I.T. Press, Cambridge, Massachusetts.
  54. Hockney, R. W. et Jesshope, C. R. (1986). *Parallel Computers: Architecture Programing and Algorithms*. Adam Hilger Ltd.
  55. Holland, J. H. (1993). "Complex Adaptative Systems". in Metropolis et Rota (1993), pages 17–30. Originally appeared in the Winter 1992 issue of *Dædalus*, volume 121, number 1, from the proceedings of the American Academy of Arts and Sciences.
  56. Hord, R. M. (1990). *Parallel Supercomputing in SIMD Architectures*. CRC Press, Boca Raton.
  57. Huberman, B. A. (1987). "Computing With Attractors: From Self-Repairing Computers, to Ultradiffusion, and the Application of Dynamical Systems to Human Behavior". in Pines (1988), pages 167–174.
  58. Jackson, E. A. (1991). "Coupled Maps and Cellular Automata". in *Perspectives of Nonlinear Dynamics*, tome 2, pages 427–504. Cambridge University Press, Cambridge. First published 1990.
  59. Jen, E. (1990). "Aperiodicity in One-Dimensional Cellular Automata". *Physica D*, 45, 3–18.
  60. Kandinsky (1991). *Point et ligne sur plan: Contribution à une analyse de la peinture*. Folio/essais. Gallimard, Paris. Titre original *PUNKT UND LINIE ZU FLÄCHE: Beitrag zur Analyse der malerischen Elemente*, 1926. Nouvelle édition établie, présentée et annotée par Philippe Sers. Traduit de l'allemand par Suzanne et Jean Leppien.
-

61. Kayama, Y., Anada, H., et Imamura, Y. (1993). "Characteristic Representation of Elementary Cellular Automata". Rap. tech. BAIKA-93-01, BAIKA Women's College, Japan. `gopher://xyz.lanl.gov:151/77/?kayama`.
  62. Keller, P. R. et Keller, M. M. (1993). *Visual Cues*. IEEE Computer Society Press and IEEE Press, Los Alamitos, CA.
  63. Kung, S. Y. (1988). *VLSI Array Processors*. Prentice Hall Information and System Sciences Series. Prentice Hall, Englewood Cliffs, NJ.
  64. Langton, C. G., éditeur (1989). *Artificial Life*, tome VI de *Santa Fe Institute Studies in the Sciences of Complexity, Proc. Vol.* Addison-Wesley Publishing Company, Redwood City, CA. The Proceedings of An Interdisciplinary Workshop on the Synthesis and Simulation of Living Systems Held September, 1987, in Los Alamos, New Mexico.
  65. Langton, C. G. (1990). "Computation at the edge of chaos: phase transition and emergent computation". *Physica D*, 42, 12–37.
  66. Langton, C. G. (1992). "Life at the Edge of Chaos". in C. G. Langton, C. Taylor, J. D. Farmer, et S. Rasmussen, éditeurs, *Artificial Life II*, tome X de *Santa Fe Institute Studies in the Sciences of Complexity, Proc. Vol.*, pages 41–91. Addison-Wesley Publishing Company, Redwood City, CA.
  67. Le Bellac, M. (1992). "Invariance d'échelle, universalité et limite statistique continue en physique". in Salanskis et Sinaceur (1992), pages 296–307.
  68. Legendi, T. (1987). "Cellware Implementation". in Legendi et al. (1987), pages 61–65.
  69. Legendi, T., Parkinson, D., Vollmar, R., et Wolf, G., éditeurs (1987). *Parallel Processing by Cellular Automata and Arrays*. North-Holland. Proceeding of the Third International Workshop on Parallel Processing by Cellular Automata and Arrays Berlin, GDR, 9-11 September, 1986.
  70. Lévy, P. (1987). *La machine univers: Création, cognition et culture informatique*. Edition de la découverte.
  71. Lewin, R. (1992). *Complexity: Life at the Edge of Chaos*. MacMillan Publishing Company, New York.
  72. Li, W., Packard, N. H., et Langton, C. G. (1990). "Transition Phenomena in Cellular Automata Rule Space". *Physica D*, 45, 77–94.
-

- 
73. Li, W. et Packard, N. (1990). "Structure of the elementary cellular automata rule space". *Complex System*, 4(3), 281–297.
  74. Li, W. (1988). "Pretty Picture Generated by Two-state Five-neighbor Cellular Automata". Rap. tech. CCSR-88-15, Center for Complex Systems Research, University of Illinois at Urbana-Champaign.
  75. Li, W. (1989). "Problems in Complex Systems". Rap. tech. CCSR-89-9, Center for Complex Systems Research, University of Illinois at Urbana-Champaign.
  76. Lindgren, K. et Nordahl, M. G. (1990). "Universal Computation in Simple One-Dimensional Cellular Automata". *Complex System*, 4(3), 299–318.
  77. Lindgren, K. et Nordahl, M. G. (1993). "Evolutionary Dynamics of Spatial Games". in Deneubourg et al. (1993), pages 604–615.
  78. Mandelbrot, B. (1984). *Les objets fractals: Forme, hasard et dimension*. Nouvelle Bibliothèque Scientifique. Flammarion, deuxième édition. Première édition 1975.
  79. Margolus, N. et Toffoli, T. (1990). "Cellular Automata Machine". in Doolen et al. (1990), pages 219–249.
  80. Margolus, N. (1984). "Physic-Like Models of Computation". *Physica D*, 10, 81–95.
  81. McIntosh, H. V. (1990). "Wolfram's class IV Automata and a Good Life". *Physica D*, 45, 105–121.
  82. Méhat, J. (1989). *Contrôle et programmation parallèles des architectures pyramidales*. Thèse de Doctorat, Université Paris-8, Département d'Informatique.
  83. Mérigot, A. (1992). "Associative Nets: A New Parallel Computing Model". Rap. tech. IEF 92-02, Institut d'Electronique Fondamentale, CNRS URA 022, Université Paris Sud.
  84. Metropolis, N. et Rota, G.-C., éditeurs (1993). *A New Area in Computation*. The M.I.T. Press, Cambridge, Massachusetts.
  85. Metropolis, N. (1993). "The Age of Computing: A Personal Memoir". in Metropolis et Rota (1993), pages 119–130. Originally appeared in the Winter 1992 issue of *Dædalus*, volume 121, number 1, from the proceedings of the American Academy of Arts and Sciences.
-

86. Milgram, M. (1991). "Complexité et hasard". in Fogelman-Soulie et al. (1991), chapitre 5, pages 93–99.
  87. Mitchell, M., Hraber, P. T., et Crutchfield, J. P. (1993). "Revisiting the Edge of Chaos: Evolving Cellular Automata to Perform Computations". Rap. tech. SFI-93-03-014, Santa Fe Institute, Santa Fe, NM.  
To appear in *Complex Systems*.  
`ftp://ftp.santafe.edu/pub/Users/mm/rev-edge-part[1-4].ps.Z`.
  88. Murray, J. D. (1990). *Mathematical Biology*, tome 19 de *Biomathematics Texts*. Springer-Verlag, Berlin.
  89. NCR (1985). "Processeur GAPP: Geometric Arithmetic Parallel Processor". NCR Corporation, Dayton, Ohio.
  90. Ousterhout, J. K. (1990). "Tcl: An Embeddable Command Language". in *1990 Winter USENIX conference proceedings*. In Tcl/Tk distribution.
  91. Ousterhout, J. K. (1991). "An X11 Toolkit Based on the Tcl Language". in *1991 Winter USENIX conference proceedings*. In Tcl/Tk distribution.
  92. Packard, N. H. (1988). "Adaptation toward the edge of chaos". in J. A. S. Kelso, A. J. Mandell, et M. F. Shlesinger, éditeurs, *Dynamic Patterns in Complex Systems*, pages 293–301. World Scientific, Singapore.
  93. Penrose, R. (1989). *The Emperor's New Mind: Concerning Computers, Minds, and The Laws of Physics*. Oxford University Press, New York.
  94. Pickover, C. A. (1992). "Lava Lamps in the 21st Century". in *Mazes for the Mind: Computers and the Unexpected*, chapitre 19, pages 109–115. St. Martin's Press, New York.
  95. Pike, R., Guibas, L., et Ingalls, D. (1984). "Bitmap Graphics". SIGGRAPH '84 Course Notes.
  96. Pines, D., éditeur (1988). *Emerging Syntheses in Sciences: Proceeding of the Founding Workshops of the Santa Fe Institute*, tome I de *Santa Fe Institute Studies in the Sciences of Complexity, Proc. Vol.* Addison-Wesley Publishing Company, Redwood City, CA. The two workshops took place on October 5-6 and November 10-11, 1984.
  97. Poundstone, W. (1987). *The Recursive Universe: Cosmic Complexity and the Limits of Scientific Knowledge*. Oxford University Press, Oxford. First published 1985 by William Morrow and Company, Inc., New York. Computer Consultation by Robert T. Wainwright.
-

98. Pour La Science, éditeur (1989). *L'ordre du chaos*. Bibliothèque pour la science. Pour La Science/Diffusion Belin, Paris.
  99. Preston, J. K. et Duff, M. J. B. (1984). *Modern Cellular Automata: Theory and Applications*. Plenum Press, New York.
  100. Quéau, P. (1986). *Eloge de la simulation: De la vie des langages à la synthèse des images*. collection milieux. Champ Vallon/INA, Seyssel.
  101. Quéau, P. (1989). *Metaxu: Théorie de l'art intermédiaire*. collection milieux. Champ Vallon/INA, Seyssel.
  102. Quinton, P. (1987). "An Introduction To Systolic Architecture". in Treleaven (1987), chapitre 13, pages 387–399.
  103. Reggia, A., Armentrout, S. L., Chou, H.-H., et Peng, Y. (1993). "Simple Systems That Exhibit Self-Directed Replication". *Science*, 259, 1282–1287.
  104. Richards, F. C., Meyer, T. P., et Packard, N. H. (1990). "Extracting Cellular Automata Rules Directly From Experimental Data". *Physica D*, 45, 189–202.
  105. Rossi, F. (1993). *Automates cellulaires et croissance des cristaux de neige*. Mémoire de Maitrise, Université Paris-8, Département d'Informatique.
  106. Rozin, M.-N. (1987). *Etude pratique des ressources de la programmation plane*. Mémoire de Maitrise, Université Paris-8, Département d'Informatique.
  107. Rozin, M.-N. (1988). *De l'Etude des Caractéristiques du Milieu Parallle Synchrone la Programmation Plane*. Mémoire de D.E.A, Université Paris-8, Département d'Informatique.
  108. Rozin, M.-N. (1993). *Étude géométrique et cinématique de la programmations des très grandes grilles de processeurs synchrones*. Thèse de Doctorat, Université Paris-8, Département d'Informatique.
  109. Rucker, R. (1987). *Mind Tools: The Mathematics of Information*. Mathematics Science. Penguin Books, London. First published in the USA by Houghton Mifflin 1987.
  110. Rucker, R. (1989). *CA Lab: Exploring Cellular Automata*. Autodesk, Inc. Documentation of the CA Lab package.
-

111. Ruelle, D. (1989). “Determinisme et prédictibilité”. in *Pour La Science* (1989), chapitre 4, pages 52–63.
  112. Ruelle, D. (1991). *Hasard et chaos*. Points. Édition Odile Jacob, Paris.
  113. Salanskis, J.-M. et Sinaceur, H., éditeurs (1992). *Le labyrinthe du continu: Colloque de Cerisy*. Springer–Verlag France, Paris.
  114. Schroeder, M. (1990). “Cellular Automata”. in *Fractals, Chaos, Power Laws: Minutes from an Infinite Paradise*, chapitre 17, pages 371–390. W. H. Freeman and Company, New York.
  115. Sigmund, K. (1993). *Games of Life: Explorations in Ecology, Evolution, and Behaviour*. Oxford University Press, Oxford.
  116. Signorini, J. (1994). “Computing with Q2R”. in *Proceedings of the 6th Joint EPS-APS International Conference On Physics Computing*, Geneva, Switzerland. European Physical Society.
  117. Signorini, J. (1986). *Un routeur pour le chip GAPP: étude simulation et programmation*. Mémoire de D.E.A, Université Paris-8, Département d’Informatique.
  118. Signorini, J. (1992). *Programmation par configurations des ordinateurs cellulaires à très grande échelle*. Thèse de Doctorat, Université Paris-8, Département d’Informatique.
  119. Smith, M. A. (1990). “Representation of Geometrical and Topological Quantities in Cellular Automata”. *Physica D*, 45, 271–277.
  120. Sonka, M., Hlavac, V., et Boyle, R. (1993). *Image Processing, Analysis and Machine Vision*. Chapman & Hall Computing Series. Chapman & Hall, London.
  121. Stanley, H. E. et Ostrowsky, N., éditeurs (1986). *On Growth and Form: Fractal and Non-Fractal Patterns in Physics*. Martinus Nijhoff Publisher.
  122. Stein, W. et Varela, F. J., éditeurs (1993). *Thinking About Biology: An Invitation to Current Theoretical Biology*, tome III de *Santa Fe Institute Studies in the Sciences of Complexity, Lecture Notes Vol.* Addison–Wesley Publishing Company, Reading, MA.
  123. Tagonski, G. (1985). “Embedding of Discrete Dynamical Systems”. in Demongeot et al. (1985), pages 65–72.
-

- 
124. Tamayo, P. et Hartman, H. (1989). "Cellular Automata, Reaction-Diffusion Systems and the Origin of Life". in Langton (1989), pages 105–124.
  125. Thom, R. (1977). *Stabilité structurelle et morphogénèse: Essai d'une théorie générale des modèles*. InterEditions, Paris, deuxième édition.
  126. Thom, R. (1992). "L'antériorité ontologique du continu sur le discret". in Salanskis et Sinaceur (1992), pages 137–143.
  127. Toffoli, T. et Margolus, N. (1987). *Cellular Automata Machines: A New Environment for Modeling*. MIT Press series in Scientific Computation. The M.I.T. Press, Cambridge, Massachusetts.
  128. Toffoli, T. (1984). "Cellular Automata as an Alternative to (Rather than an Approximation of) Differential Equations in Modeling Physics". *Physica D*, 10, 117–127.
  129. Toulouse, G. (1991). "Verres de spin et applications de la physique aux problèmes complexes". in Fogelman-Soulie et al. (1991), chapitre 10, pages 151–170.
  130. Treleaven, M. V. P., éditeur (1987). *Future Parallel Computers*, tome 272 de *Lecture Notes in Computer Science*. Springer-Verlag, Berlin. A Advanced Course Pisa, Italy, June 1986 Proceedings.
  131. Uhr, L. (1984). *Algorithm-Structured Computer Arrays and Networks: Architectures and Processes for Images, Percept, Models, Information*. Computer Science and Applied Mathematics. Academic Press, Inc., Orlando, Florida 32887.
  132. Uhr, L. (1987). *Multi-Computer Architectures for Artificial Intelligence: Toward Fast, Robust, Parallel Systems*. John Wiley & Sons, New York.
  133. Varela, F. J. (1989). *Autonomie et connaissance: Essai sur le vivant*. La Couleur des idées. Éditions du Seuil, Paris. Titre original: Principle of Biological Autonomy, Elsevier North Holland, New York, 1980. Nouveaux chapitres: 1982, 1984, 1985, 1987.
  134. Vichniac, G. Y. (1986). "Cellular Automata Models of Disorder and Organization". in E. Bienenstock, F. Fogelman-Soulié, et G. Wiesbuch, éditeurs, *Cellular Automata Models of Disorder and Organization*, tome 20 de *NATO Advanced Science Institutes Series F: Computer and Systems Sciences*, pages 3–20, Berlin. Springer-Verlag.
-

- 
135. Vichniac, G. Y. (1990). "Boolean Derivatives on Cellular Automata". *Physica D*, 45, 63–74.
  136. von Neumann, J. (1966). *Theory of Self-Reproducing Automata*. University of Illinois Press, Urbana. Manuscrit édité et complété par A. W. Burks.
  137. Wainwright<sup>71</sup> (1971-73). "LIFELINE: A Quarterly Newsletter for enthusiast of John Conway's Game of Life". R. T. Wainwright. Issues may be obtained from Lifeline, 12 Longvue Ave., New Rochelle, N.Y. 10804.
  138. Waldrop, M. M. (1993). *Complexity: The Emerging Science at The Edge of Order and Chaos*. Viking/Penguin Group, New York. First published in the USA by Simon & Schuster 1992.
  139. Weisbuch, G. (1989). *Dynamique des systèmes complexes: Une introduction aux réseaux d'automates*. InterEditions/Editions du CNRS.
  140. Wilson, K. (1989). "Les phénomènes de physique et les échelles de longueur". in *Pour La Science* (1989), chapitre 6, pages 88–115.
  141. Wolfram, S. (1983). "Statistical mechanics of cellular automata". *Rev. Mod. Phys.*, 55(3), 601–644.
  142. Wolfram, S. (1986). *Theory and Applications of Cellular Automata: (including selected papers 1983 - 1986)*, tome 1 de *Advanced series on complex systems*. World Scientific, Singapore.
  143. Wootters, W. K. et Langton, C. G. (1990). "Is There a Sharp Phase Transition for Deterministic Cellular Automata?". *Physica D*, 45, 95–104.
  144. Wuensche, A. et Lesser, M. (1992). *The Global Dynamics of Cellular Automata*, tome I de *Santa Fe Institute Studies in the Sciences of Complexity, Ref. Vol.* Addison-Wesley Publishing Company, Reading, MA.
  145. Yoccoz, J.-C. (1992). "Idées géométrique en systèmes dynamiques". in A. D. Dalmenico, J.-L. Chabert, et K. Chemla, éditeurs, *Chaos et déterminisme*, Points Sciences, chapitre 2. Éditions du Seuil, Paris.
  146. Zsótér, A. J. (1987). "A Cellprocessor Based on 16-State Cells". in Legendi et al. (1987), pages 66–69.
-